TEACHME-LAKEHOUSE · TRAINING PROGRAMME

Module 01 · Basics — Redshift Deep Dive

01

From Node.js developer to competent Redshift engineer

45 LESSONS | COMPLETE | ~30 HOURS | NO REDSHIFT BACKGROUND ASSUMED

AUTHOR
Surender Sara

COMPANY
Northbay Solutions

ISSUED
August 2026

MODULE 01 · LESSON 01

Redshift Is Not Your App Database

Same SQL dialect, completely different machine underneath. Four instincts that serve you well in Postgres will make you slow here.

1 · WHAT IT IS

It speaks Postgres. That similarity is the trap.

psql connects. Most of your SQL runs unchanged. Then the same query that took 40ms in Postgres takes four minutes — and nothing in the error log explains why.

2 · FOUR THINGS THAT ARE GENUINELY DIFFERENT

WHAT YOU KNOW → WHAT IS TRUE HERE

ROWS → COLUMNS — `columnar storage`

Postgres keeps a row together. Redshift keeps a column together, so a query touching 3 of 200 columns reads 1.5% of the bytes.

ONE BOX → MANY SLICES — `MPP`

Your app database is one machine. Redshift splits every table across slices and runs the same plan on all of them at once.

INDEXES → PHYSICAL LAYOUT — `Lesson 14`

There is no CREATE INDEX. You choose a sort key and a distribution style instead, and you choose them once.

ROW WRITES → BULK LOADS — `Lesson 21`

One INSERT of a million rows is cheap. A million INSERTs of one row will take hours and bloat the table.

```
same SQL, opposite instincts — unlearning them IS the module
```

3 · THE SQL

```
-- the Node.js habit
INSERT INTO sales VALUES (...); x1,000,000

-- the Redshift way
COPY sales
FROM 's3://bucket/dt=2026-08-12/'
IAM_ROLE '...' FORMAT AS PARQUET;
-- one statement. minutes, not hours.
```

4 · GOTCHAS

Your ORM will connect, and then behave badly.
PRIMARY KEY does not prevent duplicates.
There is no CREATE INDEX to reach for.

unlearn first, then build

IN PLAIN ENGLISH

A van and a freight train both move boxes and both have a steering wheel. Driving the train like a van is how you end up stationary.

MODULE 01 · LESSON 01

L01 · Redshift Is Not Your App Database

Slide: `_render/L01-not-your-app-database.html`

What it is

Redshift speaks the PostgreSQL wire protocol. `psql` connects. Most of your SQL runs unchanged. Your ORM will even connect to it.

That similarity is the trap. Underneath it is a completely different machine, and the instincts that make you good at Postgres will make you slow here — silently, with no error to tell you why.

The four differences that matter

1 · Rows → columns

Postgres stores a row together on disk, because an application fetches whole rows: *give me user 4471*.

Redshift stores each **column** together, because analytics fetches few columns of very many rows: *give me the sum of net_amount for last quarter*.

A query touching 3 columns of a 200-column table reads roughly **1.5% of the bytes**. That single fact is most of the performance difference — and it is why `SELECT *` is a much worse habit here than it is in an app.

2 · One box → many slices

Your app database is one machine. Redshift splits every table across **slices** and runs the same compiled plan on all of them simultaneously (L02).

The consequence: performance depends on how *evenly* your data is spread, which is a design decision you make at `CREATE TABLE` time (L15).

3 · Indexes → physical layout

```
-- does not exist in Redshift
CREATE INDEX idx_sales_date ON sales (sale_date);
```

There is **no** `CREATE INDEX`. Instead you choose a **sort key** and a **distribution style**, once, when you create the table. L14 is the whole lesson; take from here only that reaching for an index is the reflex to lose first.

4 · Row writes → bulk loads

```
-- Node habit: fine in Postgres, fatal here
INSERT INTO sales VALUES (...);   -- × 1,000,000
```

Columnar storage rewrites blocks on every write. One `INSERT` of a million rows is cheap; a million single-row `INSERT`s takes hours and bloats the table with deleted-but-not-reclaimed space.

```
-- Redshift: one statement, minutes not hours
COPY sales
FROM 's3://bucket/dt=2026-08-12/'
IAM_ROLE 'arn:aws:iam::123456789012:role/redshift-loader'
FORMAT AS PARQUET;
```

Try it

Connect with Query Editor v2 and run these three. They tell you where you are:

L01 · Redshift Is Not Your App Database

```
SELECT current_database(), current_user, version();

-- how big are things, and how well laid out?
SELECT "schema", "table", size AS mb, tbl_rows,
       diststyle, sortkey1, skew_rows, unsorted, stats_off
FROM   svv_table_info
ORDER  BY size DESC
LIMIT  20;

-- what have I run, and how long did it take?
SELECT query_id, elapsed_time / 1000000.0 AS secs, status,
       LEFT(query_text, 80) AS sql
FROM   sys_query_history
WHERE  user_id = current_user_id
ORDER  BY start_time DESC
LIMIT  20;
```

`svv_table_info` is the single most useful view in Redshift. You will come back to it in L02, L15, L16 and L42.

The five habits to unlearn

FROM NODE + APP DB	IN REDSHIFT
`INSERT` per row	`COPY` in bulk, then `MERGE`
Add an index	Choose a **sort key** and a **distribution style**
`PRIMARY KEY` prevents duplicates	It does **not** — you test for them (L18)
Hold a connection pool	Use the **Data API** — nothing to hold (L06)
Normalise everything	**Denormalise** the reporting layer deliberately

Gotchas

- **Your ORM will connect and then behave badly.** It will issue row-at-a-time writes and `SELECT *`. Do not point Sequelize or Prisma at a warehouse.
- `SELECT *` **is expensive here** in a way it is not in Postgres — you have just asked for every column of a columnar store.
- **The first run of a query is slow** because Redshift compiles it (L05). Never benchmark a first run.

Checklist

- ☐ I can explain columnar storage to someone who only knows row storage
- ☐ I know there is no `CREATE INDEX` and what replaces it
- ☐ I would never write a row-at-a-time insert loop against Redshift
- ☐ I have run the three queries above against a real cluster
- ☐ I know what `svv_table_info` is for

You've got it when you can…

…explain to another Node developer why the pattern they are about to write — an ORM loop that inserts records one at a time — will work perfectly in dev and fall over in production, and what to write instead.

MODULE 01 · LESSON 02

Clusters, Nodes and Slices

A slice is the unit of parallelism — the thing that actually reads bytes. Almost every performance problem is data spread badly across them.

1 · WHAT IT IS

One leader, several compute nodes, and each node divided into slices

Every table you create is spread across every slice. How evenly it spreads is a decision you make (Lesson 15), and it decides your query time.

2 · HOW IT WORKS

WHO DOES WHAT WHEN YOU RUN A QUERY

LEADER NODE — holds no user data

Parses, plans, compiles the plan into C++ segments, ships them to the compute nodes, and merges the final result.

COMPUTE NODES — where the data lives

Each holds a share of every table and runs the same compiled code against its own share, in parallel.

SLICES — the unit of parallelism

Each node is divided into slices, each with its own share of memory and disk. A slice is what actually scans.

SKEW IS THE ENEMY — svv_table_info.skew_rows

If most rows land on one slice, one slice does most of the work and the other fifteen wait. That is a design bug, not a capacity one.

```
a query is only as fast as its slowest slice
```

3 · THE SQL

```
-- how skewed are my tables?
SELECT "table", size, tbl_rows,
skew_rows, unsorted, stats_off
FROM svv_table_info
ORDER BY skew_rows DESC NULLS LAST;

-- skew_rows > 4 means one slice
-- holds 4x the rows of the lightest
```

4 · GOTCHAS

A DISTKEY on a low-cardinality column (country, status) is the usual cause.
More nodes will not fix skew.

check skew before tuning anything

IN PLAIN ENGLISH

Sixteen people unloading a lorry. If fourteen boxes land at one door, fifteen of them stand about while one person works.

L02 · Clusters, Nodes and Slices

Slide: `_render/L02-clusters-nodes-slices.html`

What it is

A Redshift cluster is **one leader node and several compute nodes**, and each compute node is divided into **slices**. A slice is the unit of parallelism — the thing that actually reads bytes off disk.

Every table you create is spread across every slice. *How evenly* it spreads is a decision you make (L15), and it decides your query time more than any other single factor.

Who does what

Leader node

Parses your SQL, plans it, compiles the plan into C++ segments, ships those to the compute nodes, and merges the final result. **It holds no user data.**

Two consequences worth knowing: - Queries returning millions of rows make the leader the bottleneck — it has to merge and stream all of them. - Some functions can only run on the leader (many `CURRENT_*` and catalogue queries), which is why a query mixing them with a table scan occasionally behaves oddly.

Compute nodes

Each holds a share of every table and runs the same compiled code against its own share, in parallel.

Slices

Each node is divided into slices, each with its own share of memory and disk. **A slice is what scans.** A cluster with 4 nodes × 4 slices has 16 workers; a query is only as fast as its slowest one.

Skew is the enemy

If most rows land on one slice, one slice does most of the work and the rest wait. This is a **design bug, not a capacity problem** — adding nodes will not fix it.

```
-- the skew check. run this before you tune anything.
SELECT "schema", "table", tbl_rows, size AS mb,
       diststyle, skew_rows, unsorted, stats_off
FROM   svv_table_info
ORDER  BY skew_rows DESC NULLS LAST
LIMIT  20;
```

Reading it:

COLUMN	WHAT IT MEANS	ACT WHEN
`skew_rows`	ratio of the heaviest slice to the lightest	**> 4**
`unsorted`	% of rows not in sort-key order	**> 10**
`stats_off`	% staleness of the planner's statistics	**> 10**
`diststyle`	the distribution actually in effect	it is not what you intended

`skew_rows > 4` means one slice holds four times the rows of the lightest. The usual cause is a `DISTKEY` on a low-cardinality column — `country`, `status`, `is_active`. Ten distinct values across sixteen slices leaves six slices empty and the rest overloaded.

How many slices do I have?

```
SELECT COUNT(*) AS slice_count
FROM   stv_slices
WHERE  type = 'D';        -- data slices, excluding the leader
```

On Serverless this varies with the RPU capacity in play, which is one reason performance testing there needs more than one run.

L02 · Clusters, Nodes and Slices

Try it

```
-- 1. how wide is my cluster?
SELECT COUNT(*) AS slices FROM stv_slices WHERE type = 'D';

-- 2. which tables are skewed?
SELECT "table", tbl_rows, skew_rows, diststyle
FROM   svv_table_info
WHERE  tbl_rows > 1000000
ORDER  BY skew_rows DESC NULLS LAST;

-- 3. is a specific column a sensible DISTKEY?
--    (rule of thumb: distinct values should exceed slice count by a lot)
SELECT COUNT(DISTINCT store_sk) AS distinct_values,
       COUNT(*)                AS total_rows
FROM   gold.fct_sales_line;
```

If `distinct_values` is smaller than your slice count, that column is a bad `DISTKEY`.

Gotchas

- **More nodes will not fix skew.** You will pay more and wait the same.
- `skew_rows` is `NULL` for `DISTSTYLE ALL` — that is expected; every slice holds a full copy.
- **A small table with skew does not matter.** Check tables above a million rows first.
- **Serverless hides the node count** but the slice concept is identical.

Checklist

- ☐ I can explain leader vs compute vs slice
- ☐ I know a query is as fast as its slowest slice
- ☐ I run the `svv_table_info` skew check before tuning
- ☐ I know `skew_rows > 4` is the threshold to act on
- ☐ I would not pick a low-cardinality column as a `DISTKEY`

You've got it when you can...

...be shown a query that is slow on a big cluster and check skew before touching the SQL — because if one slice holds most of the rows, no amount of rewriting will help.

MODULE 01 · LESSON 03

Provisioned vs Serverless

Two ways to buy the same engine. Serverless splits the data from the compute into two objects you will meet constantly.

1 · WHAT IT IS

Same SQL, same features. You are choosing how capacity is bought.

Provisioned: you pick nodes and they run until you stop them. Serverless: you pick a capacity range and it scales with load. We use Serverless.

2 · HOW IT WORKS

FOUR TERMS YOU WILL SEE EVERY DAY

NAMESPACE — the data

Databases, schemas, tables, users, snapshots. Everything that persists. It exists whether or not anything is running.

WORKGROUP — the compute

Capacity, network placement, security groups, endpoint. You connect to a workgroup; it reads a namespace.

RPU — Redshift Processing Unit

The unit of Serverless capacity, billed per second while queries run. You set a base and a maximum.

NODE TYPE (PROVISIONED) — RA3 uses managed storage

If you meet a provisioned cluster, the node type decides slices per node and whether storage scales separately.

```
one namespace can be read by more than one workgroup
```

3 · THE SQL

```
-- where am I, and as whom?
SELECT current_database(), current_user,
version();

-- what has Serverless been doing?
SELECT * FROM sys_serverless_usage
ORDER BY end_time DESC LIMIT 20;
```

4 · GOTCHAS

Serverless is not free when idle — a dashboard polling every minute keeps it awake all night.
Max RPU is a cost ceiling, not a target.

find out what queries at 3am

IN PLAIN ENGLISH

The namespace is the warehouse and its stock. The workgroup is the shift of staff you have booked to work in it today.

L03 · Provisioned vs Serverless

Slide: `_render/L03-provisioned-vs-serverless.html`

What it is

Two ways to buy the same engine. Same SQL, same features — you are choosing how capacity is purchased.

- **Provisioned** — you pick a node type and a node count. It runs until you stop it. Predictable cost, manual sizing.
- **Serverless** — you pick a capacity range. It scales with load and pauses when idle. **This is what we use.**

The four terms you will see daily

Namespace — *the data*

Databases, schemas, tables, users, snapshots. Everything that persists. It exists whether or not anything is running.

Workgroup — *the compute*

Capacity, VPC placement, security groups, the endpoint you connect to. **You connect to a workgroup; it reads a namespace.**

One namespace can be read by more than one workgroup — which is how you give a heavy ETL job and a light BI workload separate compute over the same data.

RPU — Redshift Processing Unit

The unit of Serverless capacity, billed per second while queries run. You set a **base** capacity and a **maximum**. The maximum is a cost ceiling, not a target.

Node type — *provisioned only*

If you meet a provisioned cluster, the node type decides slices per node and whether storage scales separately. **RA3** uses Redshift Managed Storage (L04); older **DC2** has fixed local storage.

Try it

```
-- where am I, and as whom?
SELECT current_database(), current_user, version();

-- what has Serverless actually been doing?
SELECT start_time, end_time, compute_seconds, compute_capacity
FROM   sys_serverless_usage
ORDER  BY end_time DESC
LIMIT  50;

-- which queries burned the most time today?
SELECT query_id,
       elapsed_time / 1000000.0 AS secs,
       LEFT(query_text, 100)    AS sql,
       user_id
FROM   sys_query_history
WHERE  start_time > DATEADD(day, -1, GETDATE())
ORDER  BY elapsed_time DESC
LIMIT  20;
```

That last query is the one to run when someone says "Redshift is expensive". It usually names a single dashboard.

The cost trap

Serverless is not free when idle. It pauses when *nothing is running* — but a dashboard polling every minute, a monitoring check, or a BI tool with auto-refresh keeps it awake all night at base capacity.

Before you conclude Serverless is expensive, find out what queries at 3am:

L03 · Provisioned vs Serverless

```
SELECT DATE_TRUNC('hour', start_time) AS hr,
       COUNT(*)                       AS queries,
       SUM(elapsed_time) / 1000000.0  AS total_secs
FROM   sys_query_history
WHERE  start_time > DATEADD(day, -3, GETDATE())
GROUP  BY 1
ORDER  BY 1;
```

Hours with queries but no humans awake are the finding.

Which to choose

CHOOSE SERVERLESS WHEN	CHOOSE PROVISIONED WHEN
load is spiky or unknown	load is steady and predictable
there are quiet periods	it runs hot 24/7
you do not want to size anything	you want reserved-instance pricing
you are starting out	you have measured and know your shape

For a nightly-batch retail warehouse with daytime reporting, Serverless is almost always right.

Gotchas

- **Max RPU is a ceiling, not a goal.** Setting it very high does not make queries faster; it removes the cap on what a runaway query can cost.
- **Base RPU sets the floor** for what an idle-but-not-paused workgroup costs.
- **Snapshots belong to the namespace**, not the workgroup — deleting a workgroup does not delete data.

Checklist

- ☐ I can explain namespace vs workgroup without hesitating
- ☐ I know what an RPU is and how it is billed
- ☐ I have run `sys_serverless_usage` on a real workgroup
- ☐ I know how to find what queries outside working hours
- ☐ I know max RPU is a cost ceiling

You've got it when you can…

…be told "the Redshift bill went up" and find the cause in two queries — the hourly query profile and the top queries by elapsed time.

MODULE 01 · LESSON 04

Redshift Managed Storage

Storage sits on S3 with a local SSD cache in front of it. That is why you size compute for queries and stop thinking about disk.

1 · WHAT IT IS

Compute and storage scale independently — and are billed separately

In your app database, running out of disk means a bigger machine. Here, storage grows on its own and compute is sized for the query load alone.

2 · HOW IT WORKS

FOUR CONSEQUENCES WORTH KNOWING

S3 UNDERNEATH, SSD IN FRONT — `automatic tiering`

Hot blocks stay on local SSD; cold blocks live in S3 and are fetched on demand. You do not manage this.

PETABYTE SCALE — `no capacity planning`

You will not run out of space and have to migrate. That entire class of operational work disappears.

CHEAP SNAPSHOTS — `incremental`

Backups are incremental against RMS, so they are fast to take and restore, and cheap to keep.

DATA SHARING BECOMES POSSIBLE — `datashares`

Because storage is decoupled, another warehouse can read your live data without a copy being made.

```
you can outgrow your compute. you will not outgrow your storage.
```

3 · THE SQL

```sql
-- what is actually taking space?
SELECT "schema", "table",
size AS mb, tbl_rows,
pct_used
FROM svv_table_info
ORDER BY size DESC LIMIT 20;

-- size is in MB, per table
```

4 · GOTCHAS

Only RA3 and Serverless use RMS. Older DC2 clusters have fixed local storage.

Deleted rows still occupy space until VACUUM.

storage is cheap, not free

IN PLAIN ENGLISH

A shop with an unlimited stockroom next door and a small shelf behind the counter. Popular stock stays on the shelf; the rest is fetched.

MODULE 01 · LESSON 04

L04 · Redshift Managed Storage

Slide: `_render/L04-managed-storage.html`

What it is

Storage sits on **S3 underneath, with a local SSD cache in front of it**. Compute and storage scale independently and are billed separately.

In your app database, running out of disk means a bigger machine and a migration. Here, storage grows on its own and you size compute for the query load alone. An entire category of operational work disappears.

Four consequences

Automatic tiering

Frequently accessed blocks stay on local SSD; cold blocks live in S3 and are fetched on demand. **You do not manage this** and there is no knob for it.

Petabyte scale, no capacity planning

You will not run out of space and have to migrate. Plan compute; do not plan disk.

Cheap, fast snapshots

Backups are incremental against RMS. They are quick to take, quick to restore, and cheap to retain — which makes a restore drill practical rather than theoretical.

Data sharing becomes possible

Because storage is decoupled from compute, another warehouse can read your **live** data without a copy being made. That is what a datashare is, and it only works because of RMS.

Try it

```
-- what is actually taking space?
SELECT "schema", "table",
       size           AS mb,
       tbl_rows,
       pct_used,
       unsorted,
       diststyle
FROM   svv_table_info
ORDER  BY size DESC
LIMIT  20;

-- total, by schema
SELECT "schema",
       SUM(size) / 1024.0 AS gb,
       COUNT(*)           AS tables
FROM   svv_table_info
GROUP  BY 1
ORDER  BY gb DESC;
```

`size` is in **megabytes**, per table. `pct_used` tells you how much of the allocated space actually holds data — a low value on a large table usually means deleted rows awaiting `VACUUM` (L42).

The one place storage still costs you

Deleted and updated rows are **not** removed immediately. Redshift marks them and reclaims the space later. Until then you are paying for them, and queries still skip past them.

```
-- how much dead space am I carrying?
SELECT "table", size AS mb, tbl_rows, unsorted, pct_used
FROM   svv_table_info
WHERE  size > 1000            -- tables over ~1GB
ORDER  BY pct_used ASC        -- worst first
LIMIT  20;
```

A big table with low `pct_used` and high `unsorted` is a table that needs maintenance.

L04 · Redshift Managed Storage

Gotchas

- **Only RA3 and Serverless use RMS.** Older DC2 clusters have fixed local storage, and there the disk-full problem is real.
- **Storage is billed separately from compute.** A paused Serverless workgroup still stores data, and you still pay for that.
- **Cold blocks are slower on first read.** A query over data untouched for months pays an S3 fetch. This is normal and usually invisible.
- `size` **is MB, not GB.** People misread this constantly and conclude a table is a thousand times bigger than it is.

Checklist

- ☐ I can explain why compute and storage scale separately
- ☐ I know `svv_table_info.size` is in megabytes
- ☐ I know deleted rows still occupy space until `VACUUM`
- ☐ I can find the biggest tables and the ones with most dead space
- ☐ I know only RA3 and Serverless use RMS

You've got it when you can…

…be asked "how big is our warehouse and what is driving it" and answer with two queries — total by schema, and the top twenty tables by size.

MODULE 01 · LESSON 05

How A Query Actually Runs

Five stages. The third one — compilation — explains why your first run is slow and every run after it is not.

1 · WHAT IT IS

Redshift compiles your SQL into C++ and ships the binary to the slices

It is not interpreting your statement row by row. It builds a program, distributes it, and every slice runs it against its own share of the data.

2 · HOW IT WORKS

FIVE STAGES, ON THE LEADER THEN THE NODES

1 · PARSE · 2 · PLAN — `leader node`

The planner uses table statistics to choose join order and distribution. Stale statistics produce a bad plan — that is what ANALYZE is for.

3 · COMPILE — `cached afterwards`

The plan becomes compiled C++ segments. First run pays for compilation; later runs of the same shape reuse the cache.

4 · EXECUTE IN PARALLEL — `every slice, at once`

Each slice runs the same code over its own data. If a join needs rows from another slice, they are moved — and moving is the expensive part.

5 · RETURN — `leader merges`

The leader merges partial results and sends them to you. A query returning millions of rows makes the leader the bottleneck.

```
"the first run is always slow" = compilation, not your SQL
```

3 · THE SQL

```
-- what did my last queries cost?
SELECT query_id, elapsed_time/1000000.0
AS secs, status, query_text
FROM sys_query_history
WHERE user_id = current_user_id
ORDER BY start_time DESC LIMIT 20;

-- elapsed_time is microseconds
```

4 · GOTCHAS

Never benchmark a query on its first run.
Changing a literal can still hit the cache;
changing the shape will not.

run it twice before you judge it

IN PLAIN ENGLISH

The first time you order a dish the kitchen writes the recipe. After that they just cook it. Judging the kitchen on the first order is unfair.

L05 · How A Query Actually Runs

Slide: `_render/L05-how-a-query-runs.html`

What it is

Redshift does not interpret your SQL row by row. It **compiles your query into C++** and ships the binary to the slices, where every slice runs it against its own share of the data.

That is why your first run is slow and every run after it is not — and why benchmarking a first run is meaningless.

The five stages

1 · Parse — on the leader. Syntax and object resolution.

2 · Plan — on the leader. The planner uses **table statistics** to choose join order and how data will be moved. Stale statistics produce a bad plan; that is what `ANALYZE` is for (L42).

3 · Compile — the plan becomes compiled C++ segments. This costs real time on the first execution of a given query *shape*. The result is cached, so later runs skip it.

4 · Execute — every slice runs the same code over its own data, in parallel. If a join needs rows that live on another slice, they get **moved** — and moving data is the expensive part (L29).

5 · Return — the leader merges partial results and streams them to you. A query returning millions of rows makes the leader the bottleneck.

Try it — see the compile cost yourself

```
-- run this once, note the time
SELECT store_sk, SUM(net_amount)
FROM   gold.fct_sales_line
WHERE  sale_date >= '2026-01-01'
GROUP  BY 1;

-- run the identical statement again — usually much faster
```

Then look at what actually happened:

```
SELECT query_id,
       elapsed_time / 1000000.0        AS total_secs,
       queue_time  / 1000000.0         AS queued_secs,
       execution_time / 1000000.0      AS exec_secs,
       compile_time / 1000000.0        AS compile_secs,
       status,
       LEFT(query_text, 90)            AS sql
FROM   sys_query_history
WHERE  user_id = current_user_id
ORDER  BY start_time DESC
LIMIT  10;
```

`compile_secs` on the first run and near zero on the second is the whole lesson, visible.

> **Times in `sys_query_history` are microseconds.** Divide by 1,000,000. Forgetting this is how people report a 3-second query as 3 million.

What counts as "the same shape"

The compile cache keys on the **shape** of the query, not its literals. So:

```
-- these two share a compiled plan
SELECT * FROM sales WHERE sale_date = '2026-01-01';
SELECT * FROM sales WHERE sale_date = '2026-02-14';

-- this one does not — different shape
SELECT * FROM sales WHERE sale_date BETWEEN '2026-01-01' AND '2026-02-14';
```

L05 · How A Query Actually Runs

This is a strong argument for **parameterised queries** from application code: they reuse the cache. String-concatenated SQL that varies its structure recompiles constantly.

Why the leader can be a bottleneck

```
-- bad: leader must merge and stream 20 million rows to you
SELECT * FROM gold.fct_sales_line;

-- good: the compute nodes do the work, the leader returns one row
SELECT COUNT(*), SUM(net_amount) FROM gold.fct_sales_line;
```

If you genuinely need millions of rows out, use `UNLOAD` to S3 (L27) rather than dragging them through the leader and over the wire.

Gotchas

- **Never benchmark a first run.** Run it at least twice and quote the second.
- **A cluster restart or version upgrade clears the compile cache**, so the "first slow run" returns.
- **Queue time is not execution time.** A "slow" query may have spent most of its life waiting for a WLM slot (L41) — `sys_query_history` separates them, so check before you optimise SQL that was never the problem.

Checklist

- ☐ I can name the five stages
- ☐ I know why the first run is slow
- ☐ I always run a query twice before judging it
- ☐ I know times in `sys_query_history` are microseconds
- ☐ I check `queue_time` before optimising `execution_time`
- ☐ I use parameterised queries so the compile cache is reused

You've got it when you can…

…be shown a "slow query" and determine in one query whether it was slow because of compilation, queuing, or actual execution — before changing a single line of SQL.

MODULE 01 · LESSON 06

Connecting To Redshift

Four ways in. As a Node developer your instinct is a connection pool — and for most of what you will write, that is the wrong one.

1 · WHAT IT IS

Redshift speaks the Postgres wire protocol — but it is not an app database

Connections are a scarce, expensive resource here, and warehouse queries take seconds not milliseconds. Holding a pool open is a habit worth losing.

2 · HOW IT WORKS

FOUR DOORS, AND WHEN EACH IS RIGHT

REDSHIFT DATA API — `@aws-sdk/client-redshift-data`

HTTPS, IAM-authenticated, no connection to hold. Submit, poll, fetch. The right choice from Lambda and from almost all Node code.

JDBC / ODBC — `port 5439`

What BI tools use. Also the `pg` driver from Node — correct for a long-lived service, wrong for a Lambda that runs for 400ms.

QUERY EDITOR V2 — `in the console`

Browser SQL with saved queries and result export. Where you will actually live while learning. No local setup at all.

psql — `yes, really`

The Postgres CLI connects. Handy for scripting and for anyone who already lives in a terminal.

```
from Lambda, use the Data API. always. see Lesson 39.
```

3 · THE CODE

```
// Node, no connection held
const c = new RedshiftDataClient({});
const { Id } = await c.send(
new ExecuteStatementCommand({
WorkgroupName, Database,
Sql: 'select count(*) from sales' }));
// then poll Describe/GetResult by Id
```

4 · GOTCHAS

The Data API is asynchronous. You submit and poll — it does not block like `pg` does.
A pool in Lambda leaks connections.

connections are scarce here

IN PLAIN ENGLISH

You do not keep a phone line open to the warehouse all day. You send the order, get a ticket number, and collect the result when it is ready.

L06 · Connecting To Redshift

Slide: `_render/L06-connecting-to-redshift.html`

What it is

Four ways in. Your instinct as a Node developer is a connection pool — and for most of what you will write here, that is the wrong one.

Connections are a **scarce, expensive resource** in a warehouse, and queries take seconds rather than milliseconds. Holding a pool open across a fleet of Lambdas is how you exhaust the connection limit and take the warehouse down for everyone.

The four doors

1 · Redshift Data API — *use this from Node*

HTTPS, IAM-authenticated, **no connection to hold**. You submit a statement, get an ID back, poll for completion, then fetch results.

```
import {
  RedshiftDataClient,
  ExecuteStatementCommand,
  DescribeStatementCommand,
  GetStatementResultCommand,
} from "@aws-sdk/client-redshift-data";

const client = new RedshiftDataClient({});

export async function runSql(sql, params = []) {
  const { Id } = await client.send(new ExecuteStatementCommand({
    WorkgroupName: process.env.REDSHIFT_WORKGROUP,
    Database:      process.env.REDSHIFT_DATABASE,
    Sql:           sql,
    Parameters:    params,          // [{ name: 'p1', value: '2026-01-01' }]
  }));

  // poll — the Data API is asynchronous, unlike `pg`
  for (;;) {
    const st = await client.send(new DescribeStatementCommand({ Id }));
    if (st.Status === "FINISHED") break;
    if (st.Status === "FAILED" || st.Status === "ABORTED") {
      throw new Error(`${st.Status}: ${st.Error}`);
    }
    await new Promise(r => setTimeout(r, 500));
  }

  const out = await client.send(new GetStatementResultCommand({ Id }));
  return out.Records ?? [];
}
```

Note `Parameters` — **use them**. It keeps you out of SQL-injection trouble and it reuses the compile cache (L05).

2 · JDBC / ODBC — *for long-lived services and BI*

Port **5439**. What Power BI and Tableau use. The `pg` driver from Node also works, and is correct for a long-running service that can hold a pool — **wrong for a Lambda that lives for 400ms**.

3 · Query Editor v2 — *where you will actually live*

Browser SQL in the console. Saved queries, result export, no local setup, no credentials on your laptop. Use it for everything while learning.

4 · psql

The Postgres CLI connects. Useful for scripting and for anyone who lives in a terminal.

L06 · Connecting To Redshift

Data API vs `pg` — the decision

	DATA API	`PG` DRIVER
Connection	none held	pooled, must be managed
Auth	IAM	username/password or IAM
Lambda	✅ correct	❌ leaks connections
Long-lived service	works	✅ correct
Result size	paginated, modest	streams large results
Style	async submit + poll	blocking call
VPC needed	no	yes, for private clusters

Rule: if the code is serverless or short-lived, use the Data API. If it is a long-running service that can own a pool properly, `pg` is fine.

Try it

```
# from a shell with AWS credentials
aws redshift-data execute-statement \
  --workgroup-name my-workgroup \
  --database dev \
  --sql "select current_user, current_database()"

# then, with the Id it returned
aws redshift-data get-statement-result --id <statement-id>
```

Gotchas

- **The Data API is asynchronous.** It does not block like `pg`. Code written expecting a synchronous return will silently get nothing.
- **A connection pool inside Lambda leaks.** Each concurrent invocation opens its own; scale to 200 concurrency and you have 200 connections.
- **There is a connection limit**, and hitting it locks out everyone including your ETL.
- **Data API results are paginated** and not meant for pulling millions of rows — use `UNLOAD` for that (L27).

Checklist

- ☐ I know why a connection pool in Lambda is wrong
- ☐ I can write the submit → poll → fetch loop from memory
- ☐ I use `Parameters` rather than string concatenation
- ☐ I know when `pg` is the right choice instead
- ☐ I know large result sets belong in `UNLOAD`, not the Data API

You've got it when you can…

…review a colleague's Lambda that opens a `pg` pool and explain — in terms of connection limits, not style — why it will work in test and fail under load.

MODULE 01 · LESSON 07

Databases, Schemas and search_path

The object hierarchy, and the one setting that decides which table your unqualified SQL actually hit.

1 · WHAT IT IS

database → schema → table, and you cannot join across databases

That last part surprises people. One connection is to one database. Schemas — not databases — are how you separate layers, teams and environments inside it.

2 · HOW IT WORKS

FOUR THINGS THAT DECIDE WHICH TABLE YOU HIT

DATABASE — one per connection

A hard boundary. To read another database you need a datashare or an external schema — not a three-part name.

SCHEMA — raw · staging · gold

Where the real separation happens. One schema per layer, and grants applied at schema level rather than table by table.

search_path — per session

The ordered list of schemas searched for an unqualified name. Two people can run identical SQL and read different tables.

public — the default dumping ground

Exists by default and everyone can write to it. Revoke that on day one, or it fills with somebody's scratch tables.

```
always schema-qualify in anything committed to a repo
```

3 · THE SQL

```
SHOW search_path;
SET search_path TO gold, staging, public;

-- what schemas exist, and who owns them?
SELECT schema_name, schema_owner,
schema_type
FROM svv_all_schemas
ORDER BY 1;
```

4 · GOTCHAS

Unqualified names in a script are a bug waiting for someone else's search_path.
No cross-database joins. None.

qualify everything you commit

IN PLAIN ENGLISH

search_path is the PATH variable of SQL. "It works on my machine" has exactly the same cause here as it does in a shell.

L07 · Databases, Schemas and search_path

Slide: `_render/L07-databases-schemas-searchpath.html`

What it is

The hierarchy is **database** → **schema** → **table**, and you **cannot join across databases**.

That last part surprises people arriving from MySQL, where `db1.table` and `db2.table` in one query is routine. Here, one connection is to one database, and **schemas** — not databases — are how you separate layers, teams and environments inside it.

The four things that decide which table you hit

Database — a hard boundary

To read another database you need a **datashare** or an **external schema**. There is no three-part name that works.

Schema — where separation actually happens

One schema per layer. Grants applied at schema level rather than table by table (L13).

```
raw        -- landed, untouched
staging    -- being processed, safe to truncate
gold       -- modelled facts and dimensions
rpt        -- reporting views, what BI sees
lake_ext   -- external schema over S3
```

search_path — the one that catches people

An **ordered list of schemas** searched for an unqualified table name. It is a **session** setting, so two people can run identical SQL and read different tables.

```
SHOW search_path;
SET  search_path TO gold, staging, public;
```

This is the `PATH` variable of SQL. *"It works on my machine"* has exactly the same cause here as it does in a shell.

public — the default dumping ground

Exists by default, and by default everyone can create in it. Revoke that on day one or it fills with somebody's `test_2`, `test_2_final`, `test_2_final_v3`.

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
```

The rule

> **Always schema-qualify in anything committed to a repository.**

```
-- fine interactively, a bug in a repo
SELECT * FROM fct_sales_line;

-- correct anywhere
SELECT * FROM gold.fct_sales_line;
```

An unqualified name in a dbt model, a stored procedure or a Node query string is a bug waiting for someone else's `search_path`.

L07 · Databases, Schemas and search_path

Try it

```
-- what schemas exist, and what kind are they?
SELECT database_name, schema_name, schema_owner, schema_type
FROM   svv_all_schemas
ORDER  BY 1, 2;

-- what tables live in each?
SELECT table_schema, COUNT(*) AS tables
FROM   svv_all_tables
GROUP  BY 1
ORDER  BY 2 DESC;

-- who can create in public? (should be nobody)
SELECT nspname, nspacl FROM pg_namespace WHERE nspname = 'public';
```

`svv_all_schemas` and `svv_all_tables` include **external** schemas too, which the older `pg_*` catalogue views do not. Prefer the `svv_all_*` views.

Creating a schema properly

```
CREATE SCHEMA IF NOT EXISTS gold;
ALTER  SCHEMA gold OWNER TO etl_role;

GRANT USAGE ON SCHEMA gold TO ROLE bi_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA gold TO ROLE bi_reader;

-- and so tomorrow's tables are covered without anyone remembering
ALTER DEFAULT PRIVILEGES IN SCHEMA gold
  GRANT SELECT ON TABLES TO ROLE bi_reader;
```

That last statement is the one people forget, and the reason "the new table is invisible to BI" keeps happening (L13).

Gotchas

- **No cross-database joins. None.** Use a datashare or an external schema.
- `search_path` **is per session.** A stored procedure may run with a different one than you tested with.
- **Dropping a schema with** `CASCADE` **drops everything in it**, including tables other people depend on.
- **Object names are lower-cased** unless you double-quote them. Do not double-quote them.

Checklist

- ☐ I know you cannot join across databases
- ☐ I use schemas as the layer boundary
- ☐ I schema-qualify everything I commit
- ☐ I have revoked `CREATE ON SCHEMA public FROM PUBLIC`
- ☐ I set `ALTER DEFAULT PRIVILEGES` when creating a schema
- ☐ I use `svv_all_schemas` rather than the `pg_*` views

You've got it when you can…

…debug "the query returns different rows for her than for me" by asking about `search_path` before looking at the data.

MODULE 01 · LESSON 08

CREATE TABLE, Anatomy Of

The clauses that look like Postgres and mean something different — and the three that only exist here.

1 · WHAT IT IS

Physical layout is part of the schema, declared at create time

In Postgres you create the table and add indexes later. Here, distribution and sort are properties of the table itself — and changing them means rebuilding it.

2 · HOW IT WORKS

THE CLAUSES, AND WHICH ONES ACTUALLY MATTER

DISTSTYLE / DISTKEY — Lesson 15

Where rows land across slices. The single biggest lever on join performance, and the hardest to change afterwards.

SORTKEY — Lesson 16

Physical order on disk. Lets Redshift skip whole blocks that cannot satisfy your WHERE clause. Usually the date column.

ENCODE — per column

Compression, chosen per column. AUTO is good; specifying it matters on very large facts. Lesson 17.

PRIMARY KEY · NOT NULL — only NOT NULL is enforced

NOT NULL is real. PRIMARY KEY, UNIQUE and FOREIGN KEY are informational only — they inform the planner, nothing more (Lesson 18).

`get DISTKEY and SORTKEY right at create time — everything else is tunable later`

3 · THE SQL

```
CREATE TABLE gold.fct_sales_line (
sale_date DATE NOT NULL,
store_sk BIGINT NOT NULL,
product_sk BIGINT NOT NULL,
net_amount DECIMAL(14,2)
)
DISTKEY (store_sk)
SORTKEY (sale_date);
```

4 · GOTCHAS

You cannot ALTER a DISTKEY or SORTKEY on an existing table — you rebuild and swap.
There is no CREATE INDEX to fall back on.

design it before you load it

IN PLAIN ENGLISH

In Postgres you build the shelves and label them later. Here the labels are part of the shelf, and re-labelling means rebuilding.

L08 · CREATE TABLE, Anatomy Of

Slide: `_render/L08-create-table-anatomy.html`

What it is

In Postgres you create a table and add indexes later. In Redshift **physical layout is part of the schema**, declared at create time — and changing it means rebuilding the table.

That raises the stakes on `CREATE TABLE` considerably. Get `DISTKEY` and `SORTKEY` right; everything else is tunable later.

The full anatomy

```
CREATE TABLE gold.fct_sales_line (
    sale_date     DATE           NOT NULL,
    store_sk      BIGINT         NOT NULL,
    product_sk    BIGINT         NOT NULL,
    receipt_no    VARCHAR(32),
    quantity      DECIMAL(12,3),
    net_amount    DECIMAL(14,2)  ENCODE az64,
    vat_amount    DECIMAL(14,2)  ENCODE az64,
    merge_key     VARCHAR(64)    NOT NULL,
    loaded_at     TIMESTAMP      DEFAULT SYSDATE
)
DISTSTYLE KEY
DISTKEY  (store_sk)
COMPOUND SORTKEY (sale_date, store_sk);
```

CLAUSE	WHAT IT DOES	CHANGEABLE LATER?
`DISTSTYLE` / `DISTKEY`	where rows land across slices	❌ rebuild
`SORTKEY`	physical order on disk	❌ rebuild
`ENCODE`	per-column compression	⚠️ some, via `ALTER`
`NOT NULL`	**enforced**	✅
`PRIMARY KEY` / `UNIQUE` / `FOREIGN KEY`	**informational only**	✅
`DEFAULT`	default value	✅

The two that matter

`DISTKEY` — pick the column this table is most often **joined** on, ideally with high cardinality. For a fact table that is usually the busiest foreign key. Get it wrong and every join ships data between nodes (L15, L29).

`SORTKEY` — pick the column you most often **filter** on. For almost every retail fact that is the date. Redshift keeps min/max per block and skips blocks that cannot match (L16).

> A useful mnemonic: **DIST for joins, SORT for filters.**

The clause that catches everyone

```
CREATE TABLE dim_store (
  store_sk BIGINT PRIMARY KEY,   -- NOT enforced
  store_id VARCHAR(20) UNIQUE,   -- NOT enforced
  ...
);

INSERT INTO dim_store VALUES (1, 'S001');
INSERT INTO dim_store VALUES (1, 'S001');   -- succeeds. twice.
```

`PRIMARY KEY`, `UNIQUE` and `FOREIGN KEY` are **informational only** — they inform the query planner and are otherwise ignored. Only `NOT NULL` is enforced.

Declare them anyway (the planner uses them), but **test for uniqueness separately** — in dbt, or with a query in your load job (L18).

L08 · CREATE TABLE, Anatomy Of

Other creation forms

```
-- from a query, inheriting nothing about layout unless you say so
CREATE TABLE staging.sales_tmp
  DISTKEY (store_sk) SORTKEY (sale_date)
AS SELECT * FROM raw.sales WHERE dt = '2026-08-12';

-- structure only, no rows — copies dist/sort/encodings
CREATE TABLE staging.sales_like (LIKE gold.fct_sales_line);

-- session-scoped, disappears on disconnect
CREATE TEMP TABLE t_recent AS SELECT ...;
```

`CREATE TABLE LIKE` is the right way to build a staging table that matches its target — it inherits the physical design, so the final swap is cheap.

Try it

```
-- what did Redshift actually create?
SELECT "table", diststyle, sortkey1, sortkey_num, encoded
FROM   svv_table_info
WHERE  "schema" = 'gold';

-- the full DDL of an existing table
SELECT ddl FROM admin.v_generate_tbl_ddl WHERE tablename = 'fct_sales_line';
-- (v_generate_tbl_ddl is an AWS-published admin view; if it is not
--  installed, svv_table_info plus pg_table_def gets you most of the way)

SELECT "column", type, encoding, distkey, sortkey, "notnull"
FROM   pg_table_def
WHERE  schemaname = 'gold' AND tablename = 'fct_sales_line';
```

Gotchas

- **You cannot `ALTER` a `DISTKEY` or `SORTKEY`.** You create a new table, load it, and swap names.
- **`CTAS` does not inherit dist/sort** unless you state them — a very common way to accidentally create an `EVEN`-distributed copy.
- **`DEFAULT SYSDATE` is evaluated at insert**, which makes it useful as a load timestamp.
- **There is no `SERIAL` you should rely on.** `IDENTITY` exists but values are not gap-free across slices; generate surrogate keys deliberately.

Checklist

- ☐ I state `DISTKEY` and `SORTKEY` explicitly on every fact table
- ☐ I know only `NOT NULL` is enforced
- ☐ I test uniqueness rather than declaring it and trusting it
- ☐ I use `CREATE TABLE LIKE` for staging tables
- ☐ I remember `CTAS` needs dist/sort stated
- ☐ I can read `pg_table_def` to see what a table really is

You've got it when you can…

…write a `CREATE TABLE` for a new fact, justify the `DISTKEY` and `SORTKEY` out loud in one sentence each, and say what you would have to do to change them later.

MODULE 01 · LESSON 09

Data Types That Will Bite You

Four differences from Postgres that cause real bugs — and one of them silently loses money.

1 · WHAT IT IS

The type list looks familiar. Four entries behave differently enough to matter.

Column width is not free here, there is no TEXT, floating point is the wrong choice for money, and JSON is a type called SUPER.

2 · HOW IT WORKS

FOUR THAT CATCH POSTGRES DEVELOPERS

VARCHAR WIDTH COSTS — no TEXT type

Width affects the memory a query reserves per row. Declaring everything VARCHAR(65535) will make joins and sorts spill to disk.

VARCHAR COUNTS BYTES — not characters

A multi-byte character can consume four. Arabic and emoji overflow columns sized by counting letters — a real risk on this data.

DECIMAL FOR MONEY — never FLOAT

FLOAT and DOUBLE are approximate. Sum a million of them and the total will not reconcile, and nobody will know why.

TIMESTAMP vs TIMESTAMPTZ — Lesson 32

TIMESTAMP has no zone. Pick one convention across the whole warehouse and write it down, or you get off-by-one-day bugs forever.

```
size VARCHAR to the data, not to the maximum you can imagine
```

3 · THE SQL

```sql
-- how wide is the data really?
SELECT max(octet_length(product_name))
AS max_bytes,
max(length(product_name))
AS max_chars
FROM staging.products;

-- size the column from max_bytes
```

4 · GOTCHAS

COPY fails on a value one byte too long —
and the message names the row, not the fix.
Widening a column later means a rebuild.

measure the data before you size it

IN PLAIN ENGLISH

Declaring every column as huge “just in case” is like booking a coach for every passenger. It fits, and it costs you the whole journey.

L09 · Data Types That Will Bite You

Slide: `_render/L09-data-types.html`

What it is

The type list looks familiar. Four entries behave differently enough from Postgres to cause real bugs — and one of them silently loses money.

1 · VARCHAR width is not free

There is **no `TEXT` type**. Every `VARCHAR` has a declared width, and that width affects how much memory a query reserves **per row** for sorts, joins and aggregations.

Declaring everything `VARCHAR(65535)` "to be safe" makes hash joins and sorts spill to disk, and turns a fast query into a slow one for no benefit.

```
-- measure before you size
SELECT MAX(octet_length(product_name)) AS max_bytes,
       MAX(length(product_name))       AS max_chars,
       AVG(octet_length(product_name)) AS avg_bytes
FROM   staging.products;
```

Size from `max_bytes`, with a sensible margin — not from imagination.

2 · VARCHAR counts bytes, not characters

`VARCHAR(20)` means **20 bytes**. A multi-byte character can consume up to four.

```
SELECT length('محمد')        AS chars,   -- 4
       octet_length('محمد')  AS bytes;   -- 8
```

This matters directly on this data. Arabic product and store names, and any emoji in a customer field, will overflow columns sized by counting letters. `COPY` then fails on a value one byte too long, and the error names the row rather than the cause.

3 · DECIMAL for money — never FLOAT

`FLOAT4` / `FLOAT8` are **approximate**. Sum a million of them and the total will not reconcile, and nobody will be able to explain the pennies.

```
SELECT SUM(v) FROM (SELECT 0.1::FLOAT8 AS v FROM stl_scan LIMIT 10) t;
-- 0.9999999999999999

SELECT SUM(v) FROM (SELECT 0.1::DECIMAL(10,2) AS v FROM stl_scan LIMIT 10) t;
-- 1.00
```

Rule: anything that is money, quantity or a measure people reconcile → `DECIMAL(p,s)`. `FLOAT` is for scientific measures and ratios you never sum.

Watch the scale in arithmetic too — `DECIMAL(14,2) / DECIMAL(14,2)` does not give you two decimal places by default. Cast explicitly when it matters.

4 · TIMESTAMP vs TIMESTAMPTZ

`TIMESTAMP` carries **no time zone**. `TIMESTAMPTZ` does.

Mixing them across a warehouse produces off-by-one-day bugs that survive for years, because they only show up at the boundaries of a reporting period.

Pick one convention, write it down, and apply it everywhere. The usual choice: store `TIMESTAMP` in UTC, convert at the reporting layer only.

```
SELECT CONVERT_TIMEZONE('Asia/Riyadh', event_ts_utc) AS local_ts
FROM   gold.fct_sales_line;
```

L09 · Data Types That Will Bite You

The types you will actually use

TYPE	USE FOR
`SMALLINT` / `INTEGER` / `BIGINT`	keys, counts
`DECIMAL(p,s)`	**money, quantities, anything summed**
`DOUBLE PRECISION`	ratios, scientific values
`VARCHAR(n)`	text — sized to the data, in bytes
`CHAR(n)`	fixed-width codes only; pads with spaces
`DATE`	calendar dates
`TIMESTAMP`	UTC instants (the house convention)
`BOOLEAN`	flags
`SUPER`	landed JSON only (L10)

Try it

```
-- audit every VARCHAR in a schema against its real content width
SELECT schemaname, tablename, "column", type
FROM   pg_table_def
WHERE  schemaname = 'gold'
  AND  type LIKE 'character varying%'
ORDER  BY 1, 2, 3;

-- and for a specific table, what widths are actually used
SELECT MAX(octet_length(receipt_no)) AS receipt_no_bytes,
       MAX(octet_length(merge_key))  AS merge_key_bytes
FROM   gold.fct_sales_line;
```

Gotchas

- `COPY` **fails on one over-long value**, and reports the row, not the fix. L22 covers the diagnosis.
- **Widening a column later means a rebuild** on large tables — size it right the first time.
- `CHAR` **pads with spaces** and will surprise you in comparisons and joins. Use `VARCHAR`.
- **Implicit casts in a join** (`VARCHAR` to `INT`) prevent Redshift from using the distribution properly. Match your key types.

Checklist

- ☐ No `VARCHAR(65535)` anywhere I control
- ☐ Column widths measured from `octet_length`, not guessed
- ☐ All money and quantities are `DECIMAL`, never `FLOAT`
- ☐ One timestamp convention, written down
- ☐ Join keys have matching types on both sides
- ☐ I have checked Arabic text fits the columns sized for it

You've got it when you can…

…be handed a `CREATE TABLE` full of `VARCHAR(65535)` and `FLOAT` and explain, with a specific consequence for each, why it needs changing before anything is loaded.

MODULE 01 · LESSON 10

SUPER and Semi-Structured Data

JSON, for people who arrived from Node and Mongo. It works — and it is still not where your reporting columns should live.

1 · WHAT IT IS

SUPER is Redshift's schemaless column type, queried with PartiQL

Dot and bracket navigation, exactly like JavaScript. Excellent for landing an API payload whole. Poor as the permanent home of a column people filter on.

2 · HOW IT WORKS

FOUR THINGS TO KNOW BEFORE YOU REACH FOR IT

NAVIGATION IS JAVASCRIPT-LIKE — PartiQL

`payload.customer.id` and `payload.items[0].sku` do what you expect. A missing path returns NULL rather than erroring.

UNNEST ARRAYS BY JOINING — `FROM t, t.items AS i`

An array becomes rows by putting it in the FROM clause. This is how one order document becomes one row per line.

IT IS NOT COLUMNAR — the cost

A SUPER column is stored whole. Filtering on a field inside it reads the entire document — no zone maps, no block skipping.

SHRED WHAT YOU QUERY — bronze → silver

Land the payload as SUPER in bronze, then extract the fields anyone filters or joins on into real typed columns in silver.

```
SUPER is a landing zone, not a data model
```

3 · THE SQL

```
SELECT o.payload.order_id::BIGINT,
o.payload.customer.email::VARCHAR,
i.sku::VARCHAR,
i.qty::INT
FROM bronze.orders o,
o.payload.items i;

-- one order document -> one row per line
```

4 · GOTCHAS

Always cast on the way out — an uncast SUPER value compares by its own rules, not yours. A gold table should contain no SUPER at all.

shred before anyone reports on it

IN PLAIN ENGLISH

Keep the original envelope, by all means. But type the address onto the label — do not make the postman open every letter to sort the round.

L10 · SUPER and Semi-Structured Data

Slide: `_render/L10-super-and-json.html`

What it is

`SUPER` is Redshift's schemaless column type, queried with **PartiQL** — dot and bracket navigation that will feel immediately familiar from JavaScript.

It is excellent for landing an API payload whole. It is a poor permanent home for any column people filter, join or report on.

Navigation is JavaScript-like

```
SELECT payload.order_id,
       payload.customer.email,
       payload.items[0].sku
FROM   bronze.orders;
```

A missing path returns `NULL` rather than raising an error — convenient, and a good reason to validate rather than assume.

Unnesting arrays

An array becomes rows by putting it in the `FROM` clause. This is how one order document becomes one row per line:

```
SELECT o.payload.order_id::BIGINT        AS order_id,
       o.payload.customer.email::VARCHAR AS email,
       i.sku::VARCHAR                    AS sku,
       i.qty::INT                        AS qty,
       i.price::DECIMAL(12,2)            AS price
FROM   bronze.orders o,
       o.payload.items i;
```

Note the comma join to `o.payload.items` — that is the unnest. Nested arrays chain the same way.

Always cast on the way out

```
-- wrong: comparing SUPER to a number uses PartiQL's rules, not yours
SELECT * FROM bronze.orders WHERE payload.total > 100;

-- right
SELECT * FROM bronze.orders WHERE payload.total::DECIMAL(12,2) > 100;
```

Uncast `SUPER` values compare by PartiQL semantics, which order types before values. The result is not wrong so much as *not what you meant*.

Loading JSON

```
-- from JSON files on S3
COPY bronze.orders (payload)
FROM 's3://bucket/orders/dt=2026-08-12/'
IAM_ROLE 'arn:aws:iam::...:role/redshift-loader'
FORMAT AS JSON 'auto'
SERIALIZETOJSON;

-- or build SUPER in SQL
SELECT JSON_PARSE('{"a":1,"b":[1,2,3]}') AS doc;
```

The cost: it is not columnar

A `SUPER` column is stored **whole**. Filtering on a field inside it reads the entire document — no zone maps, no block skipping, none of the machinery that makes Redshift fast (L14, L16).

That is fine for a landing table nobody reports off. It is not fine for `fct_sales_line`.

L10 · SUPER and Semi-Structured Data

The pattern: shred what you query

```
bronze.orders     payload SUPER          -- land the document whole
      ↓
silver.order_line  order_id BIGINT        -- typed, real columns
                   sku      VARCHAR(40)
                   qty      INT
                   price    DECIMAL(12,2)
```

```
INSERT INTO silver.order_line
SELECT o.payload.order_id::BIGINT,
       i.sku::VARCHAR(40),
       i.qty::INT,
       i.price::DECIMAL(12,2)
FROM   bronze.orders o, o.payload.items i;
```

A gold table should contain no `SUPER` at all.

Try it

```
-- what shape is the document, really?
SELECT DISTINCT JSON_TYPEOF(payload.items) FROM bronze.orders LIMIT 10;

-- how often is a field missing?
SELECT COUNT(*)                                    AS rows,
       COUNT(payload.customer.email)               AS with_email,
       COUNT(*) - COUNT(payload.customer.email)    AS missing
FROM   bronze.orders;
```

That second query is the one to run before you promise anyone a column exists.

Gotchas

- **Cast on the way out, always.**
- **No zone maps on SUPER** — a filter on a nested field scans everything.
- **A schema change upstream is silent.** A renamed field becomes `NULL`, not an error. Add a test that counts non-null on fields you depend on.
- `SUPER` **has a size limit per value** — very large documents will be rejected on load.

Checklist

- ☐ I can navigate and unnest a document from memory
- ☐ I cast every value on the way out
- ☐ I know `SUPER` has no zone maps and why that matters
- ☐ Landed documents live in bronze; typed columns live in silver
- ☐ No gold table contains `SUPER`
- ☐ I have a test that catches an upstream field disappearing

You've got it when you can…

…take an unfamiliar API payload, land it as `SUPER`, and produce a typed silver table from it in one statement — then explain why leaving it as `SUPER` would have been slower and more fragile.

MODULE 01 · LESSON 11

Three Things Called “View”

A view, a late-binding view and a materialized view behave completely differently. Choosing wrong makes deployments painful.

1 · WHAT IT IS

One re-runs the query, one defers binding, one stores the answer

The difference that will bite you first is binding: an ordinary view locks its underlying tables, so you cannot drop and rebuild a table beneath it.

2 · HOW IT WORKS

THREE OBJECTS, THREE JOBS

VIEW — `bound at create time`

Re-runs its query every time. Depends on the tables underneath, so dropping one of them fails until the view is dropped too.

LATE-BINDING VIEW — `WITH NO SCHEMA BINDING`

Resolved at query time instead. You can drop and rebuild the tables beneath it freely — which is exactly what a nightly rebuild does.

MATERIALIZED VIEW — `AUTO REFRESH YES`

Stores the result and refreshes it. Fast to read, and exactly as stale as its last refresh. It is a cache, with all that implies.

WHAT BI SHOULD SEE — `never base tables`

Expose late-binding views or MVs to Power BI. Point a report at a base table and you have frozen your schema by accident.

```
late-binding for contracts · materialized for speed
```

3 · THE SQL

```
CREATE VIEW rpt.sales_daily AS
SELECT ... FROM gold.fct_sales_line
WITH NO SCHEMA BINDING;

CREATE MATERIALIZED VIEW rpt.mv_sales
AUTO REFRESH YES AS
SELECT sale_date, sum(net_amount) ...;

REFRESH MATERIALIZED VIEW rpt.mv_sales;
```

4 · GOTCHAS

A late-binding view over a dropped table fails at query time, not at deploy time.
A stale MV is a wrong dashboard, silently.

alarm on refresh AGE, not failure

IN PLAIN ENGLISH

A view is a recipe re-cooked to order. A materialized view is the dish kept warm. Late binding just means the recipe names ingredients, not shelves.

L11 · Three Things Called "View"

Slide: `_render/L11-views-and-materialized.html`

What it is

Three objects, three behaviours. Choosing wrong makes deployments painful in a way that is hard to diagnose later.

1 · Ordinary view — bound at create time

```
CREATE VIEW rpt.sales_daily AS
SELECT sale_date, SUM(net_amount) AS net
FROM   gold.fct_sales_line
GROUP  BY 1;
```

Re-runs its query every time. **It depends on the tables underneath**, so:

```
DROP TABLE gold.fct_sales_line;
-- ERROR: cannot drop table because other objects depend on it
```

That is fatal for a nightly rebuild pattern where the table is dropped and recreated.

2 · Late-binding view — resolved at query time

```
CREATE VIEW rpt.sales_daily AS
SELECT sale_date, SUM(net_amount) AS net
FROM   gold.fct_sales_line
GROUP  BY 1
WITH NO SCHEMA BINDING;
```

The view no longer depends on the table. You can drop and rebuild anything beneath it freely — which is exactly what a nightly load does.

This is the default choice for anything BI connects to. It makes the view a *contract*: the shape stays stable while you restructure underneath.

The trade: if the underlying table is genuinely missing, you find out **at query time**, not at deploy time.

3 · Materialized view — stores the answer

```
CREATE MATERIALIZED VIEW rpt.mv_sales_daily
AUTO REFRESH YES
AS
SELECT sale_date, store_sk, SUM(net_amount) AS net
FROM   gold.fct_sales_line
GROUP  BY 1, 2;

-- or refresh it yourself, at the end of the load
REFRESH MATERIALIZED VIEW rpt.mv_sales_daily;
```

Fast to read, and **exactly as stale as its last refresh**. It occupies storage. It is a cache, with everything that implies.

Where the query shape allows, Redshift refreshes **incrementally** — only the changed rows. Complex shapes fall back to a full recompute, so check which you are getting if refresh cost matters.

Which to use

NEED	USE
A stable contract for BI	**late-binding view**
An expensive aggregation queried all day	**materialized view**
Something inside a load, over stable tables	ordinary view is fine
Data that must be exactly current	neither — query the table

L11 · Three Things Called "View"

Try it

```
-- what views exist and are they late-binding?
SELECT schemaname, viewname, definition
FROM   pg_views
WHERE  schemaname NOT IN ('pg_catalog', 'information_schema');

-- materialized views and their freshness
SELECT database_name, schema_name, name,
       is_stale, autorefresh, last_refresh_time
FROM   svv_mv_info
ORDER  BY last_refresh_time;

-- what does a late-binding view break on?
SELECT * FROM rpt.sales_daily LIMIT 1;   -- fails here, not at deploy
```

`svv_mv_info.is_stale` and `last_refresh_time` are what you alarm on.

The failure that does not announce itself

A stale materialized view is a wrong dashboard — the query succeeds, returns fast, and gives yesterday's answer.

Alarm on **refresh age**, not just refresh failure:

```
SELECT name,
       last_refresh_time,
       DATEDIFF(hour, last_refresh_time, GETDATE()) AS hours_old
FROM   svv_mv_info
WHERE  DATEDIFF(hour, last_refresh_time, GETDATE()) > 26;
```

A refresh that quietly stopped running three days ago and never errored is far more dangerous than one that failed loudly.

Gotchas

- **BI must never point at base tables.** Point it at views and you can restructure freely.
- `AUTO REFRESH` **costs compute you did not schedule.** Fine, but know it is happening.
- **Not every query can be materialized** — some constructs are rejected at create time.
- **Late-binding views hide typos** until someone runs them.

Checklist

- ☐ Everything BI touches is a view, never a base table
- ☐ Views exposed to BI are `WITH NO SCHEMA BINDING`
- ☐ I know which MVs refresh incrementally and which do not
- ☐ I alarm on MV refresh **age**, not just failure
- ☐ I can list stale MVs in one query

You've got it when you can…

…be shown a dashboard returning yesterday's numbers with no errors anywhere, and check `svv_mv_info.last_refresh_time` first.

MODULE 01 · LESSON 12

External Schemas

Querying data you never loaded. An external schema is a pointer — the table appears, the bytes stay where they are.

1 · WHAT IT IS

Ordinary SQL, against storage Redshift does not own

You join an external table to a local one in a single statement. No load job, no second copy — and the cost moves from storage to every query.

2 · HOW IT WORKS

TWO KINDS, POINTING AT TWO DIFFERENT THINGS

SPECTRUM → S3 — `FROM DATA CATALOG`

Reads files in the lake through the Glue catalog: Parquet, ORC, CSV, JSON, Avro — plus Iceberg, Hudi CoW and Delta.

FEDERATED → LIVE DATABASE — `FROM POSTGRES / MYSQL`

Reads a live RDS or Aurora PostgreSQL or MySQL database as it is right now. Filters are pushed down to the source.

GOVERNANCE MOVES — `Lake Formation`

Redshift GRANT governs local tables. Anything read through the catalog is governed by Lake Formation instead — two systems, one query.

READ, MOSTLY — `verify before promising`

Spectrum's read support is broad. Its write support is narrow and format-dependent — check for your case rather than assuming symmetry.

```
a pointer is not free — the work moves, it does not vanish
```

3 · THE SQL

```
CREATE EXTERNAL SCHEMA lake_ext
FROM DATA CATALOG
DATABASE 'silver'
IAM_ROLE 'arn:aws:iam::...:role/spectrum';

-- then join it like anything else
SELECT f.*, s.region
FROM gold.fct_sales_line f
JOIN lake_ext.dim_store s USING(store_sk);
```

4 · GOTCHAS

"Works in Athena, not in Redshift" is almost always the two grant systems disagreeing. Never point at production for an hourly report.

know which system grants what

IN PLAIN ENGLISH

A library card, not a photocopy. You can read the book any time — but you walk to the other building every single time you open it.

L12 · External Schemas

Slide: `_render/L12-external-schemas.html`

What it is

An external schema is a **pointer, not a copy**. The table appears in Redshift and you query it with ordinary SQL, joined to local tables in one statement — but the bytes never move.

Which means: **the cost moves, it does not vanish.** You have swapped a storage bill and a load job for a per-query bill somewhere else.

Two kinds

Spectrum → S3

```
CREATE EXTERNAL SCHEMA lake_ext
FROM DATA CATALOG
DATABASE 'silver'
IAM_ROLE 'arn:aws:iam::123456789012:role/redshift-spectrum';
```

Reads files in the lake through the Glue Data Catalog. Formats: Parquet, ORC, CSV, JSON, Avro — plus the transactional table formats **Iceberg**, **Hudi Copy-on-Write** and **Delta Lake** (via symlink manifests).

Federated → a live database

```
CREATE EXTERNAL SCHEMA live_pg
FROM POSTGRES
DATABASE 'orders' SCHEMA 'public'
URI 'my-aurora.cluster-abc123.eu-west-1.rds.amazonaws.com'
IAM_ROLE   'arn:aws:iam::...:role/redshift-federated'
SECRET_ARN 'arn:aws:secretsmanager:...:secret:orders-ro';
```

Reads a live RDS or Aurora **PostgreSQL or MySQL** database as it is right now, with filters pushed down to the source.

Then join it like anything else

```
SELECT f.sale_date, s.region, SUM(f.net_amount) AS net
FROM   gold.fct_sales_line f
JOIN   lake_ext.dim_store  s USING (store_sk)
WHERE  f.sale_date >= '2026-01-01'
GROUP  BY 1, 2;
```

One statement, two storage systems. That is the whole appeal.

Governance moves too

This is the part that produces the most confusing bugs:

OBJECT	GOVERNED BY
Redshift-native tables	Redshift `GRANT`
Anything read via the catalog	**AWS Lake Formation**

"It works in Athena but not in Redshift" — or the reverse — is almost always these two systems disagreeing. When you debug an access problem, first establish **which system owns the object**.

L12 · External Schemas

Try it

```
-- what external schemas exist, and where do they point?
SELECT schemaname, esoptions
FROM   svv_external_schemas;

-- what tables are visible through them?
SELECT schemaname, tablename, location, input_format
FROM   svv_external_tables
ORDER  BY 1, 2;

-- how much did that external query actually scan?
SELECT query, SUM(s3_scanned_bytes) / 1024 / 1024 AS mb_scanned,
       SUM(s3scanned_rows)                        AS rows_scanned
FROM   svl_s3query_summary
WHERE  query = pg_last_query_id()
GROUP  BY 1;
```

That last query is the one that turns "Spectrum is free" into a number.

When to point rather than load

POINT AT IT	LOAD IT
scanned rarely	joined hard and often
very large, cold history	the reporting layer
you want one copy	you need sort keys and co-location

Copy what you join hard. Point at what you scan rarely. And never point a federated query at a production database for a report that runs hourly — you have just made someone else's OLTP system your reporting engine.

Gotchas

- **Spectrum's write support is narrow and format-dependent.** Read support is broad; do not assume symmetry. Verify before promising anyone a write path.
- **External tables have no zone maps of their own** — performance depends entirely on how the files are partitioned and formatted.
- **Predicate pushdown works on federated queries**, but a large scan is still a large scan against production.
- **The IAM role needs both catalog and S3 permissions**, and a KMS grant if the bucket is encrypted.

Checklist

- ☐ I can write both `CREATE EXTERNAL SCHEMA` forms
- ☐ I know which grant system governs which kind of object
- ☐ I can measure what a Spectrum query scanned
- ☐ I would refuse a federated query behind an hourly report
- ☐ I verify write support rather than assuming it

You've got it when you can…

…be asked to "just point Redshift at the production orders database" for a daily report, explain precisely what that does to production, and offer the right alternative.

MODULE 01 · LESSON 13

Users, Roles and GRANT

Who is allowed. The pattern that matters is not a grant — it is keeping the thing that writes and the thing that reads apart.

1 · WHAT IT IS

Grant to roles, never to people — and never to individual tables

Table-by-table grants become unauditable within a month. Grant on the schema, with a default privilege so new tables inherit it automatically.

2 · HOW IT WORKS

FOUR LAYERS, AND ONE PATTERN

USERS · GROUPS · ROLES — `prefer ROLE`

Roles can be granted to other roles and carry system privileges. Groups are the older mechanism. Use roles for anything new.

GRANT AT SCHEMA LEVEL — `+ ALTER DEFAULT PRIVILEGES`

USAGE on the schema, SELECT on all tables in it, and a default privilege so tomorrow's tables are covered without anyone remembering.

COLUMN AND ROW LEVEL — `CLS and RLS`

GRANT SELECT (col_a, col_b) hides a column. An RLS policy narrows which rows a role sees — both applied by the engine.

READER / WRITER SPLIT — `the pattern that matters`

The ETL role writes and is assumed by jobs, never by people. The BI role reads reporting views only. No human account holds both.

```
every grant in Terraform — an access change needs an author and a reviewer
```

3 · THE SQL

```
CREATE ROLE bi_reader;
GRANT USAGE ON SCHEMA rpt TO ROLE bi_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA rpt
TO ROLE bi_reader;

-- and cover tomorrow's tables too
ALTER DEFAULT PRIVILEGES IN SCHEMA rpt
GRANT SELECT ON TABLES TO ROLE bi_reader;
```

4 · GOTCHAS

Without ALTER DEFAULT PRIVILEGES, every new table is invisible until someone re-grants.
Revoke CREATE on public on day one.

grants clicked in a console are orphans

IN PLAIN ENGLISH

Give the job title the keys, not the person. When someone leaves you change one list, instead of searching the building for locks they could open.

L13 · Users, Roles and GRANT

Slide: `_render/L13-users-roles-grant.html`

What it is

Who is allowed. Two rules carry most of the value:

> **Grant to roles, never to people. Grant at schema level, never table by table.**

Table-by-table grants become unauditable within a month, and nobody ever dares revoke one.

Users, groups, roles

Roles can be granted to other roles and can carry system privileges. **Groups** are the older mechanism. Use roles for anything new.

```
CREATE USER etl_svc PASSWORD DISABLE;      -- IAM auth, no password
CREATE ROLE etl_writer;
CREATE ROLE bi_reader;

GRANT ROLE etl_writer TO etl_svc;
```

`PASSWORD DISABLE` forces IAM authentication — there is no password to leak, rotate or share.

The grant pattern that works

```
-- readers
GRANT USAGE  ON SCHEMA rpt TO ROLE bi_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA rpt TO ROLE bi_reader;

-- and so tomorrow's tables are covered without anyone remembering
ALTER DEFAULT PRIVILEGES IN SCHEMA rpt
  GRANT SELECT ON TABLES TO ROLE bi_reader;

-- writers
GRANT USAGE, CREATE ON SCHEMA gold TO ROLE etl_writer;
GRANT ALL ON ALL TABLES IN SCHEMA gold TO ROLE etl_writer;
ALTER DEFAULT PRIVILEGES IN SCHEMA gold
  GRANT ALL ON TABLES TO ROLE etl_writer;
```

`ALTER DEFAULT PRIVILEGES` **is the statement people forget**, and it is the reason "the new table is invisible to BI" keeps happening. Without it, every table created tomorrow needs a manual grant.

Column-level and row-level

```
-- column level: the ungranted column does not appear at all
GRANT SELECT (customer_sk, region, signup_date)
  ON gold.dim_customer TO ROLE bi_reader;

-- row level
CREATE RLS POLICY region_west
  WITH (region VARCHAR(32))
  USING (region = 'WEST');

ATTACH RLS POLICY region_west ON gold.dim_store TO ROLE west_analyst;
ALTER TABLE gold.dim_store ROW LEVEL SECURITY ON;
```

Both are applied **by the engine**, not by whoever wrote the dashboard. That is the difference between a control and a convention.

For PII — the Epsilon customer data — this is the mechanism. **Mask at the column; never keep a second filtered copy.** A copy is another thing to secure, another thing to keep in sync, and another thing to leak.

The pattern that matters most

Reader / writer separation.

L13 · Users, Roles and GRANT

- The **ETL role** writes. It is assumed by jobs, never by people.
- The **BI role** reads reporting views only — not base tables.
- **No human account holds both.**

Try it

```
-- who has what on a schema?
SELECT nspname, nspacl FROM pg_namespace WHERE nspname IN ('gold','rpt');

-- role membership
SELECT role_name, user_name FROM svv_user_grants ORDER BY 1, 2;
SELECT * FROM svv_role_grants;

-- what can a specific role actually see?
SELECT schema_name, object_name, privilege_type
FROM   svv_relation_privileges
WHERE  identity_name = 'bi_reader'
ORDER  BY 1, 2;

-- lock down the default dumping ground
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
```

Gotchas

- **Without `ALTER DEFAULT PRIVILEGES`, new tables are invisible** to everyone but their owner.
- **Grants clicked in a console are orphans** — no author, no reason, no reviewer, and nobody will ever dare remove them. Put every grant in Terraform.
- **Object ownership matters.** The owner can always drop the object regardless of grants; make schemas owned by a role, not a person.
- **Test masking from a real consumer role**, not from an admin session. An admin sees everything, which makes admin a useless place to test from.

Checklist

- ☐ Grants go to roles, never to individual users
- ☐ Grants are at schema level, with `ALTER DEFAULT PRIVILEGES` set
- ☐ `CREATE ON SCHEMA public` revoked from `PUBLIC`
- ☐ PII masked with column grants, not a filtered copy
- ☐ ETL role and BI role are separate, and no human holds both
- ☐ Every grant lives in Terraform
- ☐ Masking verified from a consumer role

You've got it when you can…

…be asked for "a copy of the customer table without the personal columns" and set up a column-level grant instead — then prove it works from the requester's own login.

MODULE 01 · LESSON 14

There Are No Indexes

The single hardest habit to unlearn. `CREATE INDEX` does not exist — and once you see what replaces it, you will not miss it.

1 · WHAT IT IS

An index helps you find a few rows. A warehouse reads a great many.

Indexes exist to avoid a full scan. Redshift **expects** to scan — so instead of an extra structure to look rows up in, it makes the scan itself cheap.

2 · WHAT REPLACES IT

FOUR MECHANISMS, NONE OF THEM AN INDEX

SORT KEY → ZONE MAPS — `skips blocks`

Redshift stores min and max per 1MB block. Sorted on `sale_date`, a query for one week reads only the blocks that could contain it.

DIST KEY → NO DATA MOVEMENT — `co-located joins`

Two tables distributed on the same key join slice-locally. The alternative is shipping rows between nodes, which is the real cost.

COLUMNAR → FEWER BYTES — `+ compression`

You only read the columns you name, compressed. Three columns of two hundred is 1.5% of the table before compression even helps.

STATISTICS → A GOOD PLAN — `ANALYZE`

The planner needs to know table sizes and value spread to choose a join strategy. Stale statistics are the closest thing to a "missing index".

```
an index avoids reading. Redshift makes reading cheap instead.
```

3 · THE TRANSLATION

```
-- Postgres instinct
CREATE INDEX ON sales (sale_date);
CREATE INDEX ON sales (store_id);

-- Redshift equivalent, at CREATE time
CREATE TABLE sales (...)
DISTKEY (store_sk) -- the join
SORTKEY (sale_date); -- the filter
```

4 · GOTCHAS

You get ONE sort key and ONE dist key per table — not fifteen indexes. Choose well. Unsorted rows lose you zone maps entirely.

one shot, at design time

IN PLAIN ENGLISH

An index is the contents page you use to find one chapter. Redshift instead keeps the whole book in date order and skips the years you did not ask for.

L14 · There Are No Indexes

Slide: `_render/L14-there-are-no-indexes.html`

What it is

```
CREATE INDEX idx_sales_date ON sales (sale_date);
-- ERROR:  syntax error at or near "INDEX"
```

There is no `CREATE INDEX` in Redshift. Not a limitation to work around — a different design.

An index exists to help you find a few rows without a full scan. A warehouse query reads a great many rows on purpose. So instead of building an extra structure to look rows up in, Redshift makes the scan itself cheap.

The four mechanisms that replace it

1 · Sort key → zone maps

Redshift stores **min and max per 1MB block**. If the table is sorted on `sale_date` and you filter on a week, it reads only the blocks whose range could contain that week and skips the rest without touching them.

That is the direct replacement for `CREATE INDEX ON sales (sale_date)`.

2 · Dist key → no data movement

Two tables distributed on the same key join **slice-locally**. The alternative is shipping rows between nodes mid-query — which is usually the single largest cost in a slow join (L29).

This is what replaces an index on a foreign key.

3 · Columnar + compression → fewer bytes

You only read the columns you name, and they are compressed. Three columns of two hundred is 1.5% of the table before compression helps at all.

4 · Statistics → a good plan

The planner needs to know table sizes and value distribution to choose a join strategy. **Stale statistics are the closest thing Redshift has to a "missing index"** — the plan collapses and nothing tells you why.

```
SELECT "table", stats_off, unsorted, tbl_rows
FROM   svv_table_info
WHERE  stats_off > 10
ORDER  BY tbl_rows DESC;

ANALYZE gold.fct_sales_line;
```

The translation

```
-- what you would have written in Postgres
CREATE INDEX ON sales (sale_date);   -- the filter
CREATE INDEX ON sales (store_id);    -- the join

-- what you write here, once, at CREATE time
CREATE TABLE gold.fct_sales_line (...)
DISTKEY (store_sk)      -- the join
SORTKEY (sale_date);    -- the filter
```

DIST for joins. SORT for filters.

The constraint that hurts

You get **one sort key and one distribution key per table** — not fifteen indexes. There is no adding another one later when a new query pattern appears.

This is why table design is a real design activity here (L20), and why it happens before you load rather than after you notice a problem.

L14 · There Are No Indexes

Prove it to yourself

```
-- 1. how much of the table is actually being read?
EXPLAIN
SELECT SUM(net_amount)
FROM   gold.fct_sales_line
WHERE  sale_date BETWEEN '2026-01-01' AND '2026-01-07';

-- 2. run it, then see how many blocks were skipped
SELECT slice, col, num_values, blocks_read, blocks_skipped
FROM   stl_scan
WHERE  query = pg_last_query_id()
  AND  blocks_read > 0
ORDER  BY blocks_skipped DESC
LIMIT  20;
```

A high `blocks_skipped` relative to `blocks_read` is your zone maps working. If `blocks_skipped` is near zero on a date-filtered query, either the table is not sorted on that column, or it has drifted unsorted and needs `VACUUM` (L42).

```
-- is the sort actually in effect?
SELECT "table", sortkey1, unsorted, tbl_rows
FROM   svv_table_info
WHERE  "schema" = 'gold'
ORDER  BY unsorted DESC;
```

`unsorted > 10` means zone maps are degrading.

Gotchas

- `unsorted` **rows lose you zone maps entirely.** A table sorted on `sale_date` that has had a month of unsorted appends stops skipping blocks.
- **A** `SORTKEY` **you never filter on is wasted**, and costs you on every load.
- **A** `DISTKEY` **on a low-cardinality column causes skew** (LO2) — worse than no `DISTKEY` at all.
- **You cannot** `ALTER` **either one.** Changing them means create-load-swap.

Checklist

- ☐ I no longer reach for `CREATE INDEX`
- ☐ I can name the four mechanisms that replace it
- ☐ **DIST for joins, SORT for filters** is automatic
- ☐ I know I get one of each, chosen at create time
- ☐ I can prove zone maps are working with `stl_scan`
- ☐ I check `unsorted` and `stats_off` before blaming SQL

You've got it when you can…

…be handed a slow warehouse query by a developer who wants to add an index, and instead show them — with `stl_scan` — how many blocks were read versus skipped, and what the table's design should have been.

MODULE 01 · LESSON 15

Distribution Styles

Where each row lands across the slices. The largest single lever on join performance — and the hardest thing to change afterwards.

1 · WHAT IT IS

A join is fast when both sides are already on the same slice

If they are not, Redshift moves rows between nodes mid-query. That movement — not the comparison — is what makes a join slow.

2 · THE FOUR STYLES

PICK ONE PER TABLE, AT CREATE TIME

KEY — big facts

Rows with the same key value land on the same slice. Two tables on the same key join locally, with no movement at all.

ALL — small dimensions

A full copy on every node. Any join to it is local. Costs storage × node count and slows writes — so only for small, slow-changing tables.

EVEN — staging tables

Round-robin. No skew, no co-location. Right when nothing joins on the table, or when no key is a sensible choice.

AUTO — the default

Redshift starts small tables as ALL and switches as they grow. A sane default, not a decision on a big fact table.

```
DISTKEY on the biggest join column · ALL for dimensions under a few million rows
```

3 · THE SQL

```sql
-- is this column a sensible DISTKEY?
SELECT COUNT(DISTINCT store_sk) AS distinct_vals,
COUNT(*) AS rows
FROM gold.fct_sales_line;

-- what is in effect, and is it skewed?
SELECT "table", diststyle, skew_rows, tbl_rows
FROM svv_table_info ORDER BY tbl_rows DESC;
```

4 · GOTCHAS

Low-cardinality DISTKEY = skew (Lesson 02).
DISTSTYLE ALL on a big table is expensive.
You cannot ALTER it — rebuild and swap.

one key per table. choose well.

IN PLAIN ENGLISH

Store the receipts and the shop details in the same building and the clerk never leaves the room. Split them and every lookup is a walk.

L15 · Distribution Styles

Slide: `_render/L15-distribution-styles.html`

What it is

Where each row physically lands across the slices. **The largest single lever on join performance**, and the hardest thing to change afterwards.

A join is fast when both sides are already on the same slice. If they are not, Redshift moves rows between nodes mid-query — and that movement, not the comparison, is what makes a join slow.

The four styles

`DISTSTYLE KEY` — for big fact tables

```
CREATE TABLE gold.fct_sales_line (...)
DISTSTYLE KEY DISTKEY (store_sk);
```

Rows with the same key value land on the same slice. Two tables distributed on the **same** key join locally with no movement at all.

Choose: the column this table is most often joined on, with **high cardinality**.

`DISTSTYLE ALL` — for small dimensions

```
CREATE TABLE gold.dim_store (...) DISTSTYLE ALL;
```

A full copy on every node, so any join to it is local. Costs storage × node count and slows writes.

Choose: dimensions under roughly a few million rows that change slowly. `dim_store`, `dim_date`, `dim_currency`.

`DISTSTYLE EVEN` — round robin

No skew, no co-location. Right when nothing joins on the table, or when no column is a sensible key. **Staging tables are usually EVEN.**

`DISTSTYLE AUTO` — the default

Redshift starts a small table as `ALL` and converts it to `EVEN` or `KEY` as it grows. A sane default — and **not a substitute for deciding on a big fact table**, where you know the join pattern and Redshift is guessing.

Choosing a DISTKEY

Three tests, in order:

1 · Is it the column you join on most? Not the primary key by reflex — the column that appears in the most, or the most expensive, joins.

2 · Is the cardinality high enough?

```
SELECT COUNT(DISTINCT store_sk) AS distinct_vals,
       COUNT(*)                AS rows,
       COUNT(*)::FLOAT / NULLIF(COUNT(DISTINCT store_sk), 0) AS rows_per_value
FROM   gold.fct_sales_line;
```

Distinct values should comfortably exceed your slice count. Fewer distinct values than slices leaves slices empty.

3 · Is the distribution even? A key with one dominant value (a `NULL`, a default, a catch-all `-1` "unknown" member) puts a huge share on one slice.

```
-- find a dominant value before you commit to the key
SELECT store_sk, COUNT(*) AS rows
FROM   gold.fct_sales_line
GROUP  BY 1
ORDER  BY 2 DESC
LIMIT  10;
```

L15 · Distribution Styles

Verify after loading

```
SELECT "table", diststyle, skew_rows, tbl_rows, size AS mb
FROM   svv_table_info
WHERE  tbl_rows > 1000000
ORDER  BY skew_rows DESC NULLS LAST;
```

`skew_rows > 4` means rethink the key. See LO2.

What "movement" looks like in a plan

```
EXPLAIN
SELECT s.region, SUM(f.net_amount)
FROM   gold.fct_sales_line f
JOIN   gold.dim_store      s USING (store_sk)
GROUP  BY 1;
```

PLAN OPERATOR	MEANS
`DS_DIST_NONE`	✅ co-located — no movement
`DS_DIST_ALL_NONE`	✅ the other side is `DISTSTYLE ALL`
`DS_DIST_INNER`	⚠️ inner table redistributed
`DS_BCAST_INNER`	🔴 inner table **broadcast to every node**

`DS_BCAST_INNER` on a large table is the classic disaster. L28 covers reading plans properly.

Changing it later

You cannot `ALTER` a `DISTKEY`. The pattern is create-load-swap:

```
CREATE TABLE gold.fct_sales_line_new (LIKE gold.fct_sales_line);
ALTER TABLE gold.fct_sales_line_new ALTER DISTKEY product_sk;   -- or recreate with new DDL

INSERT INTO gold.fct_sales_line_new SELECT * FROM gold.fct_sales_line;

BEGIN;
  ALTER TABLE gold.fct_sales_line     RENAME TO fct_sales_line_old;
  ALTER TABLE gold.fct_sales_line_new RENAME TO fct_sales_line;
COMMIT;

DROP TABLE gold.fct_sales_line_old;
```

Note the rename swap inside a transaction — readers see one table or the other, never neither.

Gotchas

- **A low-cardinality `DISTKEY` causes skew** and is worse than no key at all.
- **`DISTSTYLE ALL` on a large table** multiplies storage by node count and slows every write.
- **Join key types must match.** `VARCHAR` joined to `INT` forces a cast and defeats co-location.
- **`NULL`s all hash to the same slice.** A nullable `DISTKEY` concentrates them.

Checklist

- ☐ Every large fact has an explicit `DISTKEY`, not `AUTO`
- ☐ The key is the busiest join column, with high cardinality
- ☐ I checked for a dominant value before committing
- ☐ Small slow-changing dimensions are `DISTSTYLE ALL`
- ☐ Staging tables are `EVEN`
- ☐ I verified `skew_rows` after the first real load
- ☐ Join key types match on both sides

L15 · Distribution Styles

You've got it when you can...

...look at an `EXPLAIN` showing `DS_BCAST_INNER`, say which table is being broadcast and why, and name the distribution change that would remove it.

MODULE 01 · LESSON 16

Sort Keys and Zone Maps

How Redshift skips data it cannot possibly need. This is the mechanism that replaces an index on your WHERE clause.

1 · WHAT IT IS

Redshift stores min and max for every 1MB block, automatically

If a block's range cannot satisfy your WHERE clause, it is never read. Sorting the table on the column you filter is what makes those ranges narrow.

2 · HOW IT WORKS

FOUR THINGS THAT DECIDE WHETHER IT WORKS

SORT ON WHAT YOU FILTER — `usually the date`

For almost every retail fact this is sale_date. A sort key on a column nobody filters costs you on every load and buys nothing.

COMPOUND — `the default, and usually right`

Sorted by the columns in order. Only helps when your filter starts with the first column — exactly like a composite index.

INTERLEAVED — `rarely worth it`

Equal weight to every column, so any one of them filters well. Expensive to maintain. Use only when filters genuinely vary.

UNSORTED ROWS UNDO IT — `svv_table_info.unsorted`

New rows append unsorted. Once a table drifts, block ranges widen and skipping stops working. That is what VACUUM is for.

```
unsorted > 10% means your zone maps are already degrading
```

3 · THE SQL

```sql
-- prove the skipping is happening
SELECT SUM(blocks_read) AS read,
SUM(blocks_skipped) AS skipped
FROM stl_scan
WHERE query = pg_last_query_id();

-- has the table drifted unsorted?
SELECT "table", sortkey1, unsorted FROM svv_table_info;
```

4 · GOTCHAS

A function on the sort column defeats it:
WHERE date_trunc('month', sale_date) = ...
Filter on a range instead.

never wrap the sort key in a function

IN PLAIN ENGLISH

Filing cabinets labelled by date. Asked for last week, you walk past the drawers marked 2024 without opening them.

L16 · Sort Keys and Zone Maps

Slide: `_render/L16-sort-keys-zone-maps.html`

What it is

Redshift automatically stores the **min and max value for every 1MB block**. That metadata is a *zone map*.

When you filter, Redshift checks the zone map first. If a block's range cannot possibly satisfy your `WHERE` clause, **it is never read**. Sorting the table on the column you filter is what makes those ranges narrow enough to be useful.

This is the mechanism that replaces `CREATE INDEX` on your `WHERE` clause.

Compound vs interleaved

Compound — the default, and usually right

```
CREATE TABLE gold.fct_sales_line (...)
COMPOUND SORTKEY (sale_date, store_sk);
```

Sorted by the columns **in order**, exactly like a composite index. It helps when your filter starts with the first column:

```
WHERE sale_date >= '2026-01-01'                    -- ✅ excellent
WHERE sale_date >= '2026-01-01' AND store_sk = 4471   -- ✅ excellent
WHERE store_sk = 4471                              -- ❌ no help at all
```

Put the column you always filter on first. For retail facts that is the date.

Interleaved — rarely worth it

```
CREATE TABLE t (...) INTERLEAVED SORTKEY (a, b, c);
```

Gives equal weight to every column, so filtering on any one works reasonably. The cost is real: maintenance is expensive and requires `VACUUM REINDEX`.

Use only when filter patterns genuinely vary and you have measured that compound is not enough. Most teams that reach for it did not need it.

Prove it works

```
-- run a filtered query
SELECT SUM(net_amount)
FROM   gold.fct_sales_line
WHERE  sale_date BETWEEN '2026-01-01' AND '2026-01-07';

-- then see what it actually read
SELECT SUM(blocks_read)    AS blocks_read,
       SUM(blocks_skipped) AS blocks_skipped,
       ROUND(100.0 * SUM(blocks_skipped)
             / NULLIF(SUM(blocks_read) + SUM(blocks_skipped), 0), 1) AS pct_skipped
FROM   stl_scan
WHERE  query = pg_last_query_id();
```

High `pct_skipped` on a date-filtered query is your zone maps working. **Near zero means something is wrong** — either the table is not sorted on that column, or it has drifted unsorted.

The drift problem

New rows are appended **unsorted**. As unsorted rows accumulate, block ranges widen and skipping stops working.

```
SELECT "table", sortkey1, unsorted, tbl_rows, size AS mb
FROM   svv_table_info
WHERE  unsorted > 10
ORDER  BY tbl_rows DESC;
```

`unsorted > 10` means zone maps are degrading. The fix is `VACUUM` (L42) — or, for a table loaded in date order, the drift is naturally small because new rows already belong at the end.

L16 · Sort Keys and Zone Maps

> **Design tip:** sorting on an ever-increasing column like `sale_date` means new data lands where it belongs. Sorting on something random means every load fragments the table.

The mistake that silently defeats it

```
-- ❌ zone maps cannot help: the function hides the column
WHERE DATE_TRUNC('month', sale_date) = '2026-01-01'
WHERE EXTRACT(year FROM sale_date)   = 2026
WHERE sale_date::VARCHAR LIKE '2026-01%'

-- ✅ a plain range on the raw column
WHERE sale_date >= '2026-01-01' AND sale_date < '2026-02-01'
```

Never wrap the sort key in a function in a `WHERE` clause. This is the single most common way people accidentally turn a fast query into a full scan, and it looks completely innocent.

Try it

```
-- 1. what is each table sorted on?
SELECT "table", sortkey1, sortkey_num, unsorted
FROM   svv_table_info
WHERE  "schema" = 'gold'
ORDER  BY unsorted DESC;

-- 2. compare a good filter with a bad one, checking stl_scan after each
SELECT COUNT(*) FROM gold.fct_sales_line
WHERE sale_date >= '2026-01-01' AND sale_date < '2026-02-01';

SELECT COUNT(*) FROM gold.fct_sales_line
WHERE DATE_TRUNC('month', sale_date) = '2026-01-01';
```

Run both, check `stl_scan` after each, and look at the difference in `blocks_skipped`. That contrast is worth doing once by hand — it makes the lesson stick.

Gotchas

- **A sort key you never filter on** costs you on every load and buys nothing.
- **Interleaved needs `VACUUM REINDEX`**, which is expensive.
- **Only the first sort-key column** gets you good skipping in a compound key.
- **`unsorted` climbing after every load** means either wrong sort column or missing maintenance.

Checklist

- ☐ Every fact is sorted on the column it is filtered by, usually date
- ☐ The most-filtered column is **first** in a compound key
- ☐ I default to compound and only consider interleaved after measuring
- ☐ I never wrap the sort key in a function in `WHERE`
- ☐ I check `unsorted` regularly
- ☐ I have proved skipping with `stl_scan` at least once myself

You've got it when you can…

…take a slow query with `DATE_TRUNC` in the `WHERE`, rewrite it as a range, and show with `stl_scan` that the second version skipped 95% of the blocks the first one read.

MODULE 01 · LESSON 17

Compression and Encodings

Chosen per column, not per table. Fewer bytes on disk means fewer bytes read, so compression is a speed feature before it is a storage one.

1 · WHAT IT IS

Because storage is columnar, every column compresses on its own terms

A column of repeated country codes and a column of unique receipt numbers want completely different schemes — and in a row store you could not give them one.

2 · HOW IT WORKS

THE ENCODINGS YOU WILL ACTUALLY USE

az64 — numbers, dates, timestamps

AWS's own scheme for numeric and temporal types. Usually the best choice, and usually what AUTO picks for them.

zstd — text, wide VARCHARs

Strong general-purpose compression that works well on almost anything, and especially on text with repetition.

bytedict · runlength — few distinct values

For low-cardinality columns — status flags, country codes. Runlength is excellent when equal values sit next to each other after sorting.

raw — ON THE SORT KEY — deliberate exception

The first sort key column is often left uncompressed, so range checks against zone maps stay as cheap as possible.

```
let ANALYZE COMPRESSION tell you — do not guess from the type name
```

3 · THE SQL

```
-- ask Redshift, against real data
ANALYZE COMPRESSION gold.fct_sales_line;

-- what is in effect right now?
SELECT "column", type, encoding
FROM pg_table_def
WHERE schemaname = 'gold'
AND tablename = 'fct_sales_line';
```

4 · GOTCHAS

ANALYZE COMPRESSION takes a table lock — run it on a staging copy, not on production.
It only recommends. You still apply it.

measure on real data, not sample data

IN PLAIN ENGLISH

Packing each kind of item in the box that suits it. Shirts flat, mugs boxed — not one carton size for everything because it was simpler to order.

L17 · Compression and Encodings

Slide: `_render/L17-compression-encodings.html`

What it is

Compression is chosen **per column**, not per table — which is only possible because storage is columnar.

A column of repeated country codes and a column of unique receipt numbers want completely different schemes. In a row store you could not give them one.

Fewer bytes on disk means fewer bytes read, so compression is a speed feature before it is a storage one.

The encodings you will use

ENCODING	GOOD FOR
`az64`	numbers, dates, timestamps — AWS's own, usually best
`zstd`	text, wide `VARCHAR` s, almost anything
`bytedict`	low-cardinality columns (< 256 distinct values)
`runlength`	repeated values that sit next to each other after sorting
`delta`	slowly-incrementing integers
`raw`	deliberate: often the first sort-key column

Why the sort key is often `raw`

The first sort-key column is checked against zone maps on **every** scan. Leaving it uncompressed keeps that range check as cheap as possible — you trade a little space for faster block elimination on the hottest path.

It is a deliberate exception, not an oversight, and worth knowing so you do not "fix" it.

Let Redshift tell you

```
ANALYZE COMPRESSION gold.fct_sales_line;
```

Returns a recommended encoding per column, measured against **your actual data**. Do not guess from the type name — a `VARCHAR` full of three distinct values wants something very different from a `VARCHAR` of unique IDs.

```
-- what is in effect right now?
SELECT "column", type, encoding, distkey, sortkey
FROM   pg_table_def
WHERE  schemaname = 'gold'
  AND  tablename  = 'fct_sales_line'
ORDER  BY "column";

-- which tables have any encoding at all?
SELECT "table", encoded, size AS mb, tbl_rows
FROM   svv_table_info
WHERE  encoded = 'N' AND size > 500
ORDER  BY size DESC;
```

That last query finds large uncompressed tables — usually created by a `CTAS` that nobody gave encodings to.

L17 · Compression and Encodings

Applying it

```
-- on an existing column, for supported transitions
ALTER TABLE gold.fct_sales_line
  ALTER COLUMN net_amount ENCODE az64;

-- or at create time, which is cleaner
CREATE TABLE gold.fct_sales_line (
    sale_date   DATE           NOT NULL ENCODE raw,      -- sort key
    store_sk    BIGINT         NOT NULL ENCODE az64,
    product_sk  BIGINT         NOT NULL ENCODE az64,
    country     VARCHAR(2)              ENCODE bytedict,
    net_amount  DECIMAL(14,2)           ENCODE az64,
    notes       VARCHAR(500)            ENCODE zstd
)
DISTKEY (store_sk)
COMPOUND SORTKEY (sale_date);
```

COPY and automatic compression

When you `COPY` into an **empty** table with no encodings declared, Redshift samples the incoming data and applies compression automatically. That is usually good.

It does **not** happen when the table already has rows, or when you have declared encodings yourself. So a table built by an early `CTAS` and grown by appends can end up entirely uncompressed without anyone noticing.

Try it

```
-- 1. build a staging copy and ask for recommendations
CREATE TABLE staging.sales_sample (LIKE gold.fct_sales_line);
INSERT INTO staging.sales_sample
SELECT * FROM gold.fct_sales_line WHERE sale_date > DATEADD(day, -7, GETDATE());

ANALYZE COMPRESSION staging.sales_sample;

-- 2. compare sizes before and after applying the recommendations
SELECT "table", size AS mb, tbl_rows, encoded
FROM   svv_table_info
WHERE  "table" IN ('fct_sales_line', 'sales_sample');
```

Gotchas

- `ANALYZE COMPRESSION` **takes a table lock.** Run it on a staging copy, never on a production table mid-day.
- **It only recommends.** You still have to apply the encodings.
- A `CTAS` **inherits nothing** unless you say so — it is the most common source of uncompressed tables.
- **Sample data gives bad recommendations.** Run it against realistic volumes and real value distributions.

Checklist

- ☐ Large tables are encoded — I have checked `encoded = 'N'`
- ☐ Encodings came from `ANALYZE COMPRESSION`, not from guessing
- ☐ The first sort-key column is `raw` deliberately
- ☐ I ran `ANALYZE COMPRESSION` on a staging copy, not production
- ☐ `CTAS`-created tables were given encodings explicitly
- ☐ I re-check after a big change in data shape

You've got it when you can…

…find every large uncompressed table in a schema with one query, and produce a measured recommendation for the worst one without touching production.

MODULE 01 · LESSON 18

Constraints Are Hints, Not Rules

PRIMARY KEY does not prevent duplicates. It is informational, the planner reads it, and nothing enforces it — ever.

1 · WHAT IT IS

Only NOT NULL is enforced. Everything else is advice to the planner.

Enforcing uniqueness would mean checking every slice on every insert. Redshift declines that trade — and hands the responsibility to you instead.

2 · HOW IT WORKS

WHAT IS ENFORCED, AND WHAT IS NOT

NOT NULL — ENFORCED — the only one

A NULL into a NOT NULL column fails the statement. Use it on every key column, deliberately.

PRIMARY KEY · UNIQUE — NOT ENFORCED — informational

Insert the same key twice and both rows are stored. No error, no warning, and a doubled number in a report tomorrow.

DECLARE THEM ANYWAY — the planner uses them

The optimiser trusts them to eliminate joins and choose plans. A wrong declaration is worse than none — it can produce wrong results.

TEST INSTEAD — dbt unique / not_null

Uniqueness becomes a test that runs after every load, not a promise the database makes. That is the habit to build on day one.

```
if you need uniqueness, you have to check for it yourself
```

3 · THE SQL

```
-- this succeeds. twice.
INSERT INTO dim_store VALUES (1,'S001');
INSERT INTO dim_store VALUES (1,'S001');

-- so run this after every load
SELECT store_sk, COUNT(*) AS n
FROM gold.dim_store
GROUP BY 1 HAVING COUNT(*) > 1;
```

4 · GOTCHAS

A load that ran twice is the usual cause.
A false PRIMARY KEY can make the optimiser drop a join and return wrong rows.

never declare a key you have not tested

IN PLAIN ENGLISH

A sign saying “staff only” is not a locked door. It tells people what to expect — it does not stop anyone walking through.

L18 · Constraints Are Hints, Not Rules

Slide: `_render/L18-constraints-are-hints.html`

What it is

```
CREATE TABLE gold.dim_store (
  store_sk BIGINT PRIMARY KEY,
  store_id VARCHAR(20) UNIQUE,
  region   VARCHAR(32) NOT NULL
);

INSERT INTO gold.dim_store VALUES (1, 'S001', 'WEST');
INSERT INTO gold.dim_store VALUES (1, 'S001', 'WEST');
-- both succeed. no error. no warning.

SELECT COUNT(*) FROM gold.dim_store;   -- 2
```

Only `NOT NULL` is enforced. `PRIMARY KEY`, `UNIQUE` and `FOREIGN KEY` are informational.

Why

Enforcing uniqueness means checking every slice on every insert — a cross-node round trip per row. In an MPP system that would destroy the bulk-load performance the whole design exists to provide.

Redshift declines that trade and hands the responsibility to you. It is a deliberate engineering decision, not an omission.

Declare them anyway — but only if they are true

The optimiser **trusts** these declarations. It uses them to eliminate redundant joins and choose plans.

That cuts both ways:

> A **`PRIMARY KEY` you declared but did not enforce can cause wrong results**, because the planner may drop a join it believes cannot change the row count.

So: declare them where they are genuinely true, and **test that they stay true**.

The tests you actually run

```
-- 1. duplicate keys in a dimension
SELECT store_sk, COUNT(*) AS n
FROM   gold.dim_store
GROUP  BY 1
HAVING COUNT(*) > 1;

-- 2. duplicate merge keys in a fact — the load-ran-twice check
SELECT merge_key, COUNT(*) AS n
FROM   gold.fct_sales_line
WHERE  sale_date = '2026-08-12'
GROUP  BY 1
HAVING COUNT(*) > 1
LIMIT  20;

-- 3. orphaned foreign keys — rows pointing at a dimension member that is gone
SELECT COUNT(*) AS orphans
FROM   gold.fct_sales_line f
LEFT   JOIN gold.dim_store s USING (store_sk)
WHERE  s.store_sk IS NULL;

-- 4. the fast smoke test: does the row count match the distinct key count?
SELECT COUNT(*)                  AS rows,
       COUNT(DISTINCT merge_key) AS distinct_keys
FROM   gold.fct_sales_line;
```

Test 4 is the cheapest and catches most incidents. If those two numbers differ, you have duplicates.

In dbt

This is where it belongs — a test that runs after every build:

L18 · Constraints Are Hints, Not Rules

```yaml
models:
  - name: dim_store
    columns:
      - name: store_sk
        tests: [unique, not_null]
      - name: region
        tests: [not_null]

  - name: fct_sales_line
    columns:
      - name: merge_key
        tests: [unique, not_null]
      - name: store_sk
        tests:
          - not_null
          - relationships:
              to: ref('dim_store')
              field: store_sk
```

`relationships` is your foreign key. It is a test, not a constraint — which is the correct shape for this database.

Where duplicates come from

Almost always one of three:

1. **A load ran twice** — the fix is an idempotent `MERGE` on a key (L24), not a manual delete.
2. **A join fanned out** — a many-to-many you thought was one-to-many. Check the dimension for duplicates first.
3. **A Type 2 dimension without `is_current`** — multiple versions of a row are correct; joining without filtering is not.

Gotchas

- **No error is ever raised.** You find out from a doubled number in a report.
- **A wrong declaration is worse than none** — it can change results, not just performance.
- `NOT NULL` *is* **enforced**, so use it generously on key columns.
- `IDENTITY` **columns are not gap-free** across slices. Do not use them as a business key.

Checklist

- ☐ I know only `NOT NULL` is enforced
- ☐ Every dimension has a uniqueness test that runs after each load
- ☐ Every fact has a uniqueness test on its merge key
- ☐ Foreign keys are tested with `relationships`, not trusted
- ☐ I declare constraints only where I have verified they hold
- ☐ The `COUNT(*)` vs `COUNT(DISTINCT key)` check is in my routine

You've got it when you can…

…be told a dashboard number doubled overnight, run one query to confirm duplicates, and name the three likely causes before looking at any code.

MODULE 01 · LESSON 19

AUTO — When To Let Redshift Decide

Redshift can choose distribution, sort and encoding for you, and change its mind later. Useful — and not a substitute for knowing your own join pattern.

1 · WHAT IT IS

Automatic Table Optimization watches your queries and rewrites the table

It observes real workload, recommends a distribution and sort key, and applies them in the background. It is genuinely good — and it needs time and evidence.

2 · HOW IT WORKS

WHAT AUTO ACTUALLY DOES

DISTSTYLE AUTO — `ALL → EVEN → KEY`

Starts a small table as ALL, converts to EVEN as it grows, and may adopt a KEY once it sees a repeated join.

SORTKEY AUTO — `from observed filters`

Picks a sort column from the predicates it sees repeatedly, and reorganises the table in the background.

ENCODE AUTO — `on first COPY`

Samples the incoming data on a COPY into an empty table and applies compression. This one is nearly always right.

IT NEEDS TIME AND EVIDENCE — `days, not minutes`

Before it has seen a workload it guesses. On a big fact table you already know the join column — state it rather than wait.

```
AUTO where you do not know · explicit where you do
```

3 · THE SQL

```
-- what does Redshift want to change?
SELECT type, database, table_name,
group_id, ddl, auto_eligible
FROM svv_alter_table_recommendations;

-- what has it already done?
SELECT * FROM svl_auto_worker_action
ORDER BY eventtime DESC LIMIT 50;
```

4 · GOTCHAS

Reorganisation runs in the background and consumes capacity you did not schedule.
Stating a key disables AUTO for that table.

do not benchmark a table AUTO is rewriting

IN PLAIN ENGLISH

Autopilot is excellent on a route it has flown before. On the first flight, and on the one that matters most, somebody should still be flying.

L19 · AUTO — When To Let Redshift Decide

Slide: `_render/L19-auto-optimization.html`

What it is

Automatic Table Optimization (ATO) watches your actual queries, recommends a distribution and sort key, and applies them in the background.

It is genuinely good. It is also not a substitute for knowing your own join pattern on a table you designed yourself.

What AUTO does, per setting

`DISTSTYLE AUTO`

Starts a small table as `ALL`, converts it to `EVEN` as it grows, and may adopt a `KEY` once it has seen a repeated join. The transitions happen in the background.

`SORTKEY AUTO`

Picks a sort column from predicates it sees repeatedly, then reorganises the table.

`ENCODE AUTO`

Samples incoming data on a `COPY` into an **empty** table and applies compression. **This one is nearly always right** — there is little reason to override it unless you have measured something better.

Where AUTO is the right answer

- **Early development**, before the query pattern exists
- **Small and medium tables** where the cost of being wrong is low
- **Tables with unpredictable access** — ad-hoc analysis, exploratory schemas
- **Compression, almost always**

Where to state it explicitly

- **Large fact tables.** You already know the join column and the filter column. Telling Redshift is faster than waiting for it to infer, and you avoid it inferring something else.
- **Anything with a hard SLA.** ATO's background reorganisation is not something you want happening during a load window.
- **Tables where you have measured.** Once you know, encode the knowledge in the DDL where the next person can read it.

> **AUTO where you do not know. Explicit where you do.**

Try it

```
-- what does Redshift want to change, and why?
SELECT type, database, table_name, group_id, ddl, auto_eligible
FROM   svv_alter_table_recommendations
ORDER  BY table_name;

-- what has it already done, unprompted?
SELECT eventtime, "type", status, table_id
FROM   svl_auto_worker_action
ORDER  BY eventtime DESC
LIMIT  50;

-- which tables are still on AUTO?
SELECT "table", diststyle, sortkey1, tbl_rows
FROM   svv_table_info
WHERE  diststyle LIKE 'AUTO%'
ORDER  BY tbl_rows DESC;
```

That last query is worth running on any inherited warehouse — a large fact still sitting on `AUTO(EVEN)` is usually an oversight rather than a decision.

L19 · AUTO — When To Let Redshift Decide

Reading a recommendation

`svv_alter_table_recommendations` gives you the exact `ddl` it would apply. You can:

- **Let it apply automatically** (the default for eligible tables), or
- **Take the DDL and apply it deliberately**, in a migration, with a review

The second is better for anything important. It puts the decision in version control instead of leaving it to a background process nobody watched.

Gotchas

- **Background reorganisation consumes capacity you did not schedule.** On Serverless that is RPUs you pay for.
- **Do not benchmark a table ATO is actively rewriting.** Results will not be reproducible, and you will draw the wrong conclusion.
- **Stating a `DISTKEY` or `SORTKEY` turns AUTO off for that table.** That is the point — but know you now own it.
- **ATO needs days of workload**, not minutes, before its recommendations mean anything.

Checklist

- ☐ Large facts have explicit `DISTKEY` and `SORTKEY`, not `AUTO`
- ☐ Compression is left to `AUTO` unless measured otherwise
- ☐ I have checked `svv_table_info` for large tables still on `AUTO`
- ☐ I read `svv_alter_table_recommendations` before accepting changes
- ☐ I do not benchmark during background reorganisation
- ☐ Deliberate decisions live in the DDL, in version control

You've got it when you can…

…inherit a warehouse, list every large table still on `AUTO`, and say for each whether that is a reasonable default or an unmade decision.

MODULE 01 · LESSON 20

Designing A Fact And Its Dimensions

Everything in Part C, applied once, end to end. Six decisions in order — and the first one is a sentence, not a keyword.

1 · WHAT IT IS

State the grain out loud before you type CREATE TABLE

"one row = one line on one receipt" ← everything below follows from this sentence

2 · THE SIX DECISIONS, IN ORDER

DO THEM IN THIS ORDER OR YOU WILL REDO THEM

1 · GRAIN · 2 · COLUMNS — measures + foreign keys

One row equals one what? Then: the numbers you sum, and the surrogate keys pointing at the dimensions you slice by.

3 · DISTKEY — THE BUSIEST JOIN — high cardinality

Not the primary key by reflex. The column this table is most often joined on, with enough distinct values to spread evenly.

4 · SORTKEY — THE ALWAYS-FILTER — the date, first

The column in every WHERE clause. Sorting on an ever-increasing date also means new rows land where they belong.

5 · DIMENSIONS · 6 · TESTS — ALL, then prove it

Small dimensions get DISTSTYLE ALL so every join is local. Then a uniqueness test, because nothing else will check it.

DIST for joins · SORT for filters · ALL for small dimensions

3 · THE SQL

```
CREATE TABLE gold.fct_sales_line (
sale_date DATE NOT NULL ENCODE raw,
store_sk BIGINT NOT NULL ENCODE az64,
product_sk BIGINT NOT NULL ENCODE az64,
net_amount DECIMAL(14,2) ENCODE az64,
merge_key VARCHAR(64) NOT NULL
)
DISTKEY(store_sk) SORTKEY(sale_date);
```

4 · GOTCHAS

Mixing two grains in one fact guarantees double counting nobody can explain. Verify skew after the first real load.

grain first. always.

IN PLAIN ENGLISH

Decide what one row means before you decide anything else. Every later argument about a number is really an argument about that sentence.

L20 · Designing A Fact And Its Dimensions

Slide: `_render/L20-designing-a-fact.html`

What it is

Everything in Part C, applied once, end to end. Six decisions **in order** — and the first one is a sentence, not a keyword.

1 · State the grain

> **"One row = one line on one receipt."**

Say it out loud before typing `CREATE TABLE`. Every later argument about a number is really an argument about this sentence.

One grain per fact table. Mixing "one row = one receipt line" with "one row = one receipt" in the same table guarantees double counting that nobody can explain six months later.

2 · Choose the columns

Three kinds, and nothing else:

- **Measures** — numbers you sum: `quantity`, `net_amount`, `vat_amount`
- **Foreign keys** — surrogate keys to dimensions: `store_sk`, `product_sk`, `date_sk`
- **Degenerate dimensions** — identifiers with no attributes of their own: `receipt_no`

Plus operational columns: `merge_key` for idempotent loads (L24), `loaded_at` for lineage.

3 · Choose the DISTKEY — the busiest join

Not the primary key by reflex. The column this table is **most often joined on**, with enough distinct values to spread evenly across slices.

```
-- check cardinality and look for a dominant value
SELECT COUNT(DISTINCT store_sk) AS distinct_vals, COUNT(*) AS rows
FROM   staging.sales_line;

SELECT store_sk, COUNT(*) FROM staging.sales_line
GROUP BY 1 ORDER BY 2 DESC LIMIT 10;
```

4 · Choose the SORTKEY — the always-filter

The column in every `WHERE` clause. For retail facts, `sale_date`.

Sorting on an **ever-increasing** column also means new rows land where they belong, so the table drifts unsorted slowly rather than fragmenting on every load (L16).

5 · Dimensions get DISTSTYLE ALL

Small, slow-changing dimensions get a full copy on every node, so every join to them is local.

6 · Then test it

Nothing else will check uniqueness (L18).

L20 · Designing A Fact And Its Dimensions

The complete example

```
-- ── dimensions ──────────────────────────────────────────
CREATE TABLE gold.dim_date (
    date_sk      INTEGER      NOT NULL,
    full_date    DATE         NOT NULL,
    fiscal_year  SMALLINT     NOT NULL,
    fiscal_week  SMALLINT     NOT NULL,
    day_name     VARCHAR(9)   NOT NULL,
    is_weekend   BOOLEAN      NOT NULL,
    PRIMARY KEY (date_sk)
)
DISTSTYLE ALL
SORTKEY (date_sk);

CREATE TABLE gold.dim_store (
    store_sk     BIGINT       NOT NULL,
    store_id     VARCHAR(20)  NOT NULL,   -- business key
    store_name   VARCHAR(120),
    region       VARCHAR(32),
    valid_from   TIMESTAMP    NOT NULL,   -- Type 2
    valid_to     TIMESTAMP,
    is_current   BOOLEAN      NOT NULL,
    PRIMARY KEY (store_sk)
)
DISTSTYLE ALL
SORTKEY (store_sk);

-- ── the fact ────────────────────────────────────────────
CREATE TABLE gold.fct_sales_line (
    sale_date    DATE           NOT NULL ENCODE raw,     -- sort key: leave raw
    date_sk      INTEGER        NOT NULL ENCODE az64,
    store_sk     BIGINT         NOT NULL ENCODE az64,    -- dist key
    product_sk   BIGINT         NOT NULL ENCODE az64,
    receipt_no   VARCHAR(32)             ENCODE zstd,    -- degenerate dim
    line_no      SMALLINT                ENCODE az64,
    quantity     DECIMAL(12,3)           ENCODE az64,
    net_amount   DECIMAL(14,2)           ENCODE az64,
    vat_amount   DECIMAL(14,2)           ENCODE az64,
    merge_key    VARCHAR(64)    NOT NULL ENCODE zstd,
    loaded_at    TIMESTAMP      DEFAULT SYSDATE
)
DISTSTYLE KEY
DISTKEY  (store_sk)
COMPOUND SORTKEY (sale_date, store_sk);
```

Verify after the first real load

```
-- 1. did the distribution work out?
SELECT "table", diststyle, sortkey1, skew_rows, unsorted, stats_off, size AS mb
FROM   svv_table_info
WHERE  "schema" = 'gold'
ORDER  BY tbl_rows DESC;

-- 2. is the join co-located?
EXPLAIN
SELECT s.region, SUM(f.net_amount)
FROM   gold.fct_sales_line f
JOIN   gold.dim_store      s USING (store_sk)
GROUP  BY 1;
-- want: DS_DIST_ALL_NONE or DS_DIST_NONE.  not: DS_BCAST_INNER

-- 3. are zone maps skipping?
SELECT SUM(blocks_read) AS read, SUM(blocks_skipped) AS skipped
FROM   stl_scan WHERE query = pg_last_query_id();

-- 4. is the key actually unique?
SELECT COUNT(*) AS rows, COUNT(DISTINCT merge_key) AS keys
FROM   gold.fct_sales_line;
```

Those four checks, in that order, after the first load of any new fact table.

The design review questions

Ask these of any table design, including your own:

L20 · Designing A Fact And Its Dimensions

1. What is the grain, in one sentence?
2. Why that `DISTKEY`? What is the cardinality, and is there a dominant value?
3. Why that `SORTKEY`? Is it in every `WHERE` clause?
4. Which dimensions are `ALL`, and are they small enough to stay that way?
5. What tests uniqueness?
6. What would have to happen for this design to be wrong, and how would we notice?

Gotchas

- **Two grains in one fact** — guaranteed double counting.
- **A `DISTKEY` chosen because it was the PK** rather than because it is joined on.
- **A `SORTKEY` on a column nobody filters** — pure cost.
- **Forgetting the fourth check.** Duplicates are silent (L18).
- **No `merge_key`** — the load cannot be made idempotent later without a rebuild.

Checklist

- ☐ Grain written down as a sentence, in the model documentation
- ☐ One grain per fact table
- ☐ `DISTKEY` justified by join frequency and cardinality
- ☐ `SORTKEY` is the always-filter column, first
- ☐ Small dimensions are `DISTSTYLE ALL`
- ☐ `merge_key` exists from day one
- ☐ All four post-load checks run after the first real load
- ☐ A uniqueness test exists and runs on every build

You've got it when you can…

…be handed a source table you have never seen, produce a fact-and-dimension design in twenty minutes, and defend every one of the six decisions in a review — including what would make each of them wrong.

MODULE 01 · LESSON 21

COPY In Depth

The only load path that scales. Every slice reads its own file in parallel — which is why the number of files matters as much as the SQL.

1 · WHAT IT IS

Many files in, read in parallel, one file per slice at a time

One enormous file is read by one slice while the rest idle. Aim for a multiple of your slice count, each roughly 100MB–1GB compressed.

2 · HOW IT WORKS

FOUR THINGS THAT DECIDE HOW FAST IT LOADS

FILE COUNT AND SIZE — `multiple of slice count`

The single biggest factor. Sixteen slices and one file means fifteen idle workers; sixteen files means all of them working.

FORMAT — `PARQUET > CSV`

Parquet is typed and columnar, so there is no parsing and no guessing. CSV needs delimiters, quoting and NULL handling declared.

A MANIFEST FOR EXACTNESS — `MANIFEST`

A prefix loads whatever happens to be there. A manifest names the exact files — which is what makes a load reproducible.

COMPRESSION ON FIRST LOAD — `empty table only`

COPY into an empty table with no encodings samples the data and applies compression. Into a populated table it does not.

```
one big file is the most common reason a load is slow
```

3 · THE SQL

```
COPY staging.sales_line
FROM 's3://bucket/raw/sales/dt=2026-08-12/'
IAM_ROLE 'arn:aws:iam::...:role/rs-loader'
FORMAT AS PARQUET;

-- CSV needs more said out loud
COPY t FROM 's3://.../' IAM_ROLE '...'
CSV IGNOREHEADER 1 GZIP
TIMEFORMAT 'auto' BLANKSASNULL;
```

4 · GOTCHAS

COPY is not idempotent — running it twice loads the rows twice. Stage, then MERGE.

A prefix picks up files you did not expect.

never COPY straight into a live table

IN PLAIN ENGLISH

Sixteen people at the loading bay. Send one enormous crate and fifteen of them watch; send sixteen and the lorry empties in a sixteenth of the time.

L21 · COPY In Depth

Slide: `_render/L21-copy-in-depth.html`

What it is

`COPY` is the only load path that scales. Every slice reads its **own file** in parallel — which is why the number of files matters as much as the SQL.

One enormous file is read by one slice while the rest idle. That is the most common reason a load is slow, and it has nothing to do with Redshift.

File count and size — the biggest factor

	RESULT
1 file, 16 slices	1 slice works, 15 idle
16 files, 16 slices	all 16 work
160 files, 16 slices	all work, 10 rounds — fine
100,000 tiny files	overhead dominates, slower than one big file

Target: a multiple of your slice count, each roughly **100 MB – 1 GB compressed**.

```
SELECT COUNT(*) AS slices FROM stv_slices WHERE type = 'D';
```

The formats

Parquet — prefer this

```
COPY staging.sales_line
FROM 's3://bucket/raw/sales/dt=2026-08-12/'
IAM_ROLE 'arn:aws:iam::123456789012:role/rs-loader'
FORMAT AS PARQUET;
```

Typed and columnar, so there is no parsing, no delimiter ambiguity and no date-format guessing. Column names must match, and types must be compatible.

CSV — needs more said out loud

```
COPY staging.sales_line
FROM 's3://bucket/raw/sales/dt=2026-08-12/'
IAM_ROLE 'arn:aws:iam::123456789012:role/rs-loader'
CSV
IGNOREHEADER 1
GZIP
DATEFORMAT   'auto'
TIMEFORMAT   'auto'
BLANKSASNULL
EMPTYASNULL
TRUNCATECOLUMNS;
```

OPTION	WHY
`CSV`	understands quoting — use this, not `DELIMITER ','`
`IGNOREHEADER 1`	skip the header row
`GZIP` / `BZIP2` / `ZSTD`	compressed input
`BLANKSASNULL`	an empty field becomes `NULL`, not `''`
`EMPTYASNULL`	same for empty strings
`TRUNCATECOLUMNS`	⚠ silently truncates over-long values — survey only
`MAXERROR n`	⚠ tolerate n bad rows — survey only

JSON

```
COPY bronze.orders (payload)
FROM 's3://bucket/orders/dt=2026-08-12/'
IAM_ROLE '...'
FORMAT AS JSON 'auto'
SERIALIZETOJSON;
```

L21 · COPY In Depth

A manifest, for exactness

A prefix loads **whatever happens to be there**. A manifest names the exact files:

```
{
  "entries": [
    {"url": "s3://bucket/raw/sales/part-0000.parquet", "mandatory": true},
    {"url": "s3://bucket/raw/sales/part-0001.parquet", "mandatory": true}
  ]
}
```

```
COPY staging.sales_line
FROM 's3://bucket/manifests/2026-08-12.json'
IAM_ROLE '...'
FORMAT AS PARQUET
MANIFEST;
```

This is what makes a load reproducible. If someone drops an extra file into the prefix, a prefix-based load silently includes it; a manifest-based load does not.

Automatic compression — first load only

`COPY` into an **empty** table with no declared encodings samples the incoming data and applies compression. Into a populated table, it does not (L17).

So the pattern for a new table is: create it, `COPY` once into it empty, then check what encodings it picked.

Verify every load

```
-- what did the load actually do?
SELECT query, filename, lines_scanned, is_partial
FROM   stl_load_commits
WHERE  query = pg_last_query_id()
ORDER  BY filename;

-- row count sanity
SELECT COUNT(*) FROM staging.sales_line;

-- how long, and how much was queued?
SELECT elapsed_time/1000000.0 AS secs, queue_time/1000000.0 AS queued
FROM   sys_query_history
WHERE  query_id = pg_last_query_id();
```

Gotchas

- `COPY` **is not idempotent.** Running it twice loads the rows twice. Always stage, then `MERGE` (L24).
- **Never** `COPY` **straight into a live table.** A partial failure leaves nothing, but a *successful duplicate* leaves a mess.
- **A prefix picks up files you did not expect** — use a manifest for anything that matters.
- `MAXERROR` **and** `TRUNCATECOLUMNS` **hide problems.** Use them to survey a new feed, never in production.
- **The IAM role needs** `kms:Decrypt` if the bucket is encrypted, not just `s3:GetObject`.

L21 · COPY In Depth

Checklist

- ☐ File count is a multiple of the slice count
- ☐ Files are roughly 100 MB – 1 GB compressed
- ☐ Parquet where I control the producer
- ☐ `CSV` option, not `DELIMITER`, for delimited files
- ☐ `BLANKSASNULL` and `EMPTYASNULL` set for CSV
- ☐ Manifest used where reproducibility matters
- ☐ No `MAXERROR` or `TRUNCATECOLUMNS` in production
- ☐ `COPY` targets staging, never a live table

You've got it when you can…

…be told a load takes four hours, ask two questions — how many files, and how many slices — and fix it before looking at anything else.

MODULE 01 · LESSON 22

When COPY Fails

The error message names a row, not a cause. There is a system table that tells you the actual column and value — learn it now, not at 6am.

1 · WHAT IT IS

“Check ‘stl_load_errors’ for details” — so check it

Redshift records the file, line, column, declared type and the raw value that failed. That is enough to diagnose almost every load failure in one query.

2 · THE FOUR CAUSES

ALMOST EVERY FAILURE IS ONE OF THESE

VALUE TOO LONG — bytes, not characters

A VARCHAR sized by counting letters overflows on Arabic or emoji, where one character can be four bytes (Lesson 09).

TYPE MISMATCH — empty string vs NULL

An empty CSV field is not a NULL unless you say BLANKSASNULL. A date in an unexpected format needs TIMEFORMAT.

DELIMITER INSIDE A FIELD — use CSV, not DELIMITER

A comma inside a quoted product name breaks a naive parse. The CSV option understands quoting; a raw delimiter does not.

PERMISSIONS — S3 and KMS

The IAM role needs s3:GetObject and kms:Decrypt if the bucket is encrypted. Missing KMS is the confusing one.

```
MAXERROR hides bad rows. Use it to survey, never to ship.
```

3 · THE SQL

```
-- the query to know by heart
SELECT starttime, filename, line_number,
colname, type, raw_field_value,
err_reason
FROM stl_load_errors
ORDER BY starttime DESC
LIMIT 20;
```

4 · GOTCHAS

A failed COPY rolls back entirely — the table is unchanged, which is good news.
MAXERROR 1000 silently discards 1000 rows.

never leave MAXERROR in production

IN PLAIN ENGLISH

The delivery was rejected and the note says “item 47”. There is a log by the door that says which item, which field, and what was actually written.

L22 · When COPY Fails

Slide: `_render/L22-when-copy-fails.html`

What it is

```
ERROR:  Load into table 'sales_line' failed.
        Check 'stl_load_errors' system table for details.
```

That is the whole message. It names a table, not a cause. **So check the table** — it records the file, the line, the column, the declared type and the raw value that failed.

Learn this query now rather than at 6am:

```
SELECT starttime,
       filename,
       line_number,
       colname,
       type,
       col_length,
       raw_field_value,
       err_code,
       TRIM(err_reason) AS err_reason
FROM   stl_load_errors
ORDER  BY starttime DESC
LIMIT  20;
```

The four causes

1 · Value too long — bytes, not characters

The most common failure on this data. A `VARCHAR(20)` sized by counting letters overflows on Arabic or emoji, where one character can be four bytes (LO9).

```
-- find the real width before you size the column
SELECT MAX(octet_length(store_name)) AS max_bytes,
       MAX(length(store_name))       AS max_chars
FROM   staging.stores_raw;
```

2 · Type mismatch

An **empty CSV field is not a** `NULL` unless you said `BLANKSASNULL`. A date in an unexpected format needs `DATEFORMAT` / `TIMEFORMAT`. A thousands separator in a numeric field fails.

```
COPY ... CSV BLANKSASNULL EMPTYASNULL DATEFORMAT 'auto' TIMEFORMAT 'auto';
```

3 · Delimiter inside a field

A comma inside a quoted product name breaks a naive parse.

```
COPY ... CSV;                  -- ✅ understands quoting
COPY ... DELIMITER ',';        -- ❌ does not
```

4 · Permissions

The IAM role needs `s3:GetObject` **and** `kms:Decrypt` if the bucket is encrypted. Missing KMS is the confusing one — the error talks about access, and the bucket policy looks correct.

L22 · When COPY Fails

The diagnosis workflow

```
-- 1. what failed, exactly?
SELECT filename, line_number, colname, type, raw_field_value, TRIM(err_reason)
FROM   stl_load_errors
ORDER  BY starttime DESC LIMIT 5;

-- 2. is it one bad row or a systemic problem?
SELECT colname, TRIM(err_reason) AS reason, COUNT(*) AS n
FROM   stl_load_errors
WHERE  starttime > DATEADD(hour, -1, GETDATE())
GROUP  BY 1, 2
ORDER  BY n DESC;

-- 3. what got committed before it stopped?
SELECT query, filename, lines_scanned, is_partial
FROM   stl_load_commits
WHERE  query = (SELECT MAX(query) FROM stl_load_errors);
```

Step 2 is the one that saves time: **one bad row is a data problem; a thousand identical errors is a column definition problem.**

Surveying a new feed

When you have never seen a source before, tolerate errors *once*, deliberately, to find out what is in it:

```
COPY staging.new_feed
FROM 's3://bucket/new-feed/'
IAM_ROLE '...'
CSV IGNOREHEADER 1
MAXERROR 10000;             -- SURVEY ONLY

-- then read what it complained about
SELECT colname, type, TRIM(err_reason) AS reason,
       COUNT(*) AS n, MIN(raw_field_value) AS example
FROM   stl_load_errors
WHERE  starttime > DATEADD(minute, -10, GETDATE())
GROUP  BY 1, 2, 3
ORDER  BY n DESC;
```

Then fix the table definition and **remove** `MAXERROR`. Leaving it in production means silently discarding rows and reporting on incomplete data.

Good news about failure

A failed `COPY` rolls back entirely. The table is unchanged. You are diagnosing a non-event, not repairing damage — which is a much better position than a half-applied load.

That is also the argument for `COPY`-into-staging: a failure there touches nothing anyone queries.

Gotchas

- `stl_load_errors` **has a retention window.** Query it while the failure is fresh.
- `MAXERROR 1000` **silently discards up to 1000 rows** and reports success.
- `TRUNCATECOLUMNS` **silently shortens values** — the load succeeds and the data is wrong.
- **The row it names may not be the only bad one** — check whether it is systemic.

L22 · When COPY Fails

Checklist

- ☐ I know the `stl_load_errors` query without looking it up
- ☐ I check whether a failure is one row or systemic
- ☐ `BLANKSASNULL` and `EMPTYASNULL` are set on CSV loads
- ☐ I use `CSV`, never `DELIMITER`, on quoted data
- ☐ The load role has `kms:Decrypt` as well as `s3:GetObject`
- ☐ No `MAXERROR` or `TRUNCATECOLUMNS` survives into production
- ☐ Column widths measured from `octet_length` on real data

You've got it when you can…

…be handed "the load failed" and produce the column name, the offending value and the reason in one query — then say whether it is a data problem or a schema problem.

MODULE 01 · LESSON 23

INSERT, UPDATE, DELETE — And Their Cost

There is no in-place update. Every UPDATE is a delete plus an insert, and the old row stays on disk until you reclaim it.

1 · WHAT IT IS

Rows are never modified in place. They are marked deleted and rewritten.

A DELETE does not free space either — it flags rows. Until VACUUM runs you are still storing them, and scans still walk past them.

2 · HOW IT WORKS

WHAT EACH STATEMENT ACTUALLY COSTS

SINGLE-ROW INSERT — never in a loop

Every insert touches blocks across every column. A million of them takes hours and leaves the table fragmented. Use COPY.

UPDATE = DELETE + INSERT — doubles the space, briefly

The old version is flagged and a new row appended — unsorted. Updating a large fraction of a table degrades its sort order badly.

DELETE JUST FLAGS — space returns after VACUUM

Rows are marked, not removed. Storage and scan cost stay until maintenance runs (Lesson 42).

TRUNCATE IS INSTANT — and cannot be rolled back

Emptying a staging table? TRUNCATE, not DELETE. It frees the space immediately and costs almost nothing.

```
set-based statements only. if you are writing a loop, stop.
```

3 · THE SQL

```
-- wrong: 1,000,000 statements
INSERT INTO t VALUES (...); -- in a loop

-- right: one set-based statement
INSERT INTO gold.fct_sales_line
SELECT ... FROM staging.sales_line;

-- emptying staging
TRUNCATE staging.sales_line;
```

4 · GOTCHAS

TRUNCATE commits immediately — it cannot be rolled back, even inside a transaction.
A big UPDATE wrecks your sort order.

prefer rebuild-and-swap to mass UPDATE

IN PLAIN ENGLISH

You cannot edit a printed page. You cross it out and print a new one — and the crossed-out page stays in the folder until someone tidies up.

L23 · INSERT, UPDATE, DELETE — And Their Cost

Slide: `_render/L23-dml-and-its-cost.html`

What it is

Rows are never modified in place. An `UPDATE` marks the old row deleted and appends a new one. A `DELETE` only flags rows — the space is not returned until `VACUUM` runs.

For a Node developer this is the second-biggest adjustment after "no indexes". In Postgres you update a row and it is updated. Here you have created a new row and left the old one behind.

What each statement costs

Single-row `INSERT` — never in a loop

Every insert touches blocks across every column of the table. A million of them takes hours and leaves the table badly fragmented.

```
-- ❌ what an ORM does
INSERT INTO gold.fct_sales_line VALUES (...);   -- × 1,000,000

-- ✅ one set-based statement
INSERT INTO gold.fct_sales_line
SELECT sale_date, store_sk, product_sk, net_amount, merge_key
FROM   staging.sales_line;
```

If you are writing a loop that issues SQL, stop and write a `SELECT` instead.

`UPDATE` = delete + insert

The old version is flagged and a new row appended — **unsorted**. Updating a large fraction of a big table therefore wrecks its sort order and doubles its space until maintenance runs.

```
-- for a large fraction of a table, rebuild instead of updating
CREATE TABLE gold.fct_new (LIKE gold.fct_sales_line);

INSERT INTO gold.fct_new
SELECT sale_date, store_sk, product_sk,
       net_amount * 1.05 AS net_amount,     -- the "update"
       merge_key
FROM   gold.fct_sales_line;

BEGIN;
  ALTER TABLE gold.fct_sales_line RENAME TO fct_old;
  ALTER TABLE gold.fct_new        RENAME TO fct_sales_line;
COMMIT;

DROP TABLE gold.fct_old;
```

Rebuild-and-swap gives you a perfectly sorted, freshly compressed table and an atomic cutover. For anything above roughly 20% of the rows it is usually faster than the `UPDATE`.

`DELETE` just flags

```
DELETE FROM gold.fct_sales_line WHERE sale_date < '2024-01-01';
-- rows are marked. storage and scan cost stay until VACUUM.
```

```
-- how much dead space am I carrying?
SELECT "table", size AS mb, tbl_rows, pct_used, unsorted
FROM   svv_table_info
WHERE  size > 500
ORDER  BY pct_used ASC
LIMIT  20;
```

Low `pct_used` on a large table means deleted rows awaiting reclamation (L42).

`TRUNCATE` is instant

```
TRUNCATE staging.sales_line;
```

L23 · INSERT, UPDATE, DELETE — And Their Cost

Frees space immediately and costs almost nothing. **This is how you empty a staging table** — never `DELETE FROM`.

⚠️ `TRUNCATE` commits immediately and **cannot be rolled back**, even inside a transaction. That surprises people who expect Postgres behaviour.

Deleting a partition, the cheap way

If a table is sorted by date and you delete by date, the delete is cheap to find but still leaves flagged rows. For repeated partition-style deletes, prefer:

```
-- delete-then-insert in one transaction (the "delete/insert" upsert)
BEGIN;
  DELETE FROM gold.fct_sales_line
  WHERE  sale_date = '2026-08-12';

  INSERT INTO gold.fct_sales_line
  SELECT * FROM staging.sales_line
  WHERE  sale_date = '2026-08-12';
COMMIT;
```

This is idempotent for a whole day and is often simpler than a `MERGE` when the grain is a full partition. L24 covers when to use which.

Try it

```
-- 1. see the cost of row-at-a-time (do this in dev only, small n)
CREATE TEMP TABLE t (i INT);
-- compare timing of 1000 single inserts vs one INSERT ... SELECT

-- 2. watch unsorted grow after an UPDATE
SELECT "table", unsorted, tbl_rows FROM svv_table_info
WHERE "table" = 'fct_sales_line';

UPDATE gold.fct_sales_line SET net_amount = net_amount
WHERE  sale_date = '2026-08-12';

SELECT "table", unsorted, tbl_rows FROM svv_table_info
WHERE "table" = 'fct_sales_line';
```

That second experiment is worth doing once. Watching `unsorted` jump after a no-op `UPDATE` makes the lesson permanent.

Gotchas

- `TRUNCATE` **cannot be rolled back.** Not even in a transaction.
- A big `UPDATE` **wrecks sort order** and needs `VACUUM` afterwards.
- `DELETE` **does not free space** — check `pct_used`.
- **No `UPSERT` / `ON CONFLICT`** like Postgres. Use `MERGE` (L24) or delete-then-insert.
- **An ORM will do all of this wrong.** Do not point one at a warehouse.

Checklist

- ☐ I never issue row-at-a-time DML in a loop
- ☐ I use `TRUNCATE`, not `DELETE`, to empty staging
- ☐ I know `TRUNCATE` commits and cannot be rolled back
- ☐ For large-fraction changes I rebuild and swap
- ☐ I check `unsorted` and `pct_used` after heavy DML
- ☐ I know there is no `ON CONFLICT` here

You've got it when you can…

…be shown a nightly job that `UPDATE`s 80% of a fact table, explain what it does to sort order and space, and replace it with a rebuild-and-swap that runs faster and leaves the table in better shape.

MODULE 01 · LESSON 24

MERGE and Idempotent Loads

Re-running yesterday must produce exactly yesterday's table. That property turns every incident into a retry instead of a repair.

1 · WHAT IT IS

Insert-or-update on a key, in one statement

Because nothing enforces uniqueness here (Lesson 18), a second run of an appending load simply doubles the day. MERGE is what prevents that.

2 · HOW IT WORKS

FOUR THINGS THAT MAKE A LOAD RE-RUNNABLE

A REAL MERGE KEY — `deterministic`

One column that identifies a row uniquely and identically every time it is computed. Usually a hash of the natural key columns.

STAGE, THEN MERGE — `never COPY into the target`

COPY lands in staging, where you can count and validate it. Only then does it move into the table people query.

DEDUPLICATE THE SOURCE FIRST — `MERGE needs one match`

Two staging rows with the same key make the result undefined. Collapse them with ROW_NUMBER before merging.

ONE TRANSACTION — `all of it, or none`

Wrap staging into target so a reader never sees the table half-loaded, and a failure leaves nothing behind.

```
test it: run the load twice on purpose, then diff the table
```

3 · THE SQL

```
MERGE INTO gold.fct_sales_line AS t
USING staging.sales_dedup AS s
ON t.merge_key = s.merge_key
WHEN MATCHED THEN UPDATE SET
net_amount = s.net_amount,
quantity = s.quantity
WHEN NOT MATCHED THEN INSERT VALUES (
s.sale_date, s.store_sk, ... );
```

4 · GOTCHAS

Duplicate keys in the source make MERGE non-deterministic. Dedup first, every time.

A merge key built from a timestamp is not one.

idempotent, or it is not finished

IN PLAIN ENGLISH

Filing by reference number rather than adding to the pile. Deliver the same box twice and you replace the entry — you do not count it twice.

L24 · MERGE and Idempotent Loads

Slide: `_render/L24-merge-idempotent-loads.html`

The property you are buying

> **Re-running yesterday's load must produce exactly yesterday's table.**

That is idempotency, and it turns every incident into a **retry** instead of a **repair**. Sources are late, jobs die mid-run, someone needs a backfill — all routine if re-running is safe, all manual surgery if it is not.

Because nothing in Redshift enforces uniqueness (L18), an appending load run twice simply doubles the day. `MERGE` is what prevents that.

1 · A real merge key

One column that identifies a row **uniquely and identically every time it is computed**. Usually a hash of the natural key columns:

```
-- in the transform that builds staging
SELECT ...,
       MD5(receipt_no || '|' || line_no::VARCHAR || '|' || store_id) AS merge_key
FROM   raw.sales_line;
```

Rules for a merge key:

- Built only from **natural key** columns — never from a load timestamp or a row number
- Deterministic: same input row → same key, every run, forever
- `NOT NULL`, and part of the table from day one (retrofitting means a rebuild)

```
-- ✗ not a merge key — changes every run
MD5(receipt_no || GETDATE()::VARCHAR)

-- ✗ not a merge key — depends on ordering
ROW_NUMBER() OVER (ORDER BY sale_date)
```

2 · Stage, then merge

```
TRUNCATE staging.sales_line;

COPY staging.sales_line
FROM 's3://bucket/raw/sales/dt=2026-08-12/'
IAM_ROLE '...' FORMAT AS PARQUET;

-- validate before anything touches the live table
SELECT COUNT(*) AS rows,
       COUNT(DISTINCT merge_key) AS keys,
       SUM(CASE WHEN store_sk IS NULL THEN 1 ELSE 0 END) AS null_keys
FROM   staging.sales_line;
```

Never `COPY` into the table people query. Staging is where you count, validate and abandon cheaply.

3 · Deduplicate the source first

Two staging rows with the same key make `MERGE` **non-deterministic** — it is undefined which one wins.

```
CREATE TEMP TABLE sales_dedup AS
SELECT *
FROM (
  SELECT s.*,
         ROW_NUMBER() OVER (
           PARTITION BY merge_key
           ORDER BY     source_updated_at DESC, loaded_at DESC
         ) AS rn
  FROM   staging.sales_line s
)
WHERE rn = 1;
```

Pick the tie-breaker deliberately — usually "most recently updated at source".

L24 · MERGE and Idempotent Loads

4 · Merge in one transaction

```
BEGIN;

MERGE INTO gold.fct_sales_line AS t
USING sales_dedup AS s
   ON t.merge_key = s.merge_key
WHEN MATCHED THEN UPDATE SET
       sale_date  = s.sale_date,
       store_sk   = s.store_sk,
       product_sk = s.product_sk,
       quantity   = s.quantity,
       net_amount = s.net_amount,
       vat_amount = s.vat_amount,
       loaded_at  = SYSDATE
WHEN NOT MATCHED THEN INSERT (
       sale_date, store_sk, product_sk,
       quantity, net_amount, vat_amount, merge_key, loaded_at
) VALUES (
       s.sale_date, s.store_sk, s.product_sk,
       s.quantity, s.net_amount, s.vat_amount, s.merge_key, SYSDATE
);

COMMIT;
```

The alternative: delete-then-insert

When the grain is a **whole partition** (a day, a store-day), this is often simpler and faster than `MERGE`:

```
BEGIN;
  DELETE FROM gold.fct_sales_line WHERE sale_date = '2026-08-12';
  INSERT INTO gold.fct_sales_line
  SELECT ... FROM sales_dedup WHERE sale_date = '2026-08-12';
COMMIT;
```

USE `MERGE` WHEN	USE DELETE-INSERT WHEN
rows arrive scattered across dates	the load is one whole partition
updates are a minority of the batch	you replace the partition entirely
you need per-row upsert semantics	the partition boundary is clean

Both are idempotent. Both must be in one transaction.

In dbt

```
{{ config(
    materialized  = 'incremental',
    incremental_strategy = 'merge',
    unique_key    = 'merge_key',
    on_schema_change = 'fail',
    dist          = 'store_sk',
    sort          = ['sale_date']
) }}

SELECT ... FROM {{ ref('stg_sales_line') }}
{% if is_incremental() %}
WHERE sale_date >= (SELECT MAX(sale_date) FROM {{ this }})
{% endif %}
```

`unique_key` is the merge key. `on_schema_change='fail'` means an upstream column change is a decision, not a surprise.

Test it — properly

Run the load twice on purpose, then diff:

```
-- before
SELECT COUNT(*) AS rows, COUNT(DISTINCT merge_key) AS keys,
       SUM(net_amount) AS total
FROM   gold.fct_sales_line WHERE sale_date = '2026-08-12';

-- ... run the whole load again ...
```

L24 · MERGE and Idempotent Loads

Do this at table two, not at table fifty. An idempotency claim nobody tested is a hope.

Gotchas

- **Duplicate keys in the source make `MERGE` non-deterministic.** Dedup first, every time.
- **A merge key containing a timestamp is not a merge key.**
- `MERGE` **still generates deleted rows** for the updated ones — heavy merges need `VACUUM` (L42).
- **No `ON CONFLICT`.** `MERGE` or delete-insert, nothing else.
- **Retrofitting a merge key means a rebuild** — put it in the DDL on day one (L20).

Checklist

- ☐ Every fact has a deterministic `merge_key`, from natural keys only
- ☐ `COPY` lands in staging, never in the live table
- ☐ The source is deduplicated with an explicit tie-breaker before merging
- ☐ The merge runs in one transaction
- ☐ I know when delete-insert is the better shape
- ☐ **I have run the load twice on purpose and diffed the result**
- ☐ dbt models set `unique_key` and `on_schema_change='fail'`

You've got it when you can…

…be told at 7am that last night's load ran twice, check the merge key and the row/key/total triple, and say within a minute whether there is a problem at all.

MODULE 01 · LESSON 25

Transactions, Locks and Isolation

Redshift is serializable by default. That produces one error message that confuses everybody — and it is telling you something real.

1 · WHAT IT IS

ERROR 1023: Serializable isolation violation on table ...

Two transactions touched the same table in an order that cannot be made to look sequential. Redshift aborts one rather than produce a result neither of you meant.

2 · HOW IT WORKS

FOUR THINGS TO KNOW

EVERY STATEMENT IS A TRANSACTION — `autocommit is on`

Without BEGIN, each statement commits on its own. Wrap a multi-statement load in BEGIN ... COMMIT or a failure leaves it half done.

SERIALIZABLE, NOT READ COMMITTED — `the strictest level`

Stronger than Postgres' default. Two jobs writing the same table concurrently will collide — and that is the design working, not a bug.

DDL TAKES AN EXCLUSIVE LOCK — `stv_locks`

A long-running SELECT will block an ALTER or DROP behind it, and everything queued after that. One report can stall a deployment.

SHORT TRANSACTIONS, ONE WRITER — `the actual fix`

Serialise the writes yourself — one job owns one table — and retry on 1023. Do not try to make concurrent writers work.

```
1023 means "you have two writers" — fix the schedule, not the SQL
```

3 · THE SQL

```
-- who is holding a lock right now?
SELECT l.table_id, l.lock_owner_pid,
t.relname, l.lock_mode
FROM stv_locks l
JOIN pg_class t ON t.oid = l.table_id;

-- and kill the blocker if you must
SELECT pg_terminate_backend(<pid>);
```

4 · GOTCHAS

TRUNCATE commits immediately — it cannot be rolled back, even inside BEGIN.

An idle-in-transaction session holds locks.

always COMMIT or ROLLBACK

IN PLAIN ENGLISH

Two people editing the same page at once. Redshift will not merge your guesses — it stops one of you and asks you to take turns.

L25 · Transactions, Locks and Isolation

Slide: `_render/L25-transactions-and-locks.html`

The error you will meet

```
ERROR: 1023
DETAIL: Serializable isolation violation on table - 123456,
        transactions forming the cycle are: 987, 988
```

It is not a bug and it is not random. Two transactions touched the same table in an order that cannot be made to look sequential, so Redshift aborted one rather than produce a result neither of you intended.

The fix is almost never in the SQL. It is in the schedule.

Four things to know

1 · Every statement is already a transaction

Autocommit is on. Without `BEGIN`, each statement commits on its own — so a three-statement load that fails on statement two leaves statement one applied.

```
BEGIN;
  DELETE FROM gold.fct_sales_line WHERE sale_date = '2026-08-12';
  INSERT INTO gold.fct_sales_line SELECT * FROM staging.sales_dedup;
COMMIT;
```

2 · Serializable, not read committed

Redshift's default isolation is **stricter than Postgres'**. Two jobs writing the same table concurrently will collide — reliably, not occasionally.

3 · DDL takes an exclusive lock

A long-running `SELECT` blocks an `ALTER` or `DROP` behind it, and everything queued after that blocks too. **One analyst's report can stall a deployment**, and the deployment looks hung rather than blocked.

4 · The fix is one writer per table

Serialise the writes yourself. One job owns one table. Then retry on 1023 with a short backoff — because a legitimate collision can still happen at a boundary.

Diagnosing a block

```
-- who holds a lock right now?
SELECT l.table_id, t.relname AS table_name,
       l.lock_owner_pid, l.lock_mode, l.granted
FROM   stv_locks l
JOIN   pg_class t ON t.oid = l.table_id;

-- what is that session doing?
SELECT pid, user_name, starttime, DATEDIFF(second, starttime, GETDATE()) AS secs,
       TRIM(query) AS query
FROM   stv_recents
WHERE  status = 'Running'
ORDER  BY starttime;

-- open transactions holding things up
SELECT * FROM svv_transactions ORDER BY txn_start;

-- last resort
SELECT pg_terminate_backend(<pid>);
```

`svv_transactions` is the one to check when a deploy appears to hang — look for a transaction that started twenty minutes ago and never committed.

L25 · Transactions, Locks and Isolation

Retrying 1023 from Node

```
async function withRetry(fn, attempts = 5) {
  for (let i = 0; i < attempts; i++) {
    try {
      return await fn();
    } catch (e) {
      const serializable =
        /serializable isolation/i.test(e.message ?? "") ||
        /\b1023\b/.test(e.message ?? "");
      if (!serializable || i === attempts - 1) throw e;
      await new Promise(r => setTimeout(r, 200 * 2 ** i));  // backoff
    }
  }
}
```

Retry is a safety net for boundary cases. **It is not a substitute for having one writer** — if you are retrying constantly, two jobs are fighting and you should fix the schedule.

Gotchas

- `TRUNCATE` **commits immediately** and cannot be rolled back, even inside `BEGIN`. This is the one that surprises Postgres developers.
- **An idle-in-transaction session holds its locks.** A developer who ran `BEGIN` in a client and went to lunch will block your deployment.
- `VACUUM` **cannot run inside a transaction block.**
- **Commit or roll back explicitly** in any code that opens a transaction — a leaked transaction is a leaked lock.
- **Long** `SELECT` **s block DDL**, so run migrations in a quiet window or expect to wait.

Checklist

- ☐ Multi-statement loads are wrapped in `BEGIN … COMMIT`
- ☐ Exactly one job writes each table
- ☐ 1023 is retried with backoff, and treated as a signal not a nuisance
- ☐ I know `TRUNCATE` cannot be rolled back
- ☐ I can find a blocking session in one query
- ☐ Nothing in my code can leave a transaction open

You've got it when you can…

…be told "the deployment is stuck" and find the twenty-minute-old open transaction holding the lock — then say whose session it is before terminating anything.

MODULE 01 · LESSON 26

Staging Patterns

Land it, check it, then move it. The validation gate between staging and the live table is the cheapest quality control you will ever build.

1 · WHAT IT IS

Never write directly into a table someone is querying

Staging gives you somewhere to count rows, check keys and abandon a bad batch — while abandoning it is still free and nobody has seen it.

2 · THE FOUR STEPS

LAND · VALIDATE · MOVE · CLEAN

1 · LAND — CREATE TABLE LIKE

TRUNCATE the staging table, then COPY into it. Build staging with LIKE so it inherits the target's distribution and sort.

2 · VALIDATE — THE GATE — fail here, cheaply

Row count in range, no null keys, no duplicate merge keys, business total within tolerance of the source. Any failure stops the load here.

3 · MOVE, IN ONE TRANSACTION — MERGE or rename-swap

MERGE for scattered rows; rename-swap for a full rebuild. Either way readers see the old table or the new one, never a half-loaded one.

4 · CLEAN UP — TRUNCATE, not DELETE

Empty staging afterwards. Keep the table itself so its design and grants persist — recreating it every night loses both.

```
a load that cannot be abandoned safely is not finished
```

3 · THE SQL

```
-- the rename swap, atomically
BEGIN;
ALTER TABLE gold.fct RENAME TO fct_old;
ALTER TABLE gold.fct_new RENAME TO fct;
COMMIT;
DROP TABLE gold.fct_old;

-- readers never see neither table
```

4 · GOTCHAS

A rename swap breaks bound views — use WITH NO SCHEMA BINDING (Lesson 11).
Grants follow the table, not the name.

re-grant after a swap, or check first

IN PLAIN ENGLISH

Unload at the bay, check the delivery against the note, then move it to the shelves. Nobody shops in the loading bay.

L26 · Staging Patterns

Slide: `_render/L26-staging-patterns.html`

What it is

Land it, check it, then move it. The validation gate between staging and the live table is the cheapest quality control you will ever build — and the only place where abandoning a bad batch is free.

The four steps

1 · Land

```
-- staging inherits the target's physical design
CREATE TABLE IF NOT EXISTS staging.sales_line (LIKE gold.fct_sales_line);

TRUNCATE staging.sales_line;

COPY staging.sales_line
FROM 's3://bucket/raw/sales/dt=2026-08-12/'
IAM_ROLE 'arn:aws:iam::...:role/rs-loader'
FORMAT AS PARQUET;
```

`CREATE TABLE LIKE` matters: staging inherits the distribution, sort and encodings, so the eventual move is cheap and does not redistribute.

2 · Validate — the gate

```
-- one query, four checks
SELECT
  COUNT(*)                                          AS rows,
  COUNT(DISTINCT merge_key)                         AS distinct_keys,
  SUM(CASE WHEN store_sk   IS NULL THEN 1 ELSE 0 END)   AS null_store,
  SUM(CASE WHEN sale_date  IS NULL THEN 1 ELSE 0 END)   AS null_date,
  SUM(net_amount)                                   AS total_net,
  MIN(sale_date)                                    AS min_date,
  MAX(sale_date)                                    AS max_date
FROM staging.sales_line;
```

Stop the load if any of these fail:

CHECK	FAILS WHEN
`rows` in expected range	a truncated or duplicated extract
`rows = distinct_keys`	duplicates in the source
`null_store = 0`	a join or mapping broke upstream
`min_date` / `max_date` as expected	the wrong partition was loaded
`total_net` within tolerance of source	data loss or double counting

That last one — reconciling a business total against the source system — is the check that catches what row counts miss.

3 · Move, in one transaction

Scattered rows → `MERGE` (L24).

Full rebuild → rename swap:

```
CREATE TABLE gold.fct_sales_line_new (LIKE gold.fct_sales_line);

INSERT INTO gold.fct_sales_line_new
SELECT * FROM staging.sales_line;

BEGIN;
  ALTER TABLE gold.fct_sales_line     RENAME TO fct_sales_line_old;
  ALTER TABLE gold.fct_sales_line_new RENAME TO fct_sales_line;
COMMIT;

DROP TABLE gold.fct_sales_line_old;
```

Readers see the old table or the new one, never neither. The swap is two catalogue updates — effectively instant.

L26 · Staging Patterns

4 · Clean up

```
TRUNCATE staging.sales_line;
```

Keep the table, empty it. Recreating staging every night loses its design *and* its grants — and then a reader loses access for reasons nobody connects to the load.

Temp tables vs a staging schema

	`CREATE TEMP TABLE`	`STAGING.` SCHEMA
Lifetime	the session	persistent
Visible to others	no	yes
Survives a failure for inspection	❌	✅
Needs cleanup	no	`TRUNCATE`

Use a staging schema for anything scheduled. When a nightly load fails at 3am you want the staged data still sitting there to look at. A temp table vanished with the session.

Temp tables are right for intermediate steps inside one job — like the dedup step in L24.

Gotchas

- **A rename swap breaks bound views.** Use `WITH NO SCHEMA BINDING` (L11) on anything pointing at a table you swap.
- **Grants follow the table, not the name.** After a swap, the new table has the *new* table's grants. Re-grant, or make the grant schema-level with `ALTER DEFAULT PRIVILEGES` (L13).
- `TRUNCATE` **commits immediately** — it cannot be part of a rollback (L25).
- **Do not validate after moving.** The point is to fail before anyone sees it.

Checklist

- ☐ Staging built with `CREATE TABLE LIKE`
- ☐ `TRUNCATE` before every load, never `DELETE`
- ☐ A validation gate that can stop the load
- ☐ A business total reconciled against the source, not just row counts
- ☐ The move is one transaction
- ☐ Staging is emptied, not dropped
- ☐ Views over swapped tables are late-binding
- ☐ Grants verified after the first swap

You've got it when you can…

…design a load that fails safely at 3am — leaving the live table untouched, the staged data available to inspect, and one alarm that says which check failed.

MODULE 01 · LESSON 27

UNLOAD

The exit everyone forgets. Publishing gold back to S3 as Parquet is what stops Redshift becoming the only way to reach the data.

1 · WHAT IT IS

A parallel write from every slice straight to S3

The reverse of COPY, and the right way to move millions of rows out. Dragging them through the leader node and over a client connection is not.

2 · HOW IT WORKS

FOUR OPTIONS THAT MATTER

FORMAT PARQUET — typed, columnar

Types survive the round trip and every engine can read it efficiently. Prefer it over CSV unless something downstream demands text.

PARTITION BY — Hive-style prefixes

Writes dt=2026-08-12/ style folders, so Athena and Spectrum can prune partitions when they read it back.

PARALLEL and MAXFILESIZE — many files by default

Parallel is on by default — one file per slice, which is what you want. PARALLEL OFF gives one file and is much slower.

MANIFEST and CLEANPATH — reproducible output

A manifest lists exactly what was written. CLEANPATH removes what was there before, so a re-run replaces rather than mixes.

```
every unofficial CSV export started as a missing UNLOAD
```

3 · THE SQL

```
UNLOAD ('SELECT * FROM gold.fct_sales_line
WHERE sale_date >= ''2026-08-01''')
TO 's3://bucket/published/sales/'
IAM_ROLE 'arn:aws:iam::...:role/rs-unload'
FORMAT AS PARQUET
PARTITION BY (sale_date)
MANIFEST CLEANPATH
MAXFILESIZE 256 MB;
```

4 · GOTCHAS

Quotes inside the query must be doubled.
CLEANPATH deletes the prefix first — point it at a dedicated path, never a shared one.

CLEANPATH is a delete. aim carefully.

IN PLAIN ENGLISH

Give people the right exit and they stop climbing through windows. Every hand-maintained spreadsheet in a business is a door that was never built.

L27 · UNLOAD

Slide: `_render/L27-unload.html`

What it is

A **parallel write from every slice straight to S3** — the reverse of `COPY`, and the right way to move millions of rows out of Redshift.

The wrong way is to `SELECT` them through the leader node and over a client connection, which makes the leader the bottleneck (L05) and is slower by orders of magnitude.

The statement

```
UNLOAD ('SELECT * FROM gold.fct_sales_line
         WHERE sale_date >= ''2026-08-01''')
TO 's3://bucket/published/sales/'
IAM_ROLE 'arn:aws:iam::123456789012:role/rs-unload'
FORMAT AS PARQUET
PARTITION BY (sale_date)
MANIFEST
CLEANPATH
MAXFILESIZE 256 MB;
```

Note the **doubled quotes** inside the query string — the whole query is a single-quoted literal.

The options that matter

OPTION	WHY
`FORMAT AS PARQUET`	typed and columnar; types survive the round trip
`PARTITION BY (col)`	writes `col=value/` prefixes so Athena and Spectrum can prune
`PARALLEL ON` (default)	one file per slice — leave it on
`MAXFILESIZE 256 MB`	keeps files in the good range for re-reading (L21)
`MANIFEST`	lists exactly what was written
`CLEANPATH`	⚠ **deletes the target prefix first**
`HEADER`	CSV only
`GZIP` / `ZSTD`	CSV/text output compression

`PARALLEL OFF` produces a single file and is dramatically slower. Only use it when something downstream genuinely cannot handle multiple files.

Why this lesson exists

Publishing gold back to S3 is what stops Redshift becoming the only door to the data.

Once it is in Parquet on S3, Athena can query it, Spark can read it, a data scientist can open it in a notebook — none of which costs Redshift compute or a connection.

> **Every hand-maintained spreadsheet in a business is a door that was never built.**

When someone asks for "a weekly extract", the answer is a scheduled `UNLOAD` to a prefix they can read, not a CSV you email.

L27 · UNLOAD

Verify it

```
-- what did it write?
SELECT query, path, line_count, file_size
FROM   stl_unload_log
WHERE  query = pg_last_query_id()
ORDER  BY path;

-- total rows out
SELECT SUM(line_count) AS rows_unloaded
FROM   stl_unload_log
WHERE  query = pg_last_query_id();
```

A scheduled publish

```
-- refresh yesterday's partition, replacing rather than appending
UNLOAD ('SELECT * FROM gold.fct_sales_line
         WHERE sale_date = CURRENT_DATE - 1')
TO 's3://bucket/published/sales/'
IAM_ROLE '...'
FORMAT AS PARQUET
PARTITION BY (sale_date)
ALLOWOVERWRITE;
```

`ALLOWOVERWRITE` replaces files at the same keys — which makes a re-run idempotent for that partition. `CLEANPATH` is the heavier hammer: it removes everything under the prefix first.

Gotchas

- `CLEANPATH` **is a delete.** Point it at a dedicated prefix, never a shared one, and never at a bucket root. This is the most dangerous option in the statement.
- **Quotes inside the query must be doubled.** A single quote ends the literal.
- **Partition columns are removed from the file** and encoded in the path, which is normal but surprises people reading the Parquet directly.
- **The IAM role needs** `s3:PutObject`, plus `kms:GenerateDataKey` if the bucket is encrypted.
- `UNLOAD` **cannot write to a Redshift table.** It writes to S3 only.

Checklist

- ☐ Large extracts use `UNLOAD`, never a client-side `SELECT`
- ☐ `FORMAT AS PARQUET` unless something downstream needs text
- ☐ `PARTITION BY` on the column consumers filter by
- ☐ `MAXFILESIZE` set so files are re-readable efficiently
- ☐ `CLEANPATH` only ever aimed at a dedicated prefix
- ☐ Gold is published back to S3 on a schedule
- ☐ Every recurring "please send me an extract" replaced by a prefix

You've got it when you can…

…find someone maintaining a weekly CSV by hand, replace it with a scheduled `UNLOAD` to a prefix they can read directly, and explain why that is cheaper for everyone.

MODULE 01 · LESSON 28

Reading An EXPLAIN Plan

Four operator names carry almost all the information. Two of them mean “this query is moving data between nodes” — and that is where the time goes.

1 · WHAT IT IS

Read it bottom-up, and look for the letters DS_

The cost numbers are relative units, not seconds — ignore them at first. The data-movement operators are what tell you whether the design is right.

2 · THE FOUR THAT MATTER

BEST TO WORST

DS_DIST_NONE — no movement at all

Both sides are distributed on the join key, so every slice joins its own rows locally. This is what a good design looks like.

DS_DIST_ALL_NONE — also good

The other side is DISTSTYLE ALL, so a full copy is already on every node. This is why small dimensions get ALL.

DS_DIST_INNER — redistributing

One side is being reshuffled across the network to line up with the other. Acceptable on small inputs, expensive on large ones.

DS_BCAST_INNER — the one to hunt

A whole table copied to every node. Fine for a hundred rows; a disaster for a million. This is the classic cause of a slow join.

```
search the plan for BCAST first. everything else second.
```

3 · THE SQL

```
EXPLAIN
SELECT s.region, SUM(f.net_amount)
FROM gold.fct_sales_line f
JOIN gold.dim_store s USING(store_sk)
GROUP BY 1;

-- then the actuals, after running it
SELECT * FROM svl_query_summary
WHERE query = pg_last_query_id();
```

4 · GOTCHAS

is_diskbased = true means it spilled to disk — usually oversized VARCHARs (Lesson 09).

Stale statistics produce a plausible bad plan.

cost units are not seconds

IN PLAIN ENGLISH

The plan is the route the query will drive. You are not checking the mileage — you are checking whether it crosses the city sixteen times.

L28 · Reading An EXPLAIN Plan

Slide: `_render/L28-reading-explain.html`

What it is

`EXPLAIN` shows the route the query will take. **Read it bottom-up, and look for the letters `DS_`.**

The cost numbers are relative units, not seconds — ignore them until you know what you are doing. The **data-movement operators** are what tell you whether the physical design is right.

The four operators that matter

OPERATOR	MEANS	VERDICT
`DS_DIST_NONE`	both sides already co-located	✅ ideal
`DS_DIST_ALL_NONE`	other side is `DISTSTYLE ALL`	✅ good
`DS_DIST_INNER`	one side redistributed across the network	⚠️ acceptable if small
`DS_BCAST_INNER`	a whole table copied to **every** node	🔴 hunt this

`DS_BCAST_INNER` on a hundred rows is nothing. On a million rows it is the classic cause of a slow join — and it will not show up as an error, only as time.

The workflow

```
-- 1. what is the plan?
EXPLAIN
SELECT s.region, SUM(f.net_amount)
FROM   gold.fct_sales_line f
JOIN   gold.dim_store      s USING (store_sk)
WHERE  f.sale_date >= '2026-01-01'
GROUP  BY 1;
```

Look for `DS_BCAST` or `DS_DIST`. Then run it and look at what actually happened:

```
-- 2. the actuals
SELECT query, stm, seg, step, label,
       rows, bytes, is_diskbased, workmem/1024/1024 AS mem_mb
FROM   svl_query_summary
WHERE  query = pg_last_query_id()
ORDER  BY stm, seg, step;

-- 3. did anything spill?
SELECT step, label, is_diskbased, rows
FROM   svl_query_summary
WHERE  query = pg_last_query_id() AND is_diskbased = 't';

-- 4. how many blocks did the scan actually read vs skip?
SELECT SUM(blocks_read) AS read, SUM(blocks_skipped) AS skipped
FROM   stl_scan
WHERE  query = pg_last_query_id();
```

Those four steps, in that order, diagnose most slow queries without changing a line of SQL.

`is_diskbased = true`

The step ran out of memory and spilled to disk. Three usual causes:

1. **Oversized `VARCHAR` s** (L09) — memory is reserved per row based on declared width, not actual content
2. **A hash join whose build side is too large** — often because of a broadcast
3. **A sort or aggregation over more rows than expected** — often a fan-out from a duplicated dimension row

Anything with `is_diskbased = 't'` is worth fixing. It is usually the single largest contributor to elapsed time.

L28 · Reading An EXPLAIN Plan

Stale statistics produce plausible bad plans

The planner chooses join order and strategy from statistics. If they are stale it will make a *reasonable-looking* wrong choice — typically broadcasting a table it believes is small.

```
SELECT "table", stats_off, tbl_rows
FROM   svv_table_info
WHERE  stats_off > 10
ORDER  BY tbl_rows DESC;

ANALYZE gold.fct_sales_line;
```

Always check `stats_off` before rewriting SQL. This is the closest thing Redshift has to a missing index (L14).

Worked example: finding the broadcast

```
XN HashAggregate  (cost=...)
  ->  XN Hash Join DS_BCAST_INNER  (cost=...)     ← here
        Hash Cond: (f.store_sk = s.store_sk)
        ->  XN Seq Scan on fct_sales_line f
        ->  XN Hash
              ->  XN Seq Scan on dim_store s
```

`dim_store` is being broadcast to every node. Two possible fixes, cheapest first:

1. **`ALTER TABLE gold.dim_store ALTER DISTSTYLE ALL`** — it is a small dimension; give every node its own copy permanently. The plan becomes `DS_DIST_ALL_NONE`.
2. **Align the `DISTKEY`s** — only if the dimension is too large for `ALL`.

Gotchas

- **Cost units are not seconds** and are not comparable between queries.
- **`EXPLAIN` does not execute.** For real numbers you must run the query.
- **The first run includes compilation** (L05) — run twice before drawing conclusions.
- **`svl_query_summary` has a retention window.** Query it while the run is fresh.

Checklist

- ☐ I read plans bottom-up and search for `DS_` first
- ☐ I know all four movement operators and their verdicts
- ☐ I check `stats_off` before touching SQL
- ☐ I check `is_diskbased` on every slow query
- ☐ I know cost units are not seconds
- ☐ I run a query twice before judging it

You've got it when you can…

…take a slow join, find the `DS_BCAST_INNER` in its plan, name which table is being broadcast, and fix it with a distribution change rather than a rewrite.

MODULE 01 · LESSON 29

Joins and Data Distribution

The same join can be a hundred times slower on the same data. What changes is not the SQL — it is where the rows were sitting.

1 · WHAT IT IS

Three join algorithms, and the one you get depends on physical design

You do not choose the algorithm. You choose the distribution and sort keys, and the planner chooses the algorithm from those.

2 · THE THREE ALGORITHMS

WHAT YOU GET, AND WHAT IT NEEDED

MERGE JOIN — the fastest — rare, and worth engineering for

Needs both tables distributed and sorted on the join column. Then it is a single pass down two ordered streams.

HASH JOIN — the normal one — what you will usually see

Builds a hash table from the smaller side. Perfectly good — unless the build side is too big for memory and spills to disk.

NESTED LOOP — almost always a bug — check your ON clause

Usually means a missing or non-equality join condition — an accidental cross join. Rows multiply and the query never finishes.

THE FIX HIERARCHY — in this order

1 · ANALYZE for fresh statistics · 2 · make the small side DISTSTYLE ALL · 3 · align DISTKEYs · 4 · only then rewrite the SQL.

```
mismatched key types silently defeat co-location
```

3 · THE SQL

```
-- did anything spill to disk?
SELECT query, step, label, is_diskbased,
workmem/1024/1024 AS mem_mb, rows
FROM svl_query_summary
WHERE query = pg_last_query_id()
AND is_diskbased = 't';

-- anything here is worth fixing
```

4 · GOTCHAS

Joining VARCHAR to INT forces a cast and
throws away co-location silently.
A duplicated dimension row fans out the fact.

match your key types exactly

IN PLAIN ENGLISH

Two sorted lists checked side by side is fast. Two unsorted piles in different rooms is not — and the SQL looks identical either way.

L29 · Joins and Data Distribution

Slide: `_render/L29-joins-and-distribution.html`

What it is

The same join, on the same data, can be a hundred times slower. What changes is not the SQL — it is where the rows were physically sitting.

You do not choose the join algorithm. You choose the distribution and sort keys; the planner chooses the algorithm from those.

The three algorithms

Merge join — the fastest, and rare

Needs both tables **distributed and sorted** on the join column. Then it is a single pass down two ordered streams, with no hash table and no movement.

Worth engineering for on the one or two joins that dominate your workload:

```
CREATE TABLE gold.fct_sales_line (...)
DISTKEY (store_sk) SORTKEY (store_sk);      -- both on the join column

CREATE TABLE gold.dim_store (...)
DISTKEY (store_sk) SORTKEY (store_sk);
```

Note the trade: sorting the fact on `store_sk` rather than `sale_date` costs you zone-map pruning on date filters (L16). Only do this where the join genuinely dominates.

Hash join — the normal one

Builds a hash table from the smaller side, then probes it. Perfectly good, and what you will see most of the time.

It goes wrong when the **build side is too big for memory** and spills to disk — usually because it was broadcast, or because `VARCHAR` s are oversized (L09).

Nested loop — almost always a bug

```
XN Nested Loop DS_BCAST_INNER
```

Usually means a **missing or non-equality join condition** — an accidental cross join. Rows multiply and the query never finishes.

```
-- ❌ the classic: a comma join with the condition in WHERE, and a typo
SELECT * FROM fct_sales_line f, dim_store s
WHERE  f.store_sk = f.store_sk;          -- joined to itself

-- ✅ explicit JOIN makes the mistake impossible to miss
SELECT * FROM fct_sales_line f
JOIN   dim_store s ON f.store_sk = s.store_sk;
```

Always use explicit `JOIN … ON` . A comma join hides a missing condition.

The fix hierarchy — in this order

1 · `ANALYZE` . Stale statistics cause bad plans. Cheapest possible fix.

```
SELECT "table", stats_off FROM svv_table_info WHERE stats_off > 10;
ANALYZE gold.dim_store;
```

2 · Make the small side `DISTSTYLE ALL` . Turns a broadcast into `DS_DIST_ALL_NONE` permanently.

```
ALTER TABLE gold.dim_store ALTER DISTSTYLE ALL;
```

3 · Align the `DISTKEY` s. For two large tables joined constantly, distribute both on the join column (L15). This means a rebuild.

4 · Only then rewrite the SQL. Filter earlier, aggregate before joining, reduce the row count going into the join.

Most people start at step 4. Steps 1 and 2 are faster to try and usually enough.

L29 · Joins and Data Distribution

The silent killer: mismatched key types

```
-- dim_store.store_sk  BIGINT
-- fct.store_code      VARCHAR(20)

SELECT * FROM fct f JOIN dim_store s ON f.store_code = s.store_sk::VARCHAR;
```

The cast defeats co-location entirely, and nothing warns you. **Match your key types exactly** — same type, same width — on both sides of every join.

```
-- audit join key types across a schema
SELECT tablename, "column", type
FROM   pg_table_def
WHERE  schemaname = 'gold' AND "column" LIKE '%_sk'
ORDER  BY "column", tablename;
```

Any `_sk` column with two different types across tables is a bug waiting to happen.

The fan-out

A duplicated dimension row multiplies the fact rows joined to it. The symptom is a total that is suddenly too high; the cause is upstream.

```
-- always check the dimension first
SELECT store_sk, COUNT(*) FROM gold.dim_store
GROUP BY 1 HAVING COUNT(*) > 1;
```

This is why L18's uniqueness tests exist. A join cannot fan out against a genuinely unique dimension.

Try it

```
-- 1. plan before
EXPLAIN SELECT s.region, SUM(f.net_amount)
FROM gold.fct_sales_line f JOIN gold.dim_store s USING (store_sk)
GROUP BY 1;

-- 2. change the dimension distribution
ALTER TABLE gold.dim_store ALTER DISTSTYLE ALL;

-- 3. plan after — DS_BCAST_INNER should become DS_DIST_ALL_NONE
EXPLAIN SELECT s.region, SUM(f.net_amount)
FROM gold.fct_sales_line f JOIN gold.dim_store s USING (store_sk)
GROUP BY 1;
```

Doing that once, on a real table, is worth more than reading about it.

Gotchas

- **Comma joins hide missing conditions.** Use explicit `JOIN`.
- **Mismatched key types silently defeat co-location.**
- **A duplicated dimension row fans out the fact** and inflates totals.
- `DISTSTYLE ALL` **on a large dimension** costs storage × nodes and slows writes.
- **Sorting a fact on the join key** trades away date pruning — deliberate only.

Checklist

- ☐ Always explicit `JOIN … ON`, never comma joins
- ☐ All `_sk` columns have identical types everywhere
- ☐ I follow the fix hierarchy: analyze → ALL → align → rewrite
- ☐ I check the dimension for duplicates before blaming the join
- ☐ I check `is_diskbased` on any slow join
- ☐ I have watched `DS_BCAST_INNER` become `DS_DIST_ALL_NONE` myself

L29 · Joins and Data Distribution

You've got it when you can...

...be handed a join that got slow overnight and work through the hierarchy in order — finding, most of the time, that it was stale statistics rather than anything you wrote.

MODULE 01 · LESSON 30

Window Functions

The warehouse workhorse. Anything you would have written as a loop in Node is almost always one window function here.

1 · WHAT IT IS

A calculation across related rows, without collapsing them

GROUP BY reduces many rows to one. A window keeps every row and adds the group calculation alongside it — which is what reporting almost always needs.

2 · THE FOUR YOU WILL USE DAILY

PARTITION BY = the group · ORDER BY = the sequence

ROW_NUMBER() — deduplication

Numbers rows within a partition. Keep rn = 1 and you have picked one row per key — the dedup step every load needs (Lesson 24).

LAG() and LEAD() — the previous row

The value from the row before or after. This is how you get day-on-day change without joining a table to itself.

SUM() OVER — running totals, shares

A running total with ORDER BY, or a group total without it — which is how you compute each row's share of its group in one pass.

RANK vs DENSE_RANK — ties behave differently

RANK leaves gaps after a tie (1,2,2,4); DENSE_RANK does not (1,2,2,3). Pick deliberately — the business cares which.

```
writing a loop over rows? it is a window function.
```

3 · THE SQL

```
-- dedup: one row per key
SELECT * FROM (
SELECT s.*, ROW_NUMBER() OVER (
PARTITION BY merge_key
ORDER BY updated_at DESC) AS rn
FROM staging.sales_line s)
WHERE rn = 1;

-- the most useful five lines here
```

4 · GOTCHAS

You cannot filter on a window in WHERE —
wrap it in a subquery or CTE first.
A non-deterministic ORDER BY gives random rows.

always break ties explicitly

IN PLAIN ENGLISH

Everyone keeps their seat, and each person is told their position in the queue. GROUP BY would have sent the whole queue home as one number.

L30 · Window Functions

Slide: `_render/L30-window-functions.html`

What it is

A calculation across related rows **without collapsing them**.

`GROUP BY` reduces many rows to one. A window keeps every row and adds the group calculation alongside — which is what reporting almost always needs.

> **If you are about to write a loop over rows in Node, it is a window function.**

The anatomy

```
function(...) OVER (
  PARTITION BY <the group>
  ORDER BY     <the sequence>
  ROWS BETWEEN <the frame>
)
```

- `PARTITION BY` — the group. Omit it and the whole result set is one group.
- `ORDER BY` — the sequence within the group. Required for anything positional.
- **frame** — which rows in the partition count. Defaults are subtle; be explicit when it matters.

1 · ROW_NUMBER — deduplication

The most useful five lines in this module. It is the dedup step every load needs (L24):

```
SELECT *
FROM (
  SELECT s.*,
         ROW_NUMBER() OVER (
           PARTITION BY merge_key
           ORDER BY     source_updated_at DESC, loaded_at DESC
         ) AS rn
  FROM   staging.sales_line s
)
WHERE rn = 1;
```

Always break ties explicitly. With a non-deterministic `ORDER BY`, "which row wins" changes between runs and your load stops being reproducible.

2 · LAG and LEAD — the previous row

Day-on-day change without joining a table to itself:

```
SELECT sale_date,
       store_sk,
       net,
       LAG(net) OVER (PARTITION BY store_sk ORDER BY sale_date)      AS prev_net,
       net - LAG(net) OVER (PARTITION BY store_sk ORDER BY sale_date) AS delta,
       LAG(net, 7) OVER (PARTITION BY store_sk ORDER BY sale_date)   AS same_day_last_week
FROM   gold.agg_sales_daily;
```

L30 · Window Functions

3 · SUM OVER — running totals and shares

```
SELECT sale_date, store_sk, net,

       -- running total within the store, by date
       SUM(net) OVER (PARTITION BY store_sk ORDER BY sale_date
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_net,

       -- the store's total for the whole period (no ORDER BY = whole partition)
       SUM(net) OVER (PARTITION BY store_sk) AS store_total,

       -- this day's share of the store's total, in one pass
       net / NULLIF(SUM(net) OVER (PARTITION BY store_sk), 0) AS pct_of_store,

       -- 7-day moving average
       AVG(net) OVER (PARTITION BY store_sk ORDER BY sale_date
                      ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM   gold.agg_sales_daily;
```

That `pct_of_store` line is the one to internalise. In Node you would have queried the total, then looped. Here it is one expression.

4 · RANK vs DENSE_RANK — ties matter

```
SELECT store_sk, net,
       ROW_NUMBER() OVER (ORDER BY net DESC) AS rn,    -- 1,2,3,4
       RANK()       OVER (ORDER BY net DESC) AS rnk,   -- 1,2,2,4  (gap)
       DENSE_RANK() OVER (ORDER BY net DESC) AS drnk   -- 1,2,2,3  (no gap)
FROM   gold.agg_sales_by_store;
```

Pick deliberately. "Top 10 stores" means something different under each, and the business does care which.

Top-N per group — the pattern

```
-- top 3 products per store by revenue
SELECT *
FROM (
  SELECT store_sk, product_sk, net,
         ROW_NUMBER() OVER (PARTITION BY store_sk ORDER BY net DESC) AS rn
  FROM   gold.agg_sales_by_store_product
)
WHERE rn <= 3
ORDER BY store_sk, rn;
```

Gotchas

- **You cannot filter on a window function in `WHERE`.** Wrap it in a subquery or CTE first — the window is computed after `WHERE`.
- **A non-deterministic `ORDER BY` gives you random rows** on every run. Always add a tie-breaker.
- **Frame defaults differ** between `ROWS` and `RANGE`. Be explicit for running totals.
- **`ORDER BY` inside `OVER` is not the query's `ORDER BY`** — you still need one at the end for output ordering.
- **Windows over huge partitions can spill** — check `is_diskbased` (L28).

Checklist

- ☐ I reach for a window before writing a loop
- ☐ Dedup is `ROW_NUMBER` with an explicit tie-breaker
- ☐ I know `LAG` / `LEAD` replace self-joins
- ☐ I can compute a share of group total in one pass
- ☐ I know why `WHERE` cannot filter a window
- ☐ I choose between `RANK` and `DENSE_RANK` deliberately

L30 · Window Functions

You've got it when you can...

...be shown Node code that fetches rows, loops, and computes running totals in JavaScript — and replace the whole thing with one query that returns the finished answer.

MODULE 01 · LESSON 31

CTEs, Subqueries and the Optimizer

A CTE is not a temp table. Referencing one twice can execute it twice — which is how a tidy-looking query quietly does the work three times.

1 · WHAT IT IS

WITH is readability, not materialisation

Redshift generally inlines a CTE into the surrounding query. That is usually good — and it means a CTE referenced three times may be computed three times.

2 · HOW IT WORKS

FOUR RULES OF THUMB

REFERENCED ONCE → USE A CTE — free readability

Inlined, so it costs nothing and makes a long query readable. Name the steps and the SQL documents itself.

REFERENCED TWICE → TEMP TABLE — compute once, reuse

If an expensive CTE is used more than once, materialise it yourself with CREATE TEMP TABLE and let every reference read the result.

CORRELATED SUBQUERY → REWRITE — it becomes a loop

A subquery referencing the outer row runs per row. Turn it into a join or a window function — almost always possible.

FILTER EARLY, INSIDE THE CTE — help the pruning

Put the date predicate where the table is scanned, not in the outer query. It keeps zone-map pruning available (Lesson 16).

```
check the plan — the CTE you assumed ran once may appear twice
```

3 · THE SQL

```
-- expensive, and used twice: materialise
CREATE TEMP TABLE daily AS
SELECT sale_date, store_sk,
SUM(net_amount) AS net
FROM gold.fct_sales_line
WHERE sale_date >= '2026-01-01'
GROUP BY 1, 2;
```

4 · GOTCHAS

A temp table has no stats until you ANALYZE it,
so the next join may be planned badly.
Give temp tables a DISTKEY if you join them.

ANALYZE your temp tables

IN PLAIN ENGLISH

A CTE is a name for a step, not a saved result. Ask for the same expensive step three times and you may pay for it three times.

L31 · CTEs, Subqueries and the Optimizer

Slide: `_render/L31-ctes-and-optimizer.html`

What it is

`WITH` **is readability, not materialisation.**

Redshift generally **inlines** a CTE into the surrounding query. That is usually good — the planner sees the whole thing and can optimise across it. But it means a CTE referenced three times may be *computed* three times.

That is how a tidy-looking query quietly does the expensive work repeatedly.

Four rules of thumb

1 · Referenced once → use a CTE

Free readability. Name the steps and the SQL documents itself:

```
WITH recent AS (
    SELECT * FROM gold.fct_sales_line
    WHERE  sale_date >= '2026-01-01'
),
by_store AS (
    SELECT store_sk, SUM(net_amount) AS net
    FROM   recent GROUP BY 1
)
SELECT s.region, SUM(b.net)
FROM   by_store b JOIN gold.dim_store s USING (store_sk)
GROUP  BY 1;
```

2 · Referenced twice and expensive → materialise it yourself

```
CREATE TEMP TABLE daily
  DISTKEY (store_sk)              -- give it a key if you will join it
AS
SELECT sale_date, store_sk, SUM(net_amount) AS net
FROM   gold.fct_sales_line
WHERE  sale_date >= '2026-01-01'
GROUP  BY 1, 2;

ANALYZE daily;                    -- ⚠ do not skip this

-- now every reference reads a computed result
SELECT ... FROM daily d1 JOIN daily d2 ON ...;
```

`ANALYZE` **the temp table.** It has no statistics until you do, and the planner will guess badly on the next join — this is one of the most common causes of a mysteriously slow query in an otherwise sensible script.

3 · Correlated subquery → rewrite it

A subquery referencing the outer row runs **per row**:

```
-- ✗ runs once per row of f
SELECT f.*,
       (SELECT MAX(net_amount) FROM gold.fct_sales_line x
        WHERE  x.store_sk = f.store_sk) AS store_max
FROM   gold.fct_sales_line f;

-- ✓ one pass, as a window
SELECT f.*,
       MAX(net_amount) OVER (PARTITION BY store_sk) AS store_max
FROM   gold.fct_sales_line f;
```

Almost every correlated subquery is a window function or a join in disguise.

4 · Filter early, inside the CTE

```
-- ✗ the predicate is applied after the whole table is scanned
WITH all_sales AS (SELECT * FROM gold.fct_sales_line)
SELECT * FROM all_sales WHERE sale_date >= '2026-01-01';

-- ✓ the predicate reaches the scan, so zone maps prune (L16)
WITH recent AS (
  SELECT * FROM gold.fct_sales_line WHERE sale_date >= '2026-01-01'
)
SELECT * FROM recent;
```

L31 · CTEs, Subqueries and the Optimizer

Redshift will often push the predicate down anyway — but not always, and it costs nothing to write it in the right place.

EXISTS vs IN vs JOIN

```
-- IN with a subquery: fine for small lists, watch out for NULLs
WHERE store_sk IN (SELECT store_sk FROM gold.dim_store WHERE region = 'WEST')

-- EXISTS: usually the best for "does a match exist"
WHERE EXISTS (SELECT 1 FROM gold.dim_store s
              WHERE s.store_sk = f.store_sk AND s.region = 'WEST')

-- JOIN: use when you need columns from the other table
JOIN gold.dim_store s ON s.store_sk = f.store_sk AND s.region = 'WEST'
```

⚠️ **`NOT IN` with any `NULL` in the subquery returns no rows.** This catches everyone once. Use `NOT EXISTS` instead:

```
-- ❌ silently returns nothing if any store_sk is NULL
WHERE store_sk NOT IN (SELECT store_sk FROM gold.dim_store)

-- ✅ behaves as you expect
WHERE NOT EXISTS (SELECT 1 FROM gold.dim_store s WHERE s.store_sk = f.store_sk)
```

Check what the plan actually did

```
EXPLAIN
WITH recent AS (SELECT * FROM gold.fct_sales_line WHERE sale_date >= '2026-01-01')
SELECT (SELECT COUNT(*) FROM recent), (SELECT SUM(net_amount) FROM recent);
```

Look for the scan on `fct_sales_line` appearing **twice**. If it does, the CTE was inlined into both references and you are paying twice — materialise it.

Gotchas

- **A temp table without `ANALYZE`** gets a bad plan on the next join.
- **Give temp tables a `DISTKEY`** if you will join them; otherwise they are `EVEN` and every join redistributes.
- **`NOT IN` plus `NULL` returns nothing.** Use `NOT EXISTS`.
- **Temp tables live for the session** — in the Data API each statement may be a different session, so temp tables do not persist between calls (L06).
- **Deeply nested CTEs are hard to read**, which defeats the point. Three or four levels is usually the limit.

Checklist

- ☐ CTEs for readability where referenced once
- ☐ Temp table where an expensive step is reused
- ☐ Every temp table gets `ANALYZE`, and a `DISTKEY` if joined
- ☐ No correlated subqueries — rewritten as windows or joins
- ☐ Filters written inside the CTE that scans the table
- ☐ `NOT EXISTS`, never `NOT IN`, against a nullable column
- ☐ I have checked a plan for a doubly-scanned CTE

You've got it when you can…

…take a 200-line query with six CTEs, find the one that is scanned three times, materialise it with `ANALYZE`, and cut the runtime without changing a single result.

MODULE 01 · LESSON 32

Dates, Times and Time Zones

Tamimi runs in Asia/Riyadh. Redshift runs in UTC. Every “yesterday’s sales are wrong” ticket you will ever get lives in that gap.

1 · WHAT IT IS

Store UTC. Convert once, at the reporting boundary.

A business day is a local-time concept; a timestamp is a physical instant. Keep the two apart, and label every column so nobody has to guess which it is.

2 · HOW IT WORKS

THE FOUR THINGS TO GET RIGHT

TIMESTAMP vs TIMESTAMPTZ — `pick one, everywhere`

TIMESTAMP stores no zone and is assumed UTC. TIMESTAMPTZ normalises to UTC on write. Mixing them across tables causes silent drift.

DATE_TRUNC AND THE GRAIN — `day, week, month`

DATE_TRUNC('day', ts) is how you get a reporting grain. Truncate the converted local time, never the raw UTC value.

A REAL DATE DIMENSION — `fiscal calendars live here`

Fiscal periods, Ramadan, weekends that start Friday — none of that is derivable from a timestamp. It belongs in dim_date.

SARGABLE PREDICATES ONLY — `or you lose the zone map`

Wrapping the sort column in a function defeats pruning. Compare the bare column to a computed literal range instead.

```
name it: loaded_at_utc, sale_date_local. never just 'date'.
```

3 · THE SQL

```sql
-- UTC instant to the local day
SELECT DATE_TRUNC('day',
CONVERT_TIMEZONE('UTC',
'Asia/Riyadh', sold_at_utc))::DATE
AS sale_date_local

-- sargable: bare column, literal range
WHERE sold_at_utc >= '2026-08-09T21:00Z'
```

4 · GOTCHAS

GETDATE() is UTC, not the store’s clock.
CURRENT_DATE in a WHERE makes the load non-reproducible — pass the date in.

never DATE_TRUNC the sort key

IN PLAIN ENGLISH

The warehouse keeps one clock so every store agrees on what happened first. You translate to local time once, at the very end, for the human reading the report.

L32 · Dates, Times and Time Zones

Slide: `_render/L32-dates-and-timezones.html`

What it is

Tamimi runs in **Asia/Riyadh** (UTC+3, no daylight saving). Redshift runs in **UTC**.

Every "yesterday's sales are wrong" ticket you will ever get lives in that three-hour gap.

The rule: store UTC, convert once, at the reporting boundary.

A *business day* is a local-time concept. A *timestamp* is a physical instant. Keep them apart, and name every column so nobody has to guess which one they are holding.

1 · TIMESTAMP vs TIMESTAMPTZ — pick one, everywhere

TYPE	STORES A ZONE?	ON WRITE	ON READ
`TIMESTAMP`	No	Value stored verbatim	Returned verbatim
`TIMESTAMPTZ`	No (normalised)	Converted to UTC using the session zone	Rendered in the session zone

`TIMESTAMP` is by far the more common choice in a warehouse, precisely *because* it does nothing clever — you write UTC, you read UTC, no session setting can change what you get back.

`TIMESTAMPTZ` is convenient for ingest from an application that sends offsets, but it makes the value you read depend on the reader's session zone. That is fine for a human at a console and dangerous inside an ETL job.

Mixing them across tables causes silent drift. Pick one convention for the whole warehouse and write it in the standards doc.

```
-- the convention this course teaches
sold_at_utc    TIMESTAMP     NOT NULL,   -- the instant, always UTC
sale_date      DATE          NOT NULL,   -- the LOCAL business day, precomputed
loaded_at_utc TIMESTAMP     NOT NULL DEFAULT GETDATE()
```

Precomputing `sale_date` at load time is deliberate: it is your `SORTKEY`, and it lets every downstream query filter on a bare column.

2 · Naming: the cheapest fix in this lesson

```
❌ date, time, timestamp, created, dt
✅ sold_at_utc, sale_date_local, loaded_at_utc, valid_from_utc
```

A column called `date` has caused more incorrect reports than any optimiser bug. Suffix every timestamp with `_utc` and every local-day column with `_local` or nothing-but- `_date`, and half the class of problem disappears.

3 · Converting

`CONVERT_TIMEZONE(source_zone, target_zone, timestamp)`:

```
SELECT sold_at_utc,
       CONVERT_TIMEZONE('UTC', 'Asia/Riyadh', sold_at_utc)                  AS sold_at_local,
       DATE_TRUNC('day',
         CONVERT_TIMEZONE('UTC', 'Asia/Riyadh', sold_at_utc))::DATE         AS sale_date_local
FROM   staging.sales_line;
```

Two-argument form assumes the input is UTC — `CONVERT_TIMEZONE('Asia/Riyadh', sold_at_utc)`. Prefer the three-argument form; it documents the assumption.

Truncate the converted local time, never the raw UTC value. `DATE_TRUNC('day', sold_at_utc)` gives you a UTC day, which starts at 03:00 Riyadh time — so three hours of every evening's trade lands on the wrong day.

L32 · Dates, Times and Time Zones

4 · The grain: DATE_TRUNC

```
DATE_TRUNC('day',   ts)   -- 2026-08-10 00:00:00
DATE_TRUNC('week',  ts)   -- Monday-start week
DATE_TRUNC('month', ts)   -- 2026-08-01 00:00:00
DATE_TRUNC('quarter', ts)
```

⚠️ `DATE_TRUNC('week', …)` **starts on Monday.** In Saudi Arabia the working week starts Sunday. Do not fight this in SQL — put the answer in `dim_date`.

Also useful:

```
DATEDIFF('day', a, b)          -- integer difference, note the unit comes FIRST
DATEADD('month', -1, sale_date)
LAST_DAY(sale_date)
EXTRACT(hour FROM sold_at_utc)
TO_CHAR(sale_date, 'YYYY-MM')   -- for labels only, never for filtering
```

⚠️ `DATEDIFF` **counts boundary crossings, not elapsed time.** `DATEDIFF('day', '2026-08-10 23:00', '2026-08-11 01:00')` is `1`, from two hours apart.

5 · A real date dimension

Fiscal periods, Ramadan, Eid, weekends that start Friday, retail 4-4-5 calendars — **none of that is derivable from a timestamp.** It belongs in a table.

```
CREATE TABLE gold.dim_date (
    date_key           DATE     NOT NULL,   -- the local business day
    day_of_week        SMALLINT NOT NULL,
    day_name           VARCHAR(9)  NOT NULL,
    is_weekend         BOOLEAN  NOT NULL,   -- Fri/Sat here, not Sat/Sun
    week_start_sunday  DATE     NOT NULL,
    month_num          SMALLINT NOT NULL,
    month_name         VARCHAR(9)  NOT NULL,
    quarter_num        SMALLINT NOT NULL,
    year_num           SMALLINT NOT NULL,
    fiscal_year        SMALLINT NOT NULL,
    fiscal_period      SMALLINT NOT NULL,
    hijri_date         VARCHAR(20),
    is_ramadan         BOOLEAN  NOT NULL DEFAULT FALSE,
    is_public_holiday  BOOLEAN  NOT NULL DEFAULT FALSE,
    holiday_name       VARCHAR(60)
)
DISTSTYLE ALL                  -- tiny, joined by everything
SORTKEY (date_key);
```

`DISTSTYLE ALL` is exactly right here: a few thousand rows, joined by every query. See L15.

It also acts as your date spine — the thing that lets a report show a day with zero sales, which no amount of clever SQL over the fact table can do (L33).

6 · Sargable predicates only

Wrapping the sort column in a function defeats zone-map pruning (L16). Compare the **bare column** to a computed literal range:

```
-- ❌ function on the column: full scan
WHERE DATE_TRUNC('month', sale_date) = '2026-08-01'
WHERE EXTRACT(year FROM sale_date) = 2026
WHERE TO_CHAR(sale_date, 'YYYY-MM') = '2026-08'

-- ✅ bare column, literal range: zone maps prune
WHERE sale_date >= '2026-08-01' AND sale_date < '2026-09-01'
```

The same applies when you filter a UTC timestamp for a local day — convert the *boundaries*, not the column:

```
-- the local day 2026-08-10 in Riyadh is 21:00 the previous day, UTC
WHERE sold_at_utc >= '2026-08-09 21:00:00'
  AND sold_at_utc <  '2026-08-10 21:00:00'
```

This is the single most valuable habit in this lesson.

L32 · Dates, Times and Time Zones

Gotchas

- `GETDATE()` and `SYSDATE` **return UTC**, not the store's clock. `CURRENT_DATE` is a UTC date.
- `CURRENT_DATE` **inside a load makes it non-reproducible.** Pass the business date in as a parameter so a rerun of yesterday's job loads yesterday. This matters more than it sounds — it is the difference between a pipeline you can replay and one you cannot.
- `DATE_TRUNC` **on the sort key kills pruning.** Never in a `WHERE`.
- `DATEDIFF` **counts boundary crossings.**
- `DATE_TRUNC('week')` **is Monday-based.** Use `dim_date`.
- `DATE` **minus** `DATE` **gives an integer**; `TIMESTAMP` minus `TIMESTAMP` gives an interval.
- `Asia/Riyadh` **has no DST**, which makes life easy — but if the group expands to a DST zone, hard-coded `+3` breaks and `CONVERT_TIMEZONE` does not. Never hard-code the offset.

Try it

```
-- prove the three-hour problem to yourself
SELECT DATE_TRUNC('day', sold_at_utc)::DATE                          AS utc_day,
       DATE_TRUNC('day', CONVERT_TIMEZONE('UTC','Asia/Riyadh',
                                          sold_at_utc))::DATE        AS local_day,
       COUNT(*)
FROM   staging.sales_line
WHERE  sold_at_utc >= '2026-08-09' AND sold_at_utc < '2026-08-12'
GROUP  BY 1, 2
ORDER  BY 1, 2;
```

Rows where `utc_day <> local_day` are the evening trade that a naive report attributes to the wrong day. Count them and you have the size of the bug.

Checklist

- ☐ One timestamp convention for the whole warehouse
- ☐ Every timestamp column suffixed `_utc`
- ☐ `sale_date` precomputed as the local business day, and it is the `SORTKEY`
- ☐ Conversions use three-argument `CONVERT_TIMEZONE`, never a hard-coded `+3`
- ☐ `dim_date` exists, is `DISTSTYLE ALL`, and owns the fiscal and holiday calendar
- ☐ No function ever wraps the sort column in a `WHERE`
- ☐ Loads take the business date as a parameter, not `CURRENT_DATE`

You've got it when you can…

…be told "the Riyadh 10 pm sales are showing on the wrong day" and know, before opening anything, that someone truncated a UTC timestamp.

MODULE 01 · LESSON 33

Warehouse SQL Patterns

Four shapes that cover most of what a warehouse team is ever asked to write. Learn them as patterns and you stop solving them from scratch.

1 · WHAT IT IS

The recurring shapes of analytical SQL

Application SQL is mostly lookups by key. Warehouse SQL is a small number of shapes applied over and over — each one a window function underneath.

2 · THE FOUR PATTERNS

RECOGNISE THE SHAPE, NOT THE QUESTION

DEDUPLICATION — `ROW_NUMBER, rn = 1`

Latest row per key. The first step of every load, and the answer to "why is this count double what I expect".

SCD TYPE 2 HISTORY — `valid_from, valid_to, is_current`

A dimension that keeps its past. Close the old row, open a new one — so a report run for March still shows March's prices.

GAPS AND ISLANDS — `date minus ROW_NUMBER`

Consecutive runs and the holes between them: days a store had no sales, or how long a promotion actually ran.

PIVOT AND UNPIVOT — `PIVOT, UNPIVOT, or CASE`

Rows to columns for a report, columns to rows to normalise a wide source file. Redshift has both keywords now.

```
every one of these is a window function wearing a hat
```

3 · THE SQL

```sql
-- islands: consecutive selling days
SELECT store_sk,
MIN(d) AS run_start,
MAX(d) AS run_end,
COUNT(*) AS days
FROM marked
GROUP BY store_sk, grp;
```

4 · GOTCHAS

Overlapping SCD2 validity ranges double-count history, and nothing errors.
Gaps need a date spine — missing rows cannot report themselves. **test for overlaps**

IN PLAIN ENGLISH

A carpenter does not invent a new joint for every cabinet. Four joints, chosen well, build almost everything — and these are the four.

L33 · Warehouse SQL Patterns

Slide: `_render/L33-warehouse-sql-patterns.html`

What it is

Application SQL is mostly lookups by key. Warehouse SQL is a **small number of shapes applied over and over** — and each one is a window function underneath.

Learn them as patterns and you stop solving them from scratch every time someone asks a question in a slightly new way.

Pattern 1 · Deduplication

The question: "the latest row per key". **The shape:** `ROW_NUMBER`, keep `rn = 1`.

```
-- reusable as a view over any staging table
CREATE OR REPLACE VIEW staging.v_sales_line_latest AS
SELECT *
FROM (
  SELECT s.*,
         ROW_NUMBER() OVER (
           PARTITION BY merge_key
           ORDER BY     source_updated_at DESC, source_op_seq DESC, loaded_at_utc DESC
         ) AS rn
  FROM   staging.sales_line s
)
WHERE rn = 1;
```

The tie-breaker chain matters. `source_updated_at` alone is not enough when a source system stamps two changes in the same second — add a sequence, an offset, anything monotonic.

Diagnostic version — how bad is the duplication?

```
SELECT COUNT(*)                          AS rows_in,
       COUNT(DISTINCT merge_key)         AS keys,
       COUNT(*) - COUNT(DISTINCT merge_key) AS dupes
FROM   staging.sales_line;
```

Pattern 2 · SCD Type 2 history

The question: "what was this store's region *in March*?" **The shape:** `valid_from` / `valid_to` / `is_current`.

A Type 1 dimension overwrites and loses the past. A Type 2 dimension keeps it: close the old row, open a new one. A report run for March then still shows March's prices, regions and hierarchy.

```
CREATE TABLE gold.dim_store (
    store_sk        BIGINT   IDENTITY(1,1),   -- surrogate: one per VERSION
    store_id        VARCHAR(20) NOT NULL,     -- natural key: stable across versions
    store_name      VARCHAR(120),
    region          VARCHAR(60),
    valid_from_utc  TIMESTAMP NOT NULL,
    valid_to_utc    TIMESTAMP NOT NULL DEFAULT '9999-12-31',
    is_current      BOOLEAN   NOT NULL DEFAULT TRUE,
    row_hash        VARCHAR(32) NOT NULL      -- MD5 of the tracked attributes
)
DISTSTYLE ALL
SORTKEY (store_id, valid_from_utc);
```

Two keys, and the distinction is the whole idea: **`store_id` identifies the store; `store_sk` identifies one version of it.** Facts join on `store_sk`, so a fact row is permanently attached to the version of the store that was true when it happened.

`valid_to_utc` uses `'9999-12-31'` rather than `NULL` so that `BETWEEN` works without special-casing.

L33 · Warehouse SQL Patterns

The load — close, then open

```
BEGIN;

-- 1. what arrived, deduped, with a hash of the tracked attributes
CREATE TEMP TABLE incoming DISTKEY (store_id) AS
SELECT store_id, store_name, region,
       MD5(COALESCE(store_name,'') || '|' || COALESCE(region,'')) AS row_hash,
       :batch_ts::TIMESTAMP AS effective_utc
FROM   staging.v_store_latest;

ANALYZE incoming;

-- 2. close any current row whose attributes actually changed
UPDATE gold.dim_store d
SET    valid_to_utc = i.effective_utc,
       is_current   = FALSE
FROM   incoming i
WHERE  d.store_id  = i.store_id
  AND  d.is_current
  AND  d.row_hash <> i.row_hash;

-- 3. open a new version for changed rows AND brand-new stores
INSERT INTO gold.dim_store
       (store_id, store_name, region, valid_from_utc, valid_to_utc, is_current, row_hash)
SELECT i.store_id, i.store_name, i.region, i.effective_utc, '9999-12-31', TRUE, i.row_hash
FROM   incoming i
LEFT   JOIN gold.dim_store d
       ON  d.store_id = i.store_id AND d.is_current
WHERE  d.store_id IS NULL          -- new store, or the row we just closed
   OR  d.row_hash <> i.row_hash;

COMMIT;
```

The `row_hash` is what makes this idempotent. Rerun the same batch and step 2 closes nothing (hashes match) and step 3 inserts nothing. Without it you get a new version every run, forever. Note that step 2 runs before step 3 inside one transaction, so the row closed in step 2 is no longer `is_current` when step 3's `LEFT JOIN` looks — which is why the `d.store_id IS NULL` branch catches it.

Querying it

```
-- current view: what most reports want
SELECT * FROM gold.dim_store WHERE is_current;

-- as-at: what was true on a given date
SELECT * FROM gold.dim_store
WHERE  '2026-03-15' >= valid_from_utc AND '2026-03-15' < valid_to_utc;
```

The test you run every load

```
-- 1. exactly one current row per natural key
SELECT store_id, COUNT(*) FROM gold.dim_store
WHERE is_current GROUP BY 1 HAVING COUNT(*) > 1;

-- 2. no overlapping validity ranges
SELECT a.store_id, a.store_sk, b.store_sk
FROM   gold.dim_store a
JOIN   gold.dim_store b
       ON  a.store_id = b.store_id
       AND a.store_sk < b.store_sk
       AND a.valid_from_utc < b.valid_to_utc
       AND b.valid_from_utc < a.valid_to_utc;

-- 3. no gaps in the timeline
SELECT store_id, valid_to_utc AS gap_from,
       LEAD(valid_from_utc) OVER (PARTITION BY store_id ORDER BY valid_from_utc) AS gap_to
FROM   gold.dim_store
QUALIFY gap_to IS NOT NULL AND gap_to <> valid_to_utc;
```

⚠️ **Overlapping ranges double-count history and nothing errors.** An as-at query hits two rows, the join fans out, and the total is quietly wrong. Run test 2 on every load — this is the single most important test in the whole warehouse.

L33 · Warehouse SQL Patterns

> `QUALIFY` filters on a window function without a subquery. If your Redshift version rejects it, wrap the `LEAD` in a subquery and filter outside — same result.

Pattern 3 · Gaps and islands

The question: "which days did this store not sell anything?" or "how long did that promotion actually run?" **The shape:** `date - ROW_NUMBER()` is constant within a consecutive run.

```
WITH selling_days AS (
    SELECT DISTINCT store_sk, sale_date AS d
    FROM   gold.fct_sales_line
    WHERE  sale_date >= '2026-01-01'
),
marked AS (
    SELECT store_sk, d,
           -- the trick: subtracting a dense counter from a dense date
           -- gives the same value for every day in a consecutive run
           DATEADD('day',
             -ROW_NUMBER() OVER (PARTITION BY store_sk ORDER BY d), d) AS grp
    FROM   selling_days
)
SELECT store_sk,
       MIN(d)   AS run_start,
       MAX(d)   AS run_end,
       COUNT(*) AS days
FROM   marked
GROUP  BY store_sk, grp
ORDER  BY store_sk, run_start;
```

That is the **islands**. For the **gaps** — the days that are missing — you need a spine, because *missing rows cannot report themselves*:

```
SELECT s.store_sk, dd.date_key AS missing_day
FROM   gold.dim_date dd
CROSS  JOIN (SELECT DISTINCT store_sk FROM gold.dim_store WHERE is_current) s
LEFT   JOIN gold.fct_sales_line f
       ON f.sale_date = dd.date_key AND f.store_sk = s.store_sk
WHERE  dd.date_key BETWEEN '2026-01-01' AND '2026-08-10'
  AND  NOT dd.is_public_holiday
  AND  f.store_sk IS NULL
ORDER  BY 1, 2;
```

This is the query that catches a store whose feed silently stopped. **Build it, schedule it, alert on it** — it finds problems that no row-count check will.

Same trick with `LAG` for "time since the previous event":

```
SELECT store_sk, d,
       DATEDIFF('day', LAG(d) OVER (PARTITION BY store_sk ORDER BY d), d) AS days_since_prev
FROM   selling_days;
```

Pattern 4 · Pivot and unpivot

Rows to columns for a report:

```
-- explicit CASE: works everywhere, and you control the labels
SELECT store_sk,
       SUM(CASE WHEN channel = 'RETAIL'    THEN net_amount ELSE 0 END) AS retail,
       SUM(CASE WHEN channel = 'ONLINE'    THEN net_amount ELSE 0 END) AS online,
       SUM(CASE WHEN channel = 'WHOLESALE' THEN net_amount ELSE 0 END) AS wholesale
FROM   gold.fct_sales_line
GROUP  BY 1;

-- PIVOT: shorter, same result
SELECT * FROM (
    SELECT store_sk, channel, net_amount FROM gold.fct_sales_line
) PIVOT (
    SUM(net_amount) FOR channel IN ('RETAIL', 'ONLINE', 'WHOLESALE')
);
```

Both require you to **list the columns**. There is no dynamic pivot in SQL — if the channel list changes, the query changes. That is a good reason to pivot in the BI tool rather than in the warehouse, and to keep the gold table long-and-narrow.

L33 · Warehouse SQL Patterns

Columns to rows to normalise a wide source file — a spreadsheet with one column per month is the classic:

```
SELECT store_id, month_label, amount
FROM   staging.wide_budget
UNPIVOT (amount FOR month_label IN (jan, feb, mar, apr, may, jun));
```

Gotchas

- **Overlapping SCD2 validity ranges double-count history**, and nothing errors. Test every run.
- **Gaps need a date spine.** Missing rows cannot report themselves.
- **A dedup without a deterministic tie-breaker** picks a different winner each run.
- `MD5` **over concatenated columns needs** `COALESCE` — one `NULL` makes the whole hash `NULL`, so every row looks changed. And use a separator, or `'AB'||'C'` and `'A'||'BC'` hash the same.
- **Adding a column to the SCD2 hash** makes every row look changed on the next run and doubles the dimension. Plan that migration.
- `PIVOT` **needs a literal column list.** No dynamic pivot exists.
- `CROSS JOIN` **against a spine multiplies rows.** Bound the date range.

Try it

1. Write the dedup view over one staging table and count the duplicates it removes.
2. Load `dim_store` twice with the same input. Prove the second run inserts zero rows — if it does not, your hash is wrong.
3. Change one store's region in staging, load again, and read the two versions with an as-at query for a date before and after.
4. Run the overlap test. Get zero rows.
5. Run the gap query for one store and explain each missing day: holiday, closure, or broken feed.

Checklist

- ☐ A dedup view exists per staging table, with a deterministic tie-breaker
- ☐ I know why `store_sk` and `store_id` are different columns
- ☐ SCD2 loads are hash-driven and therefore idempotent
- ☐ The overlap test and the one-current-row test run on every load
- ☐ `COALESCE` and a separator in every hash expression
- ☐ `dim_date` used as the spine for gap detection
- ☐ The gap query is scheduled and alerts on a silent feed
- ☐ Pivots kept in BI, gold tables kept long and narrow

You've got it when you can…

…be handed "our March report changed" and know it is either an SCD2 overlap or a Type 1 dimension that overwrote its history — and have the query that tells you which, before you have finished reading the ticket.

MODULE 01 · LESSON 34

Result Caching and Compiled Code

The second run is often a hundred times faster than the first. Knowing why stops you drawing the wrong conclusion when you benchmark.

1 · WHAT IT IS

Two separate caches, and they explain different things

The result cache returns a finished answer without touching the compute nodes. The compiled-code cache saves the C++ compilation step for a query shape it has seen before.

2 · HOW IT WORKS

WHAT MAKES A CACHE HIT, AND WHAT BREAKS IT

RESULT CACHE — ON THE LEADER — milliseconds, zero compute

Same SQL text, same user, same permissions, and no change to the underlying tables. Then the answer is returned as-is.

WHAT INVALIDATES IT — any write, or a volatile function

A load into any referenced table. Also GETDATE, RANDOM, CURRENT_DATE, temp tables and external tables — never cached.

COMPILED CODE CACHE — shared across clusters

Segments are compiled to C++ on first execution. That cost is why a brand-new query shape has a slow first run even on small data.

BENCHMARK HONESTLY — turn the cache off first

SET enable_result_cache_for_session TO off, then run each variant three times and compare the third. Otherwise you are timing the cache.

```
a 40 second query that reruns in 8 ms did not run at all
```

3 · THE SQL

```
-- honest timing for a rewrite test
SET enable_result_cache_for_session
TO off;

-- did it hit the cache?
SELECT query_id, elapsed_time,
result_cache_hit
FROM sys_query_history LIMIT 20;
```

4 · GOTCHAS

The cache is not a performance strategy —
a dashboard after a load never hits it.
For that, use a materialized view (Lesson 11).

whitespace changes still hit

IN PLAIN ENGLISH

Ask the same question twice and you get the note the leader kept, not a fresh count of the warehouse. Change one item in stock and the note is torn up.

L34 · Result Caching and Compiled Code

Slide: `_render/L34-result-caching.html`

What it is

The second run of a query is often a hundred times faster than the first. Knowing **why** stops you drawing the wrong conclusion when you benchmark a rewrite.

There are **two separate caches** and they explain different things.

1 · The result cache — on the leader node

If the leader has already computed this exact answer and nothing has changed, it hands the answer straight back. Zero compute nodes touched, single-digit milliseconds, **zero cost on serverless** (no RPUs are consumed).

A hit needs all of these:

- The **same SQL text** — matched after whitespace and case normalisation, so reformatting is fine but changing a literal is not
- The **same user**, with the same permissions
- **No change to any referenced table** since the result was cached
- **No volatile function** in the query
- The **same session-level configuration** that could affect the result

Never cached:

- Anything containing `GETDATE()`, `SYSDATE`, `CURRENT_DATE`, `RANDOM()`, `TIMEOFDAY()` — the answer would be wrong tomorrow
- Queries over **temp tables** — they are session-scoped
- Queries over **external tables** (Spectrum, external schemas) — Redshift cannot know if the S3 files changed
- Anything that writes

Invalidated by: any write to any referenced table. One `COPY` into your fact table clears every cached result that reads it.

2 · The compiled-code cache

Redshift compiles query segments to **C++ machine code** on first execution (L05). That compilation is real work — seconds, sometimes tens of seconds — and it is why a brand-new query shape can be slow *even on tiny data*.

The compiled objects are cached **per query shape**, and Redshift also uses a **cluster-external, service-wide** compilation cache: a segment your cluster has never compiled may still be fetched already-built because another cluster compiled the same shape. This is why "first run" penalties are much smaller than they used to be, and why they are unpredictable.

Two consequences:

- A **parameterised** query keeps one shape across many values, so it compiles once. String-concatenating literals into SQL from Node produces a new shape per value and recompiles endlessly. Use parameters (L06).
- After a **cluster resize, patch, or restart**, the local cache is cold. The first run of each query shape pays again.

3 · Benchmark honestly

This is the practical point of the lesson. If you time a rewrite without turning the cache off, **you are timing the cache**:

```
-- turn it off for this session only
SET enable_result_cache_for_session TO off;

-- ... run variant A three times, variant B three times ...

SET enable_result_cache_for_session TO on;      -- or just reconnect
```

L34 · Result Caching and Compiled Code

Method:

1. `SET enable_result_cache_for_session TO off`
2. Run variant A three times. Discard the first (compilation), take the median of runs 2 and 3.
3. Run variant B three times. Same.
4. Compare medians.
5. Re-check with `EXPLAIN` that the plan actually changed — a faster time with an identical plan is noise.

4 · Did it hit? Check the system view

```
SELECT query_id,
       LEFT(query_text, 60)  AS sql,
       elapsed_time / 1000000.0 AS seconds,
       result_cache_hit,
       compile_time  / 1000000.0 AS compile_seconds
FROM   sys_query_history
WHERE  user_id = current_user_id
ORDER  BY start_time DESC
LIMIT  20;
```

`result_cache_hit = true` with `elapsed_time` in the milliseconds is the tell. **A 40-second query that reruns in 8 ms did not run at all.**

Cache hit rate over a day, which tells you how much of your dashboard traffic is free:

```
SELECT DATE_TRUNC('hour', start_time)                    AS hr,
       COUNT(*)                                          AS queries,
       SUM(CASE WHEN result_cache_hit THEN 1 ELSE 0 END)    AS hits,
       ROUND(100.0 * SUM(CASE WHEN result_cache_hit THEN 1 ELSE 0 END)
             / NULLIF(COUNT(*), 0), 1)                   AS hit_pct
FROM   sys_query_history
WHERE  start_time >= DATEADD('day', -1, GETDATE())
GROUP  BY 1 ORDER BY 1;
```

5 · The cache is not a performance strategy

The pattern that matters: **a dashboard opened after the nightly load never hits the cache.** The load invalidated everything. Every morning, the first user of every dashboard pays full price.

If you need that to be fast, the cache cannot help you. Use a **materialized view** (L11) refreshed at the end of the load, so the expensive work happens once, in the batch window, before anyone is looking.

```
result cache        → the same question, asked twice, with no writes in between
materialized view   → the expensive answer, computed once per load, on purpose
```

The first is a free accident. The second is engineering.

Gotchas

- **You cannot benchmark with the cache on.** Turn it off first, every time.
- **Whitespace and case changes still hit** — so "I changed the query" is not proof you re-ran it. Change a literal, or turn the cache off.
- **A different user gets a different cache entry**, so testing as an admin does not prove anything about the BI service account.
- **External tables and temp tables are never cached.**
- `enable_result_cache_for_session` **is per session.** In the Data API each call may be a new session, so set it in the same statement batch or accept the default.
- **Serverless auto-pause does not clear the result cache**, but it does mean the first query after a pause pays a start-up delay that has nothing to do with caching. Do not confuse the two.

L34 · Result Caching and Compiled Code

Try it

```
-- 1. cold
SET enable_result_cache_for_session TO off;
SELECT s.region, SUM(f.net_amount)
FROM gold.fct_sales_line f JOIN gold.dim_store s USING (store_sk) GROUP BY 1;

-- 2. warm
SET enable_result_cache_for_session TO on;
SELECT s.region, SUM(f.net_amount)
FROM gold.fct_sales_line f JOIN gold.dim_store s USING (store_sk) GROUP BY 1;

-- 3. now invalidate it
INSERT INTO gold.fct_sales_line SELECT * FROM gold.fct_sales_line LIMIT 1;

-- 4. run the SELECT again — full price
```

Then read all four back from `sys_query_history` and look at the `result_cache_hit` column. Seeing your own timings line up with that flag is worth more than the explanation.

Checklist

- ☐ I know the two caches are different things
- ☐ I turn the result cache off before timing anything
- ☐ I discard the first run — that is compilation
- ☐ I confirm a rewrite with `EXPLAIN`, not just a stopwatch
- ☐ I use parameters from Node so query shapes stay stable
- ☐ I know a load invalidates every cached result over that table
- ☐ Dashboards that must be fast after a load use a materialized view

You've got it when you can…

…watch a colleague declare their rewrite "50× faster", ask whether they turned the result cache off, and be right about the answer.

Part E complete. L28–L34 covered reading plans, joins and distribution, windows, CTEs, time, the four warehouse patterns, and caching. Part F turns to code: procedures, UDFs, and calling Redshift from Node.

MODULE 01 · LESSON 35 · PART F

Stored Procedures

PL/pgSQL running inside the warehouse. The right home for a multi-statement load that must succeed or fail as one thing.

1 · WHAT IT IS

Procedural code that runs where the data already is

No round trips, no rows crossing the network, and one transaction around the whole load. Called with CALL, not SELECT — a procedure is not a function.

2 · HOW IT WORKS

FOUR THINGS THAT SURPRISE APP DEVELOPERS

IT DOES NOT RETURN ROWS — INOUT args or a refcursor

There is no RETURN of a result set. Use INOUT parameters for scalars, or open a refcursor and FETCH from it.

IT IS ONE TRANSACTION BY DEFAULT — atomic unless declared otherwise

Every statement commits together or none do. Declare it NONATOMIC only when you deliberately want per-statement commits.

SECURITY DEFINER — runs as the owner

Lets a caller run a load without holding write rights on the tables. Grant EXECUTE on the procedure instead of INSERT on gold.

NO VERSION CONTROL FOR FREE — the database is not your repo

A procedure edited in a console is invisible to git. Every procedure lives in a .sql file, deployed by CI, or it does not exist.

```
CALL, never SELECT. and always CREATE OR REPLACE.
```

3 · THE SQL

```
CREATE OR REPLACE PROCEDURE
etl.load_sales(batch DATE,
INOUT rows_out INT)
AS $$ BEGIN
... ; GET DIAGNOSTICS rows_out
= ROW_COUNT;
END; $$ LANGUAGE plpgsql;
```

4 · GOTCHAS

Quote the body with $$ or every apostrophe has to be doubled.

Changing an argument type creates a second **overload — DROP the old one**

IN PLAIN ENGLISH

Instead of shipping instructions one at a time and waiting for each reply, you hand over the whole recipe and it is followed in the kitchen.

L35 · Stored Procedures

Slide: `_render/L35-stored-procedures.html`

What it is

PL/pgSQL running **inside** the warehouse. The right home for a multi-statement load that must succeed or fail as one thing.

No round trips. No rows crossing the network. One transaction around the whole load.

The shape

```
CREATE OR REPLACE PROCEDURE etl.load_sales_day(
    p_batch_date   DATE,
    INOUT p_rows_loaded INTEGER DEFAULT 0
)
AS $$
DECLARE
    v_staged INTEGER;
BEGIN
    RAISE INFO 'load_sales_day starting for %', p_batch_date;

    -- 1. how much arrived?
    SELECT COUNT(*) INTO v_staged
    FROM   staging.sales_line
    WHERE  sale_date = p_batch_date;

    IF v_staged = 0 THEN
        RAISE INFO 'nothing staged for %, exiting clean', p_batch_date;
        p_rows_loaded := 0;
        RETURN;
    END IF;

    -- 2. clear the target slice (idempotent rerun)
    DELETE FROM gold.fct_sales_line WHERE sale_date = p_batch_date;

    -- 3. load it
    INSERT INTO gold.fct_sales_line (sale_date, store_sk, product_sk, qty, net_amount)
    SELECT s.sale_date, d.store_sk, p.product_sk, s.qty, s.net_amount
    FROM   staging.v_sales_line_latest s
    JOIN   gold.dim_store   d ON d.store_id = s.store_id AND d.is_current
    JOIN   gold.dim_product p ON p.product_id = s.product_id AND p.is_current
    WHERE  s.sale_date = p_batch_date;

    GET DIAGNOSTICS p_rows_loaded = ROW_COUNT;

    RAISE INFO 'loaded % rows for %', p_rows_loaded, p_batch_date;
END;
$$ LANGUAGE plpgsql;
```

Call it:

```
CALL etl.load_sales_day('2026-08-10', 0);
```

`CALL`, **never** `SELECT`. A procedure is not a function.

Four things that surprise application developers

1 · It does not return rows

There is no `RETURN <result set>`. Two options:

`INOUT` **parameters** for scalars — the value comes back in the `CALL` result:

```
CALL etl.load_sales_day('2026-08-10', 0);
--  p_rows_loaded
--  -------------
--          48213
```

A refcursor for a result set:

L35 · Stored Procedures

```
CREATE OR REPLACE PROCEDURE etl.get_daily(p_date DATE, INOUT ref refcursor)
AS $$
BEGIN
    OPEN ref FOR SELECT store_sk, SUM(net_amount)
                 FROM gold.fct_sales_line WHERE sale_date = p_date GROUP BY 1;
END;
$$ LANGUAGE plpgsql;

BEGIN;
CALL etl.get_daily('2026-08-10', 'mycursor');
FETCH ALL FROM mycursor;
COMMIT;
```

Note the `BEGIN … COMMIT` — the cursor only lives inside the transaction. This is clumsy from an application; in practice, have Node call a procedure to *do work* and then run a plain `SELECT` to *read results*.

2 · It is one transaction by default

Every statement commits together or none do. That is usually exactly what you want: a load that fails halfway leaves the warehouse as it was.

Declare `NONATOMIC` only when you deliberately want each statement to commit independently — a long backfill that should keep its progress, for example:

```
CREATE OR REPLACE PROCEDURE etl.backfill(p_from DATE, p_to DATE)
NONATOMIC
AS $$
DECLARE d DATE := p_from;
BEGIN
    WHILE d <= p_to LOOP
        CALL etl.load_sales_day(d, 0);
        COMMIT;                       -- allowed in NONATOMIC
        d := DATEADD('day', 1, d);
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

`COMMIT` and `ROLLBACK` inside the body are only meaningful in `NONATOMIC` mode, and only when the procedure was not itself called from inside an open transaction.

3 · SECURITY DEFINER

By default a procedure runs with the **caller's** privileges (`SECURITY INVOKER`). `SECURITY DEFINER` runs it as the **owner** instead:

```
CREATE OR REPLACE PROCEDURE etl.load_sales_day(...)
SECURITY DEFINER
AS $$ ... $$ LANGUAGE plpgsql;

GRANT EXECUTE ON PROCEDURE etl.load_sales_day(DATE, INTEGER) TO ROLE etl_operator;
```

Now `etl_operator` can run the load without holding `INSERT` on `gold` . **Grant `EXECUTE` on the procedure instead of write rights on the tables** — this is the cleanest privilege pattern in the warehouse (L13).

⚠️ With `SECURITY DEFINER` , always schema-qualify every table name and set an explicit `search_path` . An unqualified name resolved against the caller's `search_path` is how privilege escalation happens.

4 · No version control for free

A procedure edited in a console is invisible to git. **Every procedure lives in a `.sql` file in the repo, deployed by CI, or it does not exist.**

```
db/
  procedures/
    etl.load_sales_day.sql
    etl.load_dim_store.sql
    etl.run_nightly.sql
  deploy.sh              # psql -f each file, in order
```

Always `CREATE OR REPLACE` so redeployment is idempotent.

L35 · Stored Procedures

An orchestrator procedure

The pattern that ties a nightly load together:

```
CREATE OR REPLACE PROCEDURE etl.run_nightly(p_batch_date DATE)
AS $$
DECLARE
    v_run_id  BIGINT;
    v_rows    INTEGER;
BEGIN
    v_run_id := (SELECT COALESCE(MAX(run_id), 0) + 1 FROM etl.run_log);

    INSERT INTO etl.run_log (run_id, batch_date, status, started_at_utc)
    VALUES (v_run_id, p_batch_date, 'RUNNING', GETDATE());

    CALL etl.load_dim_store(p_batch_date);
    CALL etl.load_dim_product(p_batch_date);
    CALL etl.load_sales_day(p_batch_date, v_rows);

    -- data quality gate: refuse to publish a bad load
    IF v_rows = 0 THEN
        RAISE EXCEPTION 'no rows loaded for %', p_batch_date;
    END IF;

    REFRESH MATERIALIZED VIEW gold.mv_sales_daily;

    UPDATE etl.run_log
    SET    status = 'SUCCESS', rows_loaded = v_rows, ended_at_utc = GETDATE()
    WHERE  run_id = v_run_id;
END;
$$ LANGUAGE plpgsql;
```

Note the **quality gate**. A procedure that publishes whatever it is given is a procedure that will publish a zero one morning.

Finding what exists

```
-- list procedures in a schema
SELECT n.nspname AS schema, p.proname AS name
FROM   pg_proc p JOIN pg_namespace n ON n.oid = p.pronamespace
WHERE  n.nspname = 'etl' AND p.prokind = 'p'
ORDER  BY 2;

-- read the source of one
SHOW PROCEDURE etl.load_sales_day(DATE, INTEGER);
```

Gotchas

- **Quote the body with `$$`.** Otherwise every apostrophe inside must be doubled, and the first time you write `'store's'` you will lose twenty minutes.
- **Changing an argument type creates a second overload**, it does not replace the first. `DROP PROCEDURE` the old signature explicitly, or you will have two and callers will get whichever matches.
- **`RAISE INFO` output goes to the client.** Through the Data API you may not see it — write to an audit table as well.
- **Max source size is 2 MB**; nesting depth is 16 levels. If you are near either, the design is wrong.
- **`SET` inside a procedure is scoped to the procedure** and reverts on exit.
- **A procedure cannot be called from inside a `SELECT`.** No `SELECT etl.load_sales_day(...)`.
- **`GET DIAGNOSTICS … = ROW_COUNT`** must come immediately after the DML statement it describes.

L35 · Stored Procedures

Try it

1. Write `etl.load_sales_day` against your own tables and call it twice for the same date. The row count must be identical both times — that is idempotency (L24).
2. Add the `IF v_staged = 0 THEN RETURN` guard and call it for a date with no data. It should exit clean, not error.
3. Make it `SECURITY DEFINER`, grant `EXECUTE` to a role that has no rights on `gold`, and prove that role can run the load but not `INSERT` directly.
4. Deliberately break the `INSERT` (rename a column) and confirm the `DELETE` rolled back too.

That fourth step is the one that makes atomicity real.

Checklist

- ☐ Multi-statement loads live in procedures, not in Node
- ☐ `CALL`, never `SELECT`
- ☐ `CREATE OR REPLACE` always, and the source is in git
- ☐ `INOUT` for the row count, so the caller can log it
- ☐ `SECURITY DEFINER` plus `GRANT EXECUTE`, with schema-qualified names
- ☐ Every procedure is idempotent for the same batch date
- ☐ A quality gate before anything is published
- ☐ `NONATOMIC` used only deliberately, for backfills

You've got it when you can…

…take a Node script that runs eight statements in sequence with `await` between each, move it into one procedure, and explain to the team why the failure behaviour is now correct rather than merely tidier.

MODULE 01 · LESSON 36

Control Flow, Cursors and Exceptions

Everything PL/pgSQL gives you for branching and error handling — and the loop you must train yourself not to write.

1 · WHAT IT IS

Branching for the pipeline, not for the rows

Use IF and loops to decide what step runs next. The moment a loop iterates over data rows, you have turned a columnar engine back into a row engine.

2 · HOW IT WORKS

THE FOUR CONSTRUCTS THAT MATTER

IF / ELSIF / WHILE / FOR — `orchestration only`

Skip a step when the batch is empty, loop over a list of ten source tables, retry once. All good uses.

RAISE — YOUR ONLY PRINTF — `INFO, NOTICE, EXCEPTION`

RAISE INFO writes to the client; RAISE EXCEPTION aborts. There is no debugger here, so log deliberately.

EXCEPTION WHEN OTHERS — `OTHERS is the only condition`

You cannot catch a specific SQL state. Read SQLERRM, write it to an audit table, then re-raise — never swallow it.

CURSORS — ALMOST NEVER — `a row loop on a columnar engine`

A cursor over a million rows will run for hours where one MERGE takes seconds. Legitimate only for returning a result set.

```
a loop over rows is a bug report you have not filed yet
```

3 · THE SQL

```
EXCEPTION WHEN OTHERS THEN
INSERT INTO etl.run_log
VALUES (run_id, 'FAILED',
SQLERRM, GETDATE());
-- then hand the error back up
RAISE;
-- silence here costs you a night
```

4 · GOTCHAS

In an atomic procedure the whole thing rolls back — including your log row.
Log the failure from the caller, or go

NONATOMIC on purpose

IN PLAIN ENGLISH

Decide which recipes to cook. Do not stand at the counter chopping one onion at a time when the machine will do the whole crate in one pass.

L36 · Control Flow, Cursors and Exceptions

Slide: `_render/L36-control-flow-exceptions.html`

What it is

Everything PL/pgSQL gives you for branching and error handling — and the one loop you must train yourself not to write.

The rule: branch for the pipeline, never for the rows. Use `IF` and loops to decide *what step runs next*. The moment a loop iterates over data rows, you have turned a columnar engine back into a row engine.

Declarations and assignment

```
DECLARE
    v_count   INTEGER := 0;              -- := for assignment, not =
    v_date    DATE    := CURRENT_DATE;
    v_name    VARCHAR(120);
    v_rec     RECORD;                    -- holds one row of anything
BEGIN
    -- assign from a query with INTO
    SELECT COUNT(*) INTO v_count FROM staging.sales_line;

    -- or plain assignment from an expression
    v_date := DATEADD('day', -1, v_date);
END;
```

`INTO` takes the **first row** and does not error if there are none — `v_count` simply stays as it was. Guard anything that must find a row:

```
SELECT store_name INTO v_name FROM gold.dim_store WHERE store_id = p_id AND is_current;
IF v_name IS NULL THEN
    RAISE EXCEPTION 'no current store for id %', p_id;
END IF;
```

Branching

```
IF v_staged = 0 THEN
    RAISE INFO 'nothing to do';
    RETURN;
ELSIF v_staged > 10000000 THEN
    RAISE EXCEPTION 'batch of % rows looks wrong, refusing', v_staged;
ELSE
    CALL etl.load_sales_day(p_batch_date, v_rows);
END IF;
```

That `ELSIF` is a **sanity gate**. A batch ten times normal size is nearly always a source-system bug, and catching it before it lands is much cheaper than unwinding it afterwards.

L36 · Control Flow, Cursors and Exceptions

Loops — the legitimate uses

```
-- 1. a fixed list of things to process
FOR i IN 1..12 LOOP
    CALL etl.load_month(p_year, i);
END LOOP;

-- 2. a driver table of source systems (a handful of rows, not millions)
FOR v_rec IN SELECT schema_name, table_name FROM etl.source_registry WHERE enabled LOOP
    RAISE INFO 'refreshing %.%', v_rec.schema_name, v_rec.table_name;
    EXECUTE 'ANALYZE ' || quote_ident(v_rec.schema_name) || '.' || quote_ident(v_rec.table_name);
END LOOP;

-- 3. a bounded retry
v_attempt := 0;
LOOP
    v_attempt := v_attempt + 1;
    BEGIN
        CALL etl.load_sales_day(p_batch_date, v_rows);
        EXIT;                                   -- success, leave the loop
    EXCEPTION WHEN OTHERS THEN
        IF v_attempt >= 3 THEN RAISE; END IF;    -- give up honestly
        RAISE INFO 'attempt % failed: %', v_attempt, SQLERRM;
    END;
END LOOP;

-- 4. a WHILE over dates in a NONATOMIC backfill
WHILE v_date <= p_to LOOP
    CALL etl.load_sales_day(v_date, v_rows);
    COMMIT;
    v_date := DATEADD('day', 1, v_date);
END LOOP;
```

All four loop over **units of work**, never over data rows. That is the distinction.

Dynamic SQL

```
EXECUTE 'DELETE FROM ' || quote_ident(v_schema) || '.' || quote_ident(v_table)
        || ' WHERE sale_date = $1'
  USING p_batch_date;
```

⚠ `quote_ident` **for identifiers,** `USING` **for values.** String-concatenating a value into dynamic SQL is a SQL injection, even inside the warehouse — a source-registry table someone can edit is an input.

RAISE — your only printf

```
RAISE INFO   'loaded % rows for %', v_rows, p_batch_date;   -- to the client
RAISE NOTICE 'stats look stale on %', v_table;
RAISE WARNING 'quality check soft-failed';
RAISE EXCEPTION 'no rows loaded for %', p_batch_date;       -- aborts
```

`%` is the placeholder — positional, no numbering. **There is no debugger here**, so log deliberately: entry, key counts, exit, and every branch you took.

Exception handling

```
BEGIN
    ... work ...
EXCEPTION WHEN OTHERS THEN
    INSERT INTO etl.run_log (run_id, status, error_text, ended_at_utc)
    VALUES (v_run_id, 'FAILED', SQLERRM, GETDATE());
    RAISE;                          -- hand the error back up
END;
```

`OTHERS` **is the only condition Redshift supports.** You cannot catch a specific SQL state the way you can in PostgreSQL — so you cannot retry-on-deadlock but fail-on-syntax-error. Inspect `SQLERRM` (the message text) if you need to distinguish.

Always re- `RAISE` **.** A swallowed exception turns a failed load into a silent one, and a silent failure is discovered by a business user three days later.

L36 · Control Flow, Cursors and Exceptions

⚠️ The trap: your log row rolls back too

In an **atomic** procedure — the default — the `EXCEPTION` block runs inside the same transaction that is about to roll back. Your carefully written `run_log` row disappears along with everything else.

Three ways out, in order of preference:

1 · Log from the caller. Node catches the error and writes the audit row on its own connection. Cleanest, and the caller knows things the procedure does not (job id, trigger, retry count).

```
try {
  await call('CALL etl.run_nightly($1)', [batchDate]);
  await log('SUCCESS', null);
} catch (err) {
  await log('FAILED', err.message);      // separate transaction — this survives
  throw err;
}
```

2 · `NONATOMIC` with an explicit `COMMIT` after the log insert, when you genuinely want per-statement commits anyway.

3 · Let it fail loudly and read the error from `stl_load_errors` / `sys_query_history` / CloudWatch. Fine for a small team, weak for an audited pipeline.

Cursors — almost never

```
-- ❌ this is the anti-pattern
DECLARE c CURSOR FOR SELECT * FROM gold.fct_sales_line;
LOOP
    FETCH c INTO v_rec;
    EXIT WHEN NOT FOUND;
    UPDATE gold.fct_sales_line SET flag = TRUE WHERE id = v_rec.id;
END LOOP;
```

A cursor over a million rows will run for **hours** where one `MERGE` takes seconds. Every `UPDATE` is a delete-plus-insert (L23), and you have just done a million of them one at a time.

```
-- ✅ what it should have been
UPDATE gold.fct_sales_line SET flag = TRUE WHERE <the same condition>;
```

The legitimate use is returning a result set from a procedure (L35), and nothing else. If you find yourself writing `FETCH` inside a loop, stop and ask what set-based statement you are avoiding.

Cursor limits are also real: cursor result sets are constrained by cluster size, and a large one will fail rather than degrade.

Gotchas

- `:=` **assigns,** `=` **compares.** A common first-day error.
- `SELECT … INTO` **with no rows leaves the variable unchanged**, it does not raise. Check explicitly.
- `OTHERS` **is the only catchable condition.**
- **The exception block rolls back with the transaction** in atomic mode — log from outside.
- **Never swallow an exception without re-raising.**
- `RAISE INFO` **may not reach you through the Data API.** Persist what matters.
- `EXECUTE` **with concatenated values is injectable.** `USING` and `quote_ident`.
- `EXIT` **leaves the innermost loop only**; label loops if you need to break further.

L36 · Control Flow, Cursors and Exceptions

Try it

1. Add the >10× sanity gate to your load and prove it refuses an oversized batch.
2. Wrap a load in a three-attempt retry and make it fail twice by locking the target table from another session (L25).
3. Write the exception handler with the `run_log` insert, run a failing load, and confirm **no log row exists**. Then move the logging to the caller and confirm it does. This is the lesson that sticks.
4. Time a cursor loop that updates 100,000 rows, then time the equivalent single `UPDATE`. Write both numbers on the board.

Checklist

- ☐ Loops iterate over units of work, never data rows
- ☐ A sanity gate on batch size before every load
- ☐ `RAISE INFO` at entry, at each branch, and at exit
- ☐ `EXCEPTION WHEN OTHERS` records `SQLERRM` and re-`RAISE`s
- ☐ Failure logging happens outside the failing transaction
- ☐ Dynamic SQL uses `quote_ident` and `USING`
- ☐ No cursor anywhere except to return a result set
- ☐ Retries are bounded and give up honestly

You've got it when you can…

…look at a procedure with a cursor loop, name the set-based statement it should have been, and quote the ratio between the two runtimes from having measured it yourself.

MODULE 01 · LESSON 37

User-Defined Functions

A function returns a value and can appear in a SELECT. A procedure cannot. That single difference decides which one you write.

1 · WHAT IT IS

Scalar logic, named once, reused everywhere

A UDF takes scalars and returns one scalar. It cannot read a table and it cannot write anything — that is what makes it safe to call a billion times.

2 · HOW IT WORKS

THREE KINDS, IN ORDER OF PREFERENCE

SQL UDF — ALWAYS FIRST CHOICE — `inlined, no overhead`

A single SELECT expression. The planner folds it into the query, so it costs the same as writing the expression by hand.

PYTHON UDF — DO NOT USE — `end of support 30 Jun 2026`

Creation blocked since patch 198 (Oct 2025) and support ended June 2026. If you inherit one, migrate it to a Lambda UDF.

LAMBDA UDF — THE ESCAPE HATCH — `covered in Lesson 38`

Any language, any library, any external call. Batched over the network, so cost and failure are now part of your query.

STABLE, IMMUTABLE, VOLATILE — `tells the planner what it may skip`

Mark a pure function IMMUTABLE and identical inputs are evaluated once. Get this wrong and you cache a wrong answer.

```
if it fits in one SELECT expression, it is a SQL UDF.
```

3 · THE SQL

```
CREATE OR REPLACE FUNCTION
f_net_of_vat(gross DECIMAL(18,4))
RETURNS DECIMAL(18,4)
IMMUTABLE AS $$
SELECT $1 / 1.15
$$ LANGUAGE sql;

-- args are $1, $2 — not by name
```

4 · GOTCHAS

A UDF cannot query a table. Lookups belong in a join, not in a function.

NULL in usually means NULL out — handle it.

the f_ prefix is a convention worth keeping

IN PLAIN ENGLISH

One agreed definition of “net of VAT”, written once, so that four reports cannot quietly disagree about what the number means.

L37 · User-Defined Functions

Slide: `_render/L37-udfs.html`

What it is

A **function returns a value and can appear in a `SELECT`**. A procedure cannot. That single difference decides which one you write.

```
SELECT f_net_of_vat(gross_amount) FROM ...   -- function ✅
CALL   etl.load_sales_day('2026-08-10', 0);  -- procedure ✅
SELECT etl.load_sales_day(...)               -- ❌ not a thing
```

A UDF takes scalars and returns one scalar. **It cannot read a table and it cannot write anything** — that is exactly what makes it safe to call a billion times.

1 · SQL UDFs — always the first choice

A single `SELECT` expression. The planner folds it into the query, so it costs the same as writing the expression by hand.

```
CREATE OR REPLACE FUNCTION f_net_of_vat(gross DECIMAL(18,4))
RETURNS DECIMAL(18,4)
IMMUTABLE
AS $$
  SELECT $1 / 1.15
$$ LANGUAGE sql;
```

⚠️ **Arguments are `$1`, `$2` — positional, not by name.** The name in the signature is documentation only. This trips up everyone once.

More useful examples:

```
-- one agreed definition of the fiscal year
CREATE OR REPLACE FUNCTION f_fiscal_year(d DATE)
RETURNS SMALLINT IMMUTABLE
AS $$ SELECT CASE WHEN EXTRACT(month FROM $1) >= 4
                  THEN EXTRACT(year FROM $1) ELSE EXTRACT(year FROM $1) - 1 END $$
LANGUAGE sql;

-- safe division, used everywhere
CREATE OR REPLACE FUNCTION f_safe_div(n DECIMAL(18,4), d DECIMAL(18,4))
RETURNS DECIMAL(18,6) IMMUTABLE
AS $$ SELECT $1 / NULLIF($2, 0) $$ LANGUAGE sql;

-- the SCD2 hash, defined once so every dimension load agrees
CREATE OR REPLACE FUNCTION f_row_hash(a VARCHAR, b VARCHAR, c VARCHAR)
RETURNS VARCHAR(32) IMMUTABLE
AS $$ SELECT MD5(COALESCE($1,'') || '|' || COALESCE($2,'') || '|' || COALESCE($3,'')) $$
LANGUAGE sql;
```

That last one is the real argument for UDFs: **four dimension loads that each write their own hash expression will eventually disagree.** One function means they cannot.

2 · Python UDFs — do not use

> **Confirmed against AWS documentation.** Amazon Redshift stopped supporting the *creation* of new scalar Python UDFs after **patch 198 (30 October 2025)**, and existing Python UDFs reached **end of support on 30 June 2026** — a date that has now passed. AWS directs users to **Lambda UDFs** as the replacement.

They are covered here only so you recognise one if you inherit it:

```
-- legacy — you may find this in an old codebase. Migrate it.
CREATE OR REPLACE FUNCTION f_parse_sku(sku VARCHAR)
RETURNS VARCHAR STABLE
AS $$
  import re
  m = re.match(r'^([A-Z]{3})-(\d+)$', sku)
  return m.group(1) if m else None
$$ LANGUAGE plpythonu;
```

Two migration routes, in order of preference:

L37 · User-Defined Functions

1. **Rewrite it in SQL.** Most Python UDFs in the wild are string manipulation that `SPLIT_PART`, `REGEXP_SUBSTR`, `TRANSLATE` and `POSITION` handle natively — and the SQL version is faster.
2. **Move it to a Lambda UDF** (L38) when the logic genuinely needs a library.

Find them before they bite:

```
SELECT n.nspname AS schema, p.proname AS name, l.lanname AS language
FROM   pg_proc p
JOIN   pg_namespace n ON n.oid = p.pronamespace
JOIN   pg_language  l ON l.oid = p.prolang
WHERE  l.lanname LIKE 'plpython%'
ORDER  BY 1, 2;
```

Run that on any inherited cluster on day one.

3 · Lambda UDFs — the escape hatch

Any language, any library, any external call. Covered fully in **L38**. The short version: right for a lookup no SQL can do, wrong for anything you could have precomputed into a column at load time.

4 · Volatility — STABLE, IMMUTABLE, VOLATILE

This tells the planner what it is allowed to skip:

CATEGORY	MEANING	PLANNER MAY
`IMMUTABLE`	Same inputs always give the same result, forever	Evaluate once and reuse; fold constants
`STABLE`	Same result within a single statement	Reuse within the statement
`VOLATILE`	May differ every call (default)	Nothing — call it per row

Mark pure functions `IMMUTABLE`. It is free performance. But get it wrong — mark something `IMMUTABLE` that reads changing state — and you cache a wrong answer with no error to tell you.

`VOLATILE` is the default, so an unmarked function is the slow case.

Naming and permissions

```
-- convention: f_ prefix, so a UDF is visibly not a built-in
CREATE OR REPLACE FUNCTION f_net_of_vat(...) ...

-- functions are schema-scoped like everything else
GRANT EXECUTE ON FUNCTION f_net_of_vat(DECIMAL) TO ROLE analyst;
```

The `f_` prefix is an AWS convention and worth keeping — it means nobody has to wonder whether `net_of_vat` is something Redshift ships or something your team wrote. It also protects you if AWS later adds a built-in with the same name.

Gotchas

- **Arguments are `$1`, `$2`, not names.**
- **A UDF cannot query a table.** Lookups belong in a join, not in a function. This is the single most common request and the answer is always "join to a dimension".
- **`NULL` in usually means `NULL` out.** Handle it explicitly with `COALESCE` or `NULLIF`.
- **Changing the argument types creates an overload**, not a replacement. `DROP FUNCTION` the old signature.
- **`VOLATILE` is the default** — mark pure functions `IMMUTABLE`.
- **A wrong `IMMUTABLE` gives silently wrong answers.**
- **UDFs are not in git unless you put them there.** Same rule as procedures (L35).
- **No aggregate UDFs.** You cannot write your own `SUM`-like function.

L37 · User-Defined Functions

Try it

1. Write `f_safe_div` and `f_row_hash`, and replace the hand-written hash in your `dim_store` load with the function. Confirm the load still produces zero changes on a rerun.
2. Create the same function twice with different argument types and watch both exist. Then drop one properly.
3. Run the `plpython%` audit query against every cluster you have access to.
4. Take any Python UDF you find and rewrite it in SQL. Time both if the Python one still runs.

Checklist

- ☐ SQL UDF if it fits in one `SELECT` expression
- ☐ No new Python UDFs — the runtime is out of support
- ☐ I have audited for existing Python UDFs
- ☐ Pure functions marked `IMMUTABLE`
- ☐ `f_` prefix on everything
- ☐ `NULL` handled explicitly
- ☐ Shared definitions (VAT, fiscal year, row hash) live in exactly one function
- ☐ Function source is in git and deployed by CI

You've got it when you can…

…find three reports that each compute "net of VAT" slightly differently, replace all three with one `IMMUTABLE` SQL UDF, and show that the numbers now agree.

MODULE 01 · LESSON 38

Lambda UDFs

Your Node.js code, called from inside a SELECT. Powerful, and the fastest way to make a warehouse query fail for reasons that are not the warehouse.

1 · WHAT IT IS

Redshift calls your Lambda, in batches, mid-query

Rows are grouped into JSON payloads, sent to the function, and the returned array is matched back by position. Any runtime, any library, your code.

2 · HOW IT WORKS

WHAT YOU ARE SIGNING UP FOR

THE CONTRACT IS POSITIONAL — `arguments in, results out`

You receive an array of argument rows and must return exactly that many results, in order. A wrong count errors the query.

IT NEEDS AN IAM ROLE — `lambda:InvokeFunction`

The cluster assumes a role to invoke the function. Same mechanism as COPY and UNLOAD — one more reason to keep roles tidy.

THE BLAST RADIUS IS THE QUERY — `throttling becomes SQL failure`

A cold start, a concurrency limit or a timeout now fails a SELECT. Reserve concurrency, and keep the function trivially fast.

USE IT FOR THE FEW, NOT THE MANY — `tokenise, mask, geocode`

Right for a lookup no SQL can do. Wrong for anything you could have precomputed into a column at load time.

```
filter first. never call a Lambda on rows you throw away.
```

3 · THE SQL

```
CREATE OR REPLACE EXTERNAL
FUNCTION f_mask_phone(VARCHAR)
RETURNS VARCHAR STABLE
LAMBDA 'tamimi-mask-phone'
IAM_ROLE 'arn:aws:iam::...:role/rs';

-- then just call it in a SELECT
SELECT f_mask_phone(phone) ...
```

4 · GOTCHAS

Every slice invokes independently, so a large scan can spike concurrency hard.

Nulls arrive as JSON null — handle them.

success:false in the response fails the query

IN PLAIN ENGLISH

The warehouse phones an outside specialist for a handful of hard cases. Phone them for every row and the report now depends on someone else answering.

L38 · Lambda UDFs

Slide: `_render/L38-lambda-udfs.html`

What it is

Your Node.js code, called from inside a `SELECT`.

Redshift groups rows into JSON payloads, invokes your Lambda, and matches the returned array back **by position**. Any runtime, any library, your code.

It is also the fastest way to make a warehouse query fail for reasons that are not the warehouse's fault — so the design discipline matters more than the syntax.

Since Python UDFs went out of support (L37), this is **the** extensibility mechanism in Redshift.

Registering one

```
CREATE OR REPLACE EXTERNAL FUNCTION f_mask_phone(phone VARCHAR)
RETURNS VARCHAR
STABLE
LAMBDA 'tamimi-mask-phone'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftLambdaRole';
```

Then call it like anything else:

```
SELECT customer_id, f_mask_phone(phone_number) AS phone
FROM   gold.dim_customer
WHERE  is_current;
```

The IAM role needs `lambda:InvokeFunction` on that function, and the role must be attached to the cluster or workgroup — the same mechanism as `COPY` and `UNLOAD` (L21, L27).

The contract — this is the part people get wrong

Redshift sends:

```
{
  "request_id": "23FF1F97-F28A-44AA-AB67-266ED976BF40",
  "cluster": "arn:aws:redshift:...",
  "user": "etl_service",
  "database": "tamimi",
  "external_function": "f_mask_phone",
  "query_id": 5678234,
  "num_records": 3,
  "arguments": [
    ["0501234567"],
    ["0559876543"],
    [null]
  ]
}
```

You must return:

```
{
  "success": true,
  "num_records": 3,
  "results": ["05012***67", "05598***43", null]
}
```

Three rules, and all three are enforced:

1. `results.length` **must equal** `num_records`. Filter or drop one row and the query errors.
2. **Order is the contract.** Result `i` belongs to argument row `i`. No keys, no ids — position only.
3. `success: false` **fails the whole query.** There is no partial-success mode.

L38 · Lambda UDFs

The handler, in Node

```
// tamimi-mask-phone/index.mjs
export const handler = async (event) => {
  try {
    const results = event.arguments.map(([phone]) => {
      if (phone === null || phone === undefined) return null;   // NULL in, NULL out
      const s = String(phone);
      return s.length < 7 ? s : s.slice(0, 5) + '***' + s.slice(-2);
    });

    return {
      success: true,
      num_records: results.length,       // ← must match event.num_records
      results,
    };
  } catch (err) {
    // one bad row would otherwise fail the entire query
    return { success: false, error_msg: String(err).slice(0, 250) };
  }
};
```

Every row must produce an output, even a bad one. A `null` for an unparseable value is almost always better than failing a 40-minute query. Decide that policy deliberately and write it in the function's README.

Handle `null` explicitly. Redshift NULLs arrive as JSON `null` and JavaScript will happily turn them into the string `"null"` if you are careless.

Multiple arguments

```
CREATE OR REPLACE EXTERNAL FUNCTION f_distance_km(
    lat1 FLOAT8, lon1 FLOAT8, lat2 FLOAT8, lon2 FLOAT8)
RETURNS FLOAT8
STABLE
LAMBDA 'tamimi-geo-distance'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftLambdaRole';
```

Each element of `arguments` is then a four-element array, in declared order:

```
const results = event.arguments.map(([lat1, lon1, lat2, lon2]) => haversine(lat1, lon1, lat2, lon2));
```

What you are signing up for

The blast radius is now the query

A cold start, a concurrency limit, or a timeout **fails a `SELECT`**. Things that were an ops concern are now a data concern.

Mitigations, in order:

- **Reserve concurrency** on the function so a big scan cannot starve it — and so it cannot starve everything else in the account.
- **Keep the function trivially fast.** No network calls inside it if you can help it. A Lambda UDF that calls an external API has that API's availability inside your SQL.
- **Raise the timeout** but keep it far below the query timeout.
- **Retry inside the handler**, not by letting Redshift fail.

Every slice invokes independently

This is the one that surprises people. A large scan spread across 64 slices produces **64 concurrent invocation streams**, and a Lambda UDF over a billion-row table can spike account-level concurrency hard enough to throttle unrelated services.

Two tuning knobs:

L38 · Lambda UDFs

```
CREATE OR REPLACE EXTERNAL FUNCTION f_mask_phone(VARCHAR)
RETURNS VARCHAR STABLE
LAMBDA 'tamimi-mask-phone'
IAM_ROLE '...'
MAX_BATCH_ROWS 1000        -- rows per invocation
RETRY_TIMEOUT 20000;       -- ms Redshift keeps retrying a failed invocation
```

Larger `MAX_BATCH_ROWS` means fewer invocations and less overhead, but a bigger payload — and there is a hard payload limit, so a wide `VARCHAR` argument forces the batch size down.

Filter first, always

```
-- ❌ masks every row, then throws most away
SELECT * FROM (SELECT f_mask_phone(phone) AS p, region FROM gold.dim_customer)
WHERE region = 'WEST';

-- ✅ filter first, then mask what survives
SELECT f_mask_phone(phone) AS p, region
FROM   gold.dim_customer
WHERE  region = 'WEST' AND is_current;
```

Never call a Lambda on rows you throw away. Check the `EXPLAIN` plan (L28) to confirm the filter really is applied first — and be aware that the planner does not always know a UDF is expensive.

When it is the right tool

GOOD	WHY
Tokenisation / detokenisation	The vault is external by design
Masking PII for a non-privileged role	Logic must be central and auditable
Geocoding, address normalisation	Needs a library and reference data
Calling an ML endpoint	Genuinely external
A licensed algorithm	Cannot be reimplemented in SQL

BAD	DO THIS INSTEAD
String formatting	SQL string functions
Date arithmetic	`DATEADD`, `DATE_TRUNC`, `dim_date`
Lookups	Join to a dimension
Anything per-row on a fact table	Precompute a column at load time

The last row is the important one. **If the value is stable, compute it once during the load and store it.** A Lambda UDF called at query time recomputes the same answer for every reader, forever.

> For masking specifically, check whether **dynamic data masking** policies solve your case natively (L13) — a policy is cheaper, declarative, and does not put a Lambda in the query path.

Monitoring

```
-- external function calls show up in query history
SELECT query_id, LEFT(query_text, 80), elapsed_time/1000000.0 AS secs
FROM   sys_query_history
WHERE  query_text ILIKE '%f_mask_phone%'
ORDER  BY start_time DESC LIMIT 20;
```

Then watch the function's own CloudWatch metrics — `Invocations`, `Throttles`, `Duration`, `Errors`. **A rise in `Throttles` will show up as intermittent query failures** that look like a Redshift problem and are not.

L38 · Lambda UDFs

Gotchas

- `results.length` **must equal** `num_records`**.** Never filter inside the handler.
- **Order is the only contract.** Never reorder, never dedupe.
- `success: false` **fails the whole query.**
- **Nulls arrive as JSON** `null` — handle them.
- **Every slice invokes independently.** Reserve concurrency.
- **There is a payload size limit**, so wide arguments force small batches.
- **Cold starts add seconds** to the first invocation of an idle function.
- **The IAM role must be attached to the cluster**, not merely to exist.
- **Cost is now two services.** A daily report calling a Lambda 200 million times has a Lambda bill.
- `STABLE` **vs** `VOLATILE` **matters more here** — a `VOLATILE` external function cannot be optimised at all.

Try it

1. Deploy `tamimi-mask-phone` and register it. Call it on ten rows.
2. Break the contract deliberately: return `results.length - 1` results. Read the error message so you recognise it later.
3. Pass a `NULL` and confirm you get `NULL` back, not `"null"`.
4. Run it over a large table with and without a selective `WHERE`, and compare invocation counts in CloudWatch.
5. Set `MAX_BATCH_ROWS 10`, rerun, and watch the invocation count explode. Then set it to 1000.

Step 5 makes the batching real in a way no explanation does.

Checklist

- ☐ Handler returns exactly `num_records` results, in order
- ☐ Every row produces an output — bad rows return `null`, not an exception
- ☐ Nulls handled explicitly
- ☐ `try/catch` wraps the whole handler
- ☐ Reserved concurrency set on the function
- ☐ `MAX_BATCH_ROWS` tuned and `RETRY_TIMEOUT` set
- ☐ Filters applied before the UDF, and confirmed in `EXPLAIN`
- ☐ CloudWatch alarm on `Throttles` and `Errors`
- ☐ I checked whether native masking or a precomputed column would do instead

You've got it when you can…

…be asked to mask phone numbers for the analyst role, and answer with three options — a masking policy, a precomputed column, and a Lambda UDF — and say which one you would pick and why.

MODULE 01 · LESSON 39 · ★ THE ONE FOR YOU

Calling Redshift From Node.js

Two clients, two completely different mental models. Choosing the wrong one is the most expensive mistake in this module.

1 · WHAT IT IS

Data API for jobs. A pg pool for interactive reads.

The Data API is HTTPS, asynchronous and IAM-authenticated — no VPC, no pool, no password. node-postgres is a real connection you must manage.

2 · HOW IT WORKS

THE FOUR HABITS TO UNLEARN

NO ROW-AT-A-TIME INSERTS — `write to S3, then COPY`

A loop of INSERTs is the number one way to destroy a Redshift cluster. Batch to S3 and COPY, always.

SUBMIT, POLL, FETCH — `the Data API is not await-a-result`

ExecuteStatement returns an Id immediately. You poll DescribeStatement, then call GetStatementResult.

NO SESSION STATE BY DEFAULT — `unless you pass SessionId`

Each Data API call may land on a new session, so temp tables and SET vanish. Use BatchExecuteStatement or reuse a session.

PARAMETERS, NEVER CONCATENATION — `injection and cache misses`

Interpolated literals are injectable and produce a new query shape every call, so nothing is ever compiled twice.

```
Redshift is not your OLTP database. Do not treat it like one.
```

3 · THE CODE

```
// submit — returns an Id, not rows
const { Id } = await rs.send(
new ExecuteStatementCommand({
WorkgroupName, Database, Sql,
Parameters: [{ name: 'd',
value: batchDate }],
}));
```

4 · GOTCHAS

Data API parameters are all strings, and results come back as typed union fields. Results paginate — follow NextToken.

a serverless connection is not free to open

IN PLAIN ENGLISH

You post the job and get a ticket number. You come back for the result. It is a courier, not a phone call — and that changes how you write the code around it.

MODULE 01 · LESSON 39 · ★ THE ONE FOR YOU

L39 · Calling Redshift From Node.js ★

Slide: `_render/L39-nodejs-integration.html`

What it is

Two clients, two completely different mental models. **Choosing the wrong one is the most expensive mistake in this module.**

	REDSHIFT DATA API	NODE-POSTGRES (PG)
Transport	HTTPS, AWS SDK	PostgreSQL wire protocol, TCP 5439
Auth	IAM	Username/password or IAM temp credentials
Network	Public AWS endpoint — no VPC needed	Must reach the cluster: VPC, SG, or public endpoint
Connections	None to manage	A pool you own and must size
Model	Submit → poll → fetch	`await` returns rows
Result limit	Paginated, size-capped	Streams, unbounded
Session state	None, unless you ask for it	Full session
Best for	**Lambda, Step Functions, ETL jobs, anything serverless**	**A long-lived API service doing interactive reads**

Default to the Data API. Reach for `pg` when you have a persistent service and genuinely need session state, streaming, or sub-100 ms latency.

The four habits to unlearn

1 · No row-at-a-time inserts

```
// ✕✕✕ this will destroy a Redshift cluster
for (const row of rows) {
  await client.query('INSERT INTO gold.fct_sales_line VALUES ($1,$2,$3)', row);
}
```

Every `INSERT` writes new 1 MB column blocks (L23). Ten thousand single-row inserts produce ten thousand near-empty blocks, the table balloons, every subsequent scan reads the bloat, and `VACUUM` has to clean it up. This is not "slow" — it is *destructive*, and the damage outlives the job.

```
// ✅ batch to S3, then COPY
import { PutObjectCommand } from '@aws-sdk/client-s3';

const body = rows.map(r => JSON.stringify(r)).join('\n');
await s3.send(new PutObjectCommand({
  Bucket: 'tamimi-staging', Key: `sales/${batchDate}/part-0001.json`, Body: body,
}));

await execute(`
  COPY staging.sales_line
  FROM 's3://tamimi-staging/sales/${batchDate}/'
  IAM_ROLE '${ROLE}'
  FORMAT AS JSON 'auto'
  TIMEFORMAT 'auto'
`);
```

Write the file, then `COPY`. Every time. There is no batch size at which the loop becomes acceptable.

If you truly have a handful of rows and no S3 path, a **multi-row `INSERT`** is the least-bad option — one statement, many `VALUES` tuples:

```
// acceptable for tens of rows, never for thousands
const values = rows.map((_, i) => `($${i*3+1}, $${i*3+2}, $${i*3+3})`).join(',');
await client.query(`INSERT INTO etl.run_log VALUES ${values}`, rows.flat());
```

2 · Submit, poll, fetch

The Data API is **not** await-a-result. `ExecuteStatement` returns an `Id` immediately.

L39 · Calling Redshift From Node.js ★

```
// db/dataApi.mjs
import {
  RedshiftDataClient, ExecuteStatementCommand,
  DescribeStatementCommand, GetStatementResultCommand,
} from '@aws-sdk/client-redshift-data';

const rs = new RedshiftDataClient({ region: 'me-central-1' });

const BASE = {
  WorkgroupName: 'tamimi-wg',       // or ClusterIdentifier + DbUser for provisioned
  Database: 'tamimi',
};

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

/** Submit SQL and wait for it to finish. Returns the statement id. */
export async function execute(sql, parameters = []) {
  const { Id } = await rs.send(new ExecuteStatementCommand({
    ...BASE,
    Sql: sql,
    Parameters: parameters.length ? parameters : undefined,
    StatementName: sql.slice(0, 60),   // shows up in the console — worth setting
  }));

  // poll with backoff: cheap queries finish fast, loads do not
  let delay = 250;
  for (;;) {
    const d = await rs.send(new DescribeStatementCommand({ Id }));
    if (d.Status === 'FINISHED') return { Id, rows: d.ResultRows, ms: d.Duration / 1e6 };
    if (d.Status === 'FAILED')   throw new Error(`[${Id}] ${d.Error}`);
    if (d.Status === 'ABORTED')  throw new Error(`[${Id}] aborted`);
    await sleep(delay);
    delay = Math.min(delay * 1.5, 5000);
  }
}
```

Poll with backoff. A fixed 100 ms poll on a 40-minute load is 24,000 pointless API calls and will get you throttled.

3 · No session state by default

Each Data API call may land on a **new session**. Temp tables vanish, `SET` reverts, and a transaction cannot span two calls.

Three ways to handle it:

`BatchExecuteStatement` — several statements, one session, one transaction:

```
import { BatchExecuteStatementCommand } from '@aws-sdk/client-redshift-data';

const { Id } = await rs.send(new BatchExecuteStatementCommand({
  ...BASE,
  Sqls: [
    `CREATE TEMP TABLE t AS SELECT * FROM staging.sales_line WHERE sale_date = '${d}'`,
    `DELETE FROM gold.fct_sales_line WHERE sale_date = '${d}'`,
    `INSERT INTO gold.fct_sales_line SELECT * FROM t`,
  ],
}));
```

All three run in order, in one session, in one transaction. If the third fails, the second rolls back.

Reuse a session by passing `SessionId` from a previous call's response — `SessionKeepAliveSeconds` controls how long it survives.

Or, better: put the multi-statement logic in a stored procedure (L35) and have Node issue a single `CALL`. The session problem disappears, the SQL is in git, and the transaction boundary is explicit.

L39 · Calling Redshift From Node.js ★

4 · Parameters, never string concatenation

```
// ❌ injectable, and a new query shape every call — nothing is ever compiled twice (L34)
await execute(`SELECT * FROM gold.fct_sales_line WHERE sale_date = '${req.query.d}'`);

// ✅
await execute(
  'SELECT store_sk, SUM(net_amount) FROM gold.fct_sales_line WHERE sale_date = :d GROUP BY 1',
  [{ name: 'd', value: req.query.d }],
);
```

⚠️ **Data API parameters are named (`:d`), not positional (`$1`)** — unlike `pg`. And **every parameter value is a string**; Redshift casts it against the column type. A bad date string becomes a runtime error, not a type error, so validate at the edge.

Reading results

```
export async function query(sql, parameters = []) {
  const { Id } = await execute(sql, parameters);

  const out = [];
  let NextToken;
  do {
    const r = await rs.send(new GetStatementResultCommand({ Id, NextToken }));
    const cols = r.ColumnMetadata.map((c) => c.name);
    for (const rec of r.Records) {
      out.push(Object.fromEntries(rec.map((f, i) => [cols[i], unwrap(f)])));
    }
    NextToken = r.NextToken;          // ⚠️ results paginate — you must follow this
  } while (NextToken);

  return out;
}

/** Data API returns a typed union per field: {stringValue}, {longValue}, {isNull} ... */
function unwrap(f) {
  if (f.isNull) return null;
  return f.stringValue ?? f.longValue ?? f.doubleValue ?? f.booleanValue ?? f.blobValue ?? null;
}
```

Two traps in that snippet, both of which bite in production:

- **Results paginate.** Forgetting `NextToken` gives you a silently truncated answer — no error, just missing rows. This is the single nastiest Data API bug because it looks like a data problem.
- **Fields are a typed union**, not plain values. `unwrap` is not optional. Note the `??` chain treats a real `0` / `false` correctly only because `??` checks for null/undefined — do not "simplify" it to `||`.

The Data API is not a bulk export. For large result sets, `UNLOAD` to S3 and read the files (L27).

The `pg` alternative

```
// db/pool.mjs
import pg from 'pg';

export const pool = new pg.Pool({
  host: 'tamimi-wg.123456789012.me-central-1.redshift-serverless.amazonaws.com',
  port: 5439,
  database: 'tamimi',
  user: process.env.RS_USER,
  password: process.env.RS_PASSWORD,       // or IAM temp credentials, below
  ssl: { rejectUnauthorized: true },
  max: 10,                                 // ⚠️ small. see below.
  idleTimeoutMillis: 30_000,
  connectionTimeoutMillis: 10_000,
  statement_timeout: 120_000,              // do not let a runaway query hold a slot
});

export const q = (text, values) => pool.query(text, values);   // $1, $2 here
```

L39 · Calling Redshift From Node.js ★

Keep the pool small. Redshift's concurrency is governed by WLM slots (L41), not by connections — typically 5–15 concurrent queries for the whole cluster. A pool of 100 does not give you 100× throughput; it gives you 100 queued queries and a much harder problem to debug. `max: 10` per service instance is a sane start.

Prefer **IAM temp credentials** over a stored password:

```
import { RedshiftServerlessClient, GetCredentialsCommand } from '@aws-sdk/client-redshift-serverless';

const rss = new RedshiftServerlessClient({ region: 'me-central-1' });
const { dbUser, dbPassword, expiration } = await rss.send(
  new GetCredentialsCommand({ workgroupName: 'tamimi-wg', dbName: 'tamimi', durationSeconds: 3600 }),
);
```

Cache them and refresh before `expiration`. No long-lived secret in the environment.

Streaming a large result

```
import QueryStream from 'pg-query-stream';

const client = await pool.connect();
try {
  const stream = client.query(new QueryStream('SELECT * FROM gold.fct_sales_line WHERE sale_date = $1',
[d]));
  for await (const row of stream) {
    // constant memory
  }
} finally {
  client.release();          // ⚠ always, in a finally
}
```

This is the one thing `pg` does that the Data API cannot.

The shape of a real ETL job

```
// jobs/nightly.mjs
import { execute, query } from '../db/dataApi.mjs';

export async function handler(event) {
  const batchDate = event.batchDate;                    // ⚠ passed IN, never CURRENT_DATE (L32)
  const started = new Date().toISOString();

  try {
    // 1. land the files (upstream job, or here)
    // 2. one CALL — all the SQL logic lives in the warehouse, in git
    await execute('CALL etl.run_nightly(:d)', [{ name: 'd', value: batchDate }]);

    // 3. verify, do not assume
    const [{ n }] = await query(
      'SELECT COUNT(*) AS n FROM gold.fct_sales_line WHERE sale_date = :d',
      [{ name: 'd', value: batchDate }],
    );
    if (Number(n) === 0) throw new Error(`no rows landed for ${batchDate}`);

    await audit({ batchDate, status: 'SUCCESS', rows: Number(n), started });
    return { ok: true, rows: Number(n) };
  } catch (err) {
    await audit({ batchDate, status: 'FAILED', error: err.message, started });   // separate txn (L36)
    throw err;                                          // let Step Functions see the failure
  }
}
```

Four things worth copying out of that:

1. **The batch date is a parameter**, so a rerun of Tuesday loads Tuesday (L32).
2. **One `CALL`.** The SQL lives in the warehouse and in git, not in a template literal.
3. **Verify, then audit.** A job that reports success without checking is not a job, it is a hope.
4. **Audit the failure on a separate call** so it survives the rollback (L36), and re-throw so the orchestrator knows.

L39 · Calling Redshift From Node.js ★

Gotchas

- **Row-at-a-time inserts damage the cluster**, not just the job.
- `NextToken` **— results paginate.** Silent truncation.
- **Fields are typed unions.** Unwrap them.
- **Data API params are** `:named` **;** `pg` **params are** `$1` **.** Mixing them up is a confusing error.
- **All Data API parameter values are strings.**
- **No session state across Data API calls** unless you pass `SessionId` or use `BatchExecuteStatement` .
- **The Data API has quotas** — statement size, result size, requests per second. Read them before designing a fan-out.
- `pg` **pools must be small.** WLM slots are the real limit.
- **Always** `client.release()` **in a** `finally` **.** A leaked client eventually deadlocks the pool.
- **Serverless auto-pause** means the first query after idle takes seconds. Set generous connect timeouts.
- **Statement timeout on the** `pg` **pool.** Without it, one bad query holds a slot forever.
- **Redshift is not your OLTP database.** No per-request writes, no key-value lookups in a hot path, no ORM sync.

Try it

1. Write `execute` and `query` with backoff and pagination. Run a query returning 5,000 rows and confirm you get all of them — then delete the `NextToken` loop and see how many you get.
2. Load 10,000 rows two ways: a loop of `INSERT` s, and S3 + `COPY` . Time both, then check `svv_table_info` for `unsorted` and `size` on each target. Show the class the block count.
3. Convert a three-statement job to one `CALL` and prove the failure now rolls back cleanly.
4. Switch from a password to `GetCredentials` temp credentials.
5. Point a `pg` pool with `max: 100` at the cluster, fire 200 concurrent queries, and watch the queue in `sys_query_history` . Then set `max: 10` and compare total throughput. It will not be worse.

Exercises 2 and 5 are the ones that change behaviour.

Checklist

- ☐ Data API for jobs and Lambda; `pg` only for a long-lived service
- ☐ No row-at-a-time inserts anywhere in the codebase
- ☐ Polling uses exponential backoff
- ☐ `NextToken` followed on every result read
- ☐ Typed-union fields unwrapped
- ☐ Named parameters, never interpolation
- ☐ Multi-statement logic lives in a stored procedure
- ☐ Batch date passed in, never derived inside the query
- ☐ Every job verifies its own output before reporting success
- ☐ Failure audit written on a separate call, then re-thrown
- ☐ `pg` pool `max` ≤ 10, `statement_timeout` set, `release()` in a `finally`
- ☐ IAM temp credentials, no stored password

L39 · Calling Redshift From Node.js ★

You've got it when you can…

…review a teammate's pull request, spot the `for (const row of rows) await insert(row)`, and explain — with the block-count evidence from exercise 2 — why it has to be S3 and `COPY`.

MODULE 01 · LESSON 40

Scheduling and Automation

Four ways to make a load run at 2 a.m. They differ mainly in what happens when step four fails at 2:40.

1 · WHAT IT IS

Choose by failure behaviour, not by convenience

Anything can start a job on a timer. What separates these options is retry, branching, visibility and what you can see at 8 a.m. when the report is empty.

2 · HOW IT WORKS

FOUR OPTIONS, WEAKEST TO STRONGEST

QUERY EDITOR SCHEDULE — `one statement, no branching`

Fine for a nightly VACUUM or a single CALL. No dependencies, no retry logic, and easy to forget it exists.

EVENTBRIDGE → LAMBDA — `code, logs, alarms`

A cron rule invoking your Node handler. Real logging and error handling, but you write the orchestration yourself.

STEP FUNCTIONS — THE DEFAULT — `retry, catch, parallel, visible`

Declarative retry and catch per step, a picture of last night's run, and it calls the Data API directly with no Lambda in between.

MWAA / AIRFLOW — `powerful, and a cluster to run`

Worth it once you have cross-system dependencies and a team that already knows it. Otherwise it is infrastructure you now own.

```
a job that cannot be safely rerun is not scheduled, it is gambled
```

3 · THE PATTERN

```
// Step Functions calls this directly
"Resource": "arn:aws:states:::
aws-sdk:redshiftdata:
executeStatement",
// then poll describeStatement
"Retry": [{ IntervalSeconds: 60,
MaxAttempts: 3 }]
```

4 · GOTCHAS

A retry only helps if the job is idempotent.
Alert on the job that did not run at all —
silence is the failure you will miss.

schedules are UTC. always.

IN PLAIN ENGLISH

Starting the job is the easy part. You are choosing who wakes up, what they can see, and whether it is safe to simply press play again.

L40 · Scheduling and Automation

Slide: `_render/L40-scheduling-automation.html`

What it is

Four ways to make a load run at 2 a.m. They differ mainly in **what happens when step four fails at 2:40**.

Anything can start a job on a timer. What separates these options is retry, branching, visibility, and what you can see at 8 a.m. when the report is empty.

The four options

1 · Query editor v2 schedule — weakest

A scheduled statement, configured in the console, backed by EventBridge Scheduler.

Good for: a nightly `VACUUM`, an `ANALYZE`, a single `CALL`. **Not good for:** anything with dependencies, retries, or branching.

⚠️ Its real weakness is **discoverability**. A schedule created in a console by someone who has since left the project is invisible to your repo, your runbook, and the next engineer. If you use it, document it in the repo.

2 · EventBridge → Lambda

A cron rule invoking your Node handler (L39).

```
{ "ScheduleExpression": "cron(0 23 * * ? *)",
  "Target": { "Arn": "arn:aws:lambda:me-central-1:...:function:tamimi-nightly" } }
```

Good for: a single job with real logging, error handling and CloudWatch alarms. **Watch:** Lambda's 15-minute maximum. A load that outgrows it must move to the async pattern — submit the statement, return, and let a second trigger handle completion — or to Step Functions.

You write the orchestration yourself, which is fine for one job and painful for eight.

3 · Step Functions — the default recommendation

Declarative retry and catch per step, a **picture of last night's run**, and it calls the Data API directly — no Lambda in between.

L40 · Scheduling and Automation

```
{
  "StartAt": "LoadDimensions",
  "States": {
    "LoadDimensions": {
      "Type": "Task",
      "Resource": "arn:aws:states:::aws-sdk:redshiftdata:executeStatement",
      "Parameters": {
        "WorkgroupName": "tamimi-wg",
        "Database": "tamimi",
        "Sql.$": "States.Format('CALL etl.load_dimensions(''{}'')', $.batchDate)"
      },
      "Retry": [{ "ErrorEquals": ["States.ALL"], "IntervalSeconds": 60, "MaxAttempts": 3,
"BackoffRate": 2 }],
      "Catch": [{ "ErrorEquals": ["States.ALL"], "Next": "NotifyFailure" }],
      "Next": "WaitForDimensions"
    },
    "WaitForDimensions": { "Type": "Wait", "Seconds": 15, "Next": "CheckDimensions" },
    "CheckDimensions": {
      "Type": "Task",
      "Resource": "arn:aws:states:::aws-sdk:redshiftdata:describeStatement",
      "Parameters": { "Id.$": "$.Id" },
      "Next": "DimensionsDone?"
    },
    "DimensionsDone?": {
      "Type": "Choice",
      "Choices": [
        { "Variable": "$.Status", "StringEquals": "FINISHED", "Next": "LoadFacts" },
        { "Variable": "$.Status", "StringEquals": "FAILED",   "Next": "NotifyFailure" }
      ],
      "Default": "WaitForDimensions"
    }
  }
}
```

That submit → wait → describe → choice loop is the **standard Redshift Step Functions idiom**. You will write it once and copy it forever. It exists because `executeStatement` is asynchronous (L39) — Step Functions is polling on your behalf instead of a Lambda burning billable milliseconds doing nothing.

Why this is the default: when it fails, you open the console and see a diagram with one red box. That is worth more at 8 a.m. than any amount of log searching.

4 · MWAA / Airflow

Worth it once you have **cross-system dependencies** — "wait for the SAP extract, then the S3 landing, then Redshift, then trigger the Power BI refresh" — and a team that already knows Airflow.

Otherwise it is a cluster you now own, patch and pay for. For a three-person team starting out, Step Functions is the better trade.

Idempotency is the prerequisite

> **A job that cannot be safely rerun is not scheduled, it is gambled.**

Every retry in every option above assumes a rerun is harmless. Make it true:

```
-- the pattern from L24 and L35 — delete the slice, reload it
DELETE FROM gold.fct_sales_line WHERE sale_date = p_batch_date;
INSERT INTO gold.fct_sales_line SELECT ... WHERE sale_date = p_batch_date;
```

Run it three times, get the same table. Then a retry is free and you can automate confidently.

What to alert on

Most teams alert on failure and stop there. **The three that matter:**

L40 · Scheduling and Automation

ALERT	WHY IT MATTERS
The job failed	Obvious, and the least dangerous — you know about it
The job did not run at all	Silence. Nothing failed because nothing started. This is the one that reaches the business first
The job succeeded with zero rows	The worst kind: green tick, empty report

The second is what a **heartbeat** catches:

```
-- run this on its own schedule, independent of the load
SELECT batch_date, status, rows_loaded, ended_at_utc
FROM   etl.run_log
WHERE  batch_date = DATEADD('day', -1, CURRENT_DATE);
-- zero rows = the job never started. alert.
```

Point an EventBridge rule at that check at 06:00 and you find out before the business does.

The third is what the quality gate in your procedure catches (L35):

```
IF v_rows = 0 THEN RAISE EXCEPTION 'no rows loaded for %', p_batch_date; END IF;
```

The run log

Everything above depends on this table existing:

```
CREATE TABLE etl.run_log (
    run_id        BIGINT      NOT NULL,
    job_name      VARCHAR(80) NOT NULL,
    batch_date    DATE        NOT NULL,
    status        VARCHAR(20) NOT NULL,   -- RUNNING | SUCCESS | FAILED
    rows_loaded   BIGINT,
    error_text    VARCHAR(2000),
    started_at_utc TIMESTAMP  NOT NULL,
    ended_at_utc   TIMESTAMP,
    triggered_by  VARCHAR(80)             -- the execution arn, so you can find the run
)
DISTSTYLE ALL
SORTKEY (batch_date, job_name);
```

`triggered_by` is small and worth having: given a bad number in a report, you can go from the row to the batch to the exact Step Functions execution.

Schedules are UTC

`cron(0 23 * * ? *)` fires at **23:00 UTC = 02:00 Riyadh**. EventBridge Scheduler does support time zones and DST, and plain EventBridge rules do not — but the safer habit is to **think in UTC, write UTC, and put the local time in a comment**:

```
cron(0 23 * * ? *)   # 23:00 UTC = 02:00 Asia/Riyadh (UTC+3, no DST)
```

Same discipline as L32. And schedule the load to start *after* the source systems have finished their own day — a nightly load that begins at local midnight often catches the last hour of trade only in tomorrow's run.

L40 · Scheduling and Automation

Gotchas

- **A retry only helps if the job is idempotent.**
- **Alert on the job that did not run.** Silence is the failure you will miss.
- **Alert on zero rows.** A green tick on an empty load is worse than a failure.
- **Schedules are UTC.**
- **Lambda caps at 15 minutes.** Design for it or use Step Functions.
- **Console-created schedules are invisible to your repo.** Document or avoid.
- **Step Functions has a payload size limit** — pass S3 keys, not data.
- **Concurrent executions of the same load** will fight over the same table. Set concurrency to 1, or use a lock row.
- **The Data API is async** — a Step Functions task that does not poll `describeStatement` will report success the moment the statement is *submitted*. This is a real and common bug: the job goes green while the SQL is still running or already failing.

That last one is worth reading twice.

Try it

1. Build the Step Functions state machine for one load, including the wait/describe/choice loop. Break the SQL and watch the retry then the catch.
2. Add the `etl.run_log` table and populate it from the job (L39).
3. Write the heartbeat check and schedule it two hours after the load. Disable the load for one night and confirm the heartbeat alerts.
4. Run the same load three times for the same date. Row count must be identical.
5. Deliberately omit the `describeStatement` poll and watch the state machine go green on a query that fails. Then put it back.

Checklist

- ☐ Every scheduled job is idempotent for its batch date
- ☐ Step Functions for anything with more than one step
- ☐ The submit → wait → describe → choice loop is in place
- ☐ Retry with backoff, and a catch that notifies
- ☐ `etl.run_log` written by every job
- ☐ Alerts on: failed, did-not-run, and zero-rows
- ☐ Schedules written in UTC with the local time in a comment
- ☐ Concurrency limited to one execution per load
- ☐ Schedules live in IaC, in the repo — not in a console

You've got it when you can…

…be told at 08:00 that a dashboard is empty, and know within a minute whether the job failed, never ran, or ran perfectly against no data — because you built the alert for each one separately.

Part F complete. L35–L40 covered procedures, control flow, UDFs, Lambda UDFs, the Node.js integration and scheduling. Part G is operations: WLM, maintenance, the system tables, diagnosing a slow query, and cost.

MODULE 01 · LESSON 41 · PART G

Workload Management

Redshift runs a handful of queries at once on purpose. Everything else queues. WLM is how you decide who waits.

1 · WHAT IT IS

Memory is the scarce resource, not CPU

A slot is a share of memory. More slots means smaller shares and more spilling to disk. Fewer, larger slots almost always beat many small ones.

2 · HOW IT WORKS

FOUR LEVERS, IN THE ORDER YOU SHOULD REACH FOR THEM

AUTO WLM — LEAVE IT ON — `the right default`

Redshift sizes memory and concurrency per query from its own history. Manual WLM is for a problem you can prove auto cannot solve.

QUEUES AND QUERY PRIORITY — `separate ETL from dashboards`

Route by user group or query group, then set priority. A dashboard should not sit behind a backfill.

QMR — RULES THAT ACT — `log, hop, or abort`

Abort anything scanning a billion rows from the dashboard queue. This is how you stop one bad query ruining a morning.

CONCURRENCY SCALING — `bursts, and it bills`

Extra clusters spin up for queued read queries. One free hour per day of accrued credit; beyond that it costs.

```
fix the query before you tune the queue
```

3 · THE SQL

```sql
-- am I queuing, or just slow?
SELECT query_id, queue_time/1e6,
execution_time/1e6,
service_class
FROM sys_query_history
WHERE queue_time > 0
ORDER BY 2 DESC LIMIT 20;
```

4 · GOTCHAS

More slots is not more throughput — it is less memory each and more disk spill. Concurrency scaling never helps writes.

a big pg pool does not buy concurrency

IN PLAIN ENGLISH

Four wide checkouts move a supermarket faster than twenty narrow ones where nobody can put their basket down. Slots are counter space.

L41 · Workload Management and Concurrency

Slide: `_render/L41-wlm-concurrency.html`

What it is

Redshift runs **a handful of queries at once, on purpose**. Everything else queues. WLM is how you decide who waits.

This is the fact that surprises application developers most. In an OLTP database you scale by adding connections. Here, adding connections adds queue depth and nothing else.

Memory is the scarce resource, not CPU. A slot is a share of memory. More slots means smaller shares and more spilling to disk (L28). **Fewer, larger slots almost always beat many small ones.**

1 · Auto WLM — leave it on

Redshift sizes memory and concurrency per query from its own execution history. It gets this right far more often than a hand-tuned configuration, and it adapts when your workload changes.

Manual WLM is for a problem you can prove auto cannot solve. In practice that is rare, and if you are new to Redshift it is never your first move.

```
-- what auto WLM has been deciding
SELECT service_class,
       COUNT(*)                          AS queries,
       AVG(queue_time)   / 1e6           AS avg_queue_secs,
       MAX(queue_time)   / 1e6           AS max_queue_secs,
       AVG(execution_time) / 1e6         AS avg_exec_secs
FROM   sys_query_history
WHERE  start_time >= DATEADD('day', -1, GETDATE())
GROUP  BY 1 ORDER BY 1;
```

2 · Queues and query priority

Route by **user group** or **query group**, then set a priority per queue. The one separation worth making on almost every cluster:

QUEUE	MEMBERS	PRIORITY
`etl`	the load service account	`high` during the batch window
`bi`	the Power BI / dashboard account	`normal`
`adhoc`	analysts	`low`

A dashboard should not sit behind a backfill. That is the whole argument.

Query groups let a session label itself:

```
SET query_group TO 'etl';
CALL etl.run_nightly('2026-08-10');
RESET query_group;
```

Priorities are `highest`, `high`, `normal`, `low`, `lowest`. Use `highest` sparingly — if everything is highest, nothing is.

3 · Query Monitoring Rules — rules that act

A QMR watches a metric and takes an action: **log**, **hop** (move to another queue), or **abort**.

Rules worth having from day one:

L41 · Workload Management and Concurrency

RULE	PREDICATE	ACTION
Runaway dashboard query	`query_execution_time > 900` in the `bi` queue	`abort`
Enormous scan	`scan_row_count > 1e10`	`log` then `abort`
Nested loop	`nested_loop_join_row_count > 1e6`	`abort`
Heavy spill	`query_temp_blocks_to_disk > 100000`	`log`

The nested-loop rule is the best value in the table. A nested loop over a million rows is almost always the accidental cross join from L29, and aborting it in the first minute saves an hour.

This is how you stop one bad query ruining a morning — and it is much better than an engineer noticing and cancelling by hand.

4 · Concurrency scaling

Extra clusters spin up automatically to handle **queued read queries**, and results come back as if they ran on the main cluster.

It is a **WLM queue setting** (or a workgroup setting on Serverless), configured in the parameter group or console — not a session `SET`. Facts to hold on to:

- You accrue **one hour of free concurrency-scaling credit per day** of main-cluster runtime, up to a cap. Beyond that it bills at the per-second on-demand rate.
- It helps **reads**. Write-heavy workloads see nothing.
- It is not a fix for a badly designed query. It is a fix for **too many acceptable queries at once**.

```
-- how much am I using?
SELECT DATE_TRUNC('hour', start_time) AS hr,
       SUM(CASE WHEN concurrency_scaling_status = 1 THEN 1 ELSE 0 END) AS on_scaling,
       COUNT(*) AS total
FROM   sys_query_history
WHERE  start_time >= DATEADD('day', -7, GETDATE())
GROUP  BY 1 ORDER BY 1;
```

On **Serverless**, this is mostly moot — RPU auto-scaling handles burst, and you set base and max RPUs instead of queues. `max_query_execution_time` on the workgroup is your equivalent of the runaway rule.

Am I queuing, or just slow?

The first question to ask about any "Redshift is slow" report:

```
SELECT query_id,
       LEFT(query_text, 60)      AS sql,
       queue_time / 1e6          AS queue_secs,
       execution_time / 1e6      AS exec_secs,
       service_class,
       status
FROM   sys_query_history
WHERE  start_time >= DATEADD('hour', -2, GETDATE())
ORDER  BY queue_time DESC
LIMIT  20;
```

- `queue_secs` **high,** `exec_secs` **low** → a concurrency problem. WLM, priorities, or concurrency scaling.
- `queue_secs` **low,** `exec_secs` **high** → a query problem. Go to L44; WLM will not help you.

Getting this diagnosis backwards is the most common waste of an afternoon in Redshift operations.

L41 · Workload Management and Concurrency

Gotchas

- **More slots is not more throughput.** It is less memory each and more disk spill.
- **A big `pg` pool does not buy concurrency.** Ten queries in flight is ten queries in flight regardless of a hundred open connections (L39).
- **Concurrency scaling never helps writes.**
- **`highest` priority for everything is no priority.**
- **QMR `abort` kills the query with an error the user sees.** Tell people the rules exist, or the abort looks like a bug.
- **Manual WLM memory percentages must sum to 100** and a wrong split can starve a queue completely.
- **Short-query acceleration** helps dashboards but can mask a genuinely slow query in the averages.
- **`SET query_group` is per session** — and Data API calls may not share a session (L39).
- **Fix the query before you tune the queue.** Tuning WLM around a broadcast join is treating a symptom.

Try it

1. Run the queue-vs-exec query for the last 24 hours. Classify your ten slowest queries into "queuing" and "slow".
2. Set up `etl` / `bi` / `adhoc` query groups and route a session into each.
3. Add the nested-loop QMR rule, then run the accidental cross join from L29 and watch it get aborted.
4. Fire 50 concurrent dashboard queries and watch `queue_time` rise, then enable concurrency scaling and compare.
5. Find out whether your cluster has ever had a quiet period — it determines whether auto vacuum is running at all (L42).

Checklist

- ☐ Auto WLM left on unless there is a proven reason
- ☐ ETL, BI and ad-hoc separated into queues with sensible priorities
- ☐ QMR rules for nested loops, huge scans and runaway dashboard queries
- ☐ Concurrency scaling enabled for the BI queue, and its usage monitored
- ☐ I check `queue_time` vs `execution_time` before touching anything
- ☐ `pg` pools sized to WLM reality, not to request volume
- ☐ The team knows QMR aborts exist and what they mean

You've got it when you can…

…hear "Redshift is slow this morning", run one query, and say within a minute whether it is a concurrency problem or a query problem — and be right.

MODULE 01 · LESSON 42

VACUUM, ANALYZE and Auto Maintenance

Deleted rows are only marked, not removed. Statistics go stale. Redshift now handles most of this — but you must know when it has not.

1 · WHAT IT IS

Two different jobs that people confuse constantly

VACUUM reclaims space and restores sort order. ANALYZE refreshes the statistics the planner uses. A slow query is far more often stale stats than unsorted data.

2 · HOW IT WORKS

WHAT IS AUTOMATIC, AND WHAT IS YOURS

AUTO VACUUM AND AUTO ANALYZE — `run in idle periods`

Both run in the background when the cluster is quiet. On a busy cluster with no quiet period, they may never get their turn.

CHECK, DO NOT ASSUME — `svv_table_info`

Two numbers tell you everything: unsorted percent and stats_off. Anything above ten on either deserves a look.

ANALYZE AFTER EVERY BIG LOAD — `it is cheap, do it explicitly`

Put it at the end of the procedure. Analyzing only the predicate columns is faster than the whole table.

DEEP COPY BEATS A HUGE VACUUM — `rebuild instead of repair`

A badly unsorted large table rebuilds faster than it vacuums. CREATE, INSERT SELECT ordered, swap the names.

```
TRUNCATE and DROP need no vacuum. DELETE does.
```

3 · THE SQL

```
-- the two numbers that matter
SELECT "table", size, unsorted,
stats_off, tbl_rows
FROM svv_table_info
WHERE unsorted > 10
OR stats_off > 10
ORDER BY size DESC;
```

4 · GOTCHAS

VACUUM takes a lock that blocks writes on that table. Never during the load window.

Only one VACUUM runs at a time, cluster-wide.

DELETE without VACUUM grows the table

IN PLAIN ENGLISH

Deleting a row crosses it off the list rather than tearing out the page. VACUUM rebinds the book. ANALYZE updates the index card the planner reads.

L42 · VACUUM, ANALYZE and Auto Maintenance

Slide: `_render/L42-vacuum-analyze.html`

What it is

Two different jobs that people confuse constantly:

- **`VACUUM`** reclaims space from deleted rows and restores sort order.
- **`ANALYZE`** refreshes the statistics the planner uses.

A slow query is far more often stale statistics than unsorted data. If you only ever do one, do `ANALYZE`.

Why they are needed at all

`DELETE` does not remove a row — it marks it deleted (L23). `UPDATE` is a delete plus an insert. And every `INSERT` lands in the **unsorted region** at the end of the table, outside the sort order.

So after a month of loads:

- The table is larger than its data, and every scan reads the dead space.
- New rows are not in sort-key order, so zone maps cannot prune them (L16).
- Row counts and value distributions no longer match what the planner believes.

Check, do not assume

Two numbers tell you everything:

```
SELECT "schema", "table",
       size            AS mb,          -- 1 MB blocks
       tbl_rows,
       unsorted,                       -- % of rows outside the sort order
       stats_off,                      -- % staleness of the statistics
       vacuum_sort_benefit             -- estimated % scan improvement from vacuuming
FROM   svv_table_info
WHERE  unsorted > 10 OR stats_off > 10
ORDER  BY size DESC;
```

- **`stats_off > 10`** → run `ANALYZE`. Cheap, safe, do it now.
- **`unsorted > 10`** *and* **`vacuum_sort_benefit`** meaningfully above zero → consider `VACUUM`.

`vacuum_sort_benefit` is the column that stops you vacuuming pointlessly. A table can be 40% unsorted and gain almost nothing, because nothing filters on its sort key.

ANALYZE

```
ANALYZE gold.fct_sales_line;                          -- whole table
ANALYZE gold.fct_sales_line (sale_date, store_sk);    -- just the predicate columns — faster
ANALYZE;                                               -- everything, rarely what you want
ANALYZE VERBOSE gold.fct_sales_line;                   -- tells you what it did
```

Put it at the end of every load procedure (L35). It is cheap and it prevents the most common cause of a mysteriously slow morning:

```
INSERT INTO gold.fct_sales_line SELECT ...;
ANALYZE gold.fct_sales_line (sale_date, store_sk);
```

Analyzing only the columns that appear in `WHERE`, `JOIN` and `GROUP BY` is markedly faster than the whole table and gives the planner everything it needs.

Temp tables get no statistics at all until you analyze them (L31). This is worth repeating because it is invisible: the query is correct, it is just planned as though the temp table had a default row count.

```
-- what has been analyzed, and when
SELECT * FROM svv_table_info WHERE stats_off > 0 ORDER BY stats_off DESC;
```

L42 · VACUUM, ANALYZE and Auto Maintenance

VACUUM

```
VACUUM gold.fct_sales_line;                      -- FULL: reclaim + resort
VACUUM DELETE ONLY gold.fct_sales_line;          -- reclaim space, do not resort
VACUUM SORT ONLY gold.fct_sales_line;            -- resort, do not reclaim
VACUUM REINDEX gold.dim_store;                   -- interleaved sort keys only
VACUUM FULL gold.fct_sales_line TO 95 PERCENT;   -- stop at 95% sorted — much faster
```

`TO 95 PERCENT` **is the practical option.** The last 5% of sorting costs a disproportionate share of the time and buys very little.

⚠️ `VACUUM` **takes a lock that blocks writes on that table.** Never during the load window. And **only one** `VACUUM` **runs at a time cluster-wide**, so a queue of them serialises.

It is also restartable — if you cancel it, the work done so far is kept.

What is automatic

FEATURE	WHAT IT DOES	CATCH
Auto vacuum delete	Reclaims space from deleted rows	Runs in idle periods
Auto table sort	Incrementally restores sort order	Runs in idle periods
Auto analyze	Refreshes stats after significant change	Runs in idle periods
Auto vacuum sort	Sorts the highest-benefit tables	Prioritises by `vacuum_sort_benefit`

The catch is the same in every row: idle periods. On a busy cluster that never goes quiet, these may never get their turn — and Redshift will not tell you. That is why you still check `svv_table_info`.

```
-- has automatic maintenance actually been running?
SELECT * FROM svl_auto_worker_action ORDER BY eventtime DESC LIMIT 50;
```

An empty or stale result on a busy cluster is your answer, and the fix is either a quieter window or explicit maintenance.

The deep copy — rebuild instead of repair

A badly unsorted large table **rebuilds faster than it vacuums**:

```
BEGIN;

CREATE TABLE gold.fct_sales_line_new (LIKE gold.fct_sales_line);

INSERT INTO gold.fct_sales_line_new
SELECT * FROM gold.fct_sales_line
ORDER BY sale_date, store_sk;              -- load in sort-key order

ALTER TABLE gold.fct_sales_line     RENAME TO fct_sales_line_old;
ALTER TABLE gold.fct_sales_line_new RENAME TO fct_sales_line;

COMMIT;

ANALYZE gold.fct_sales_line;
DROP TABLE gold.fct_sales_line_old;        -- after you have verified
```

`LIKE` copies the distribution and sort keys. Two notes: `LIKE` does not carry `IDENTITY` behaviour, so a table with an identity column needs an explicit DDL; and keep the old table until you have checked the row count, then drop it — the space is not released until you do.

This is also how you **change a** `SORTKEY` on a large table, and how you clean up a table someone loaded with a million single-row inserts (L39).

L42 · VACUUM, ANALYZE and Auto Maintenance

Gotchas

- `VACUUM` **blocks writes on that table.** Not during the load window.
- **Only one** `VACUUM` **at a time, cluster-wide.**
- `DELETE` **without** `VACUUM` **grows the table forever.** A daily "delete then reload the slice" pattern needs `VACUUM DELETE ONLY` on a schedule.
- `TRUNCATE` **and** `DROP` **need no vacuum** — they release space immediately. Prefer `TRUNCATE` to `DELETE` when clearing a whole table (L23).
- **Auto maintenance needs idle time.** Verify with `svl_auto_worker_action`.
- **Temp tables have no stats until analyzed.**
- `ANALYZE` **on a table with no changes still costs something** — `svv_table_info.stats_off` tells you whether it is worth it.
- **A deep copy needs room for two copies** of the table.
- `vacuum_sort_benefit` **near zero means do not bother**, however ugly `unsorted` looks.
- **Sort order does not matter for a table nothing filters on.** Do not vacuum for tidiness.

Try it

1. Run the `svv_table_info` query on a real cluster. Sort by size. Write down every table with `stats_off > 10`.
2. Take the slowest of those, `EXPLAIN` a typical query, run `ANALYZE`, `EXPLAIN` again. Compare the estimated row counts in the plan — that difference *is* the bug.
3. `DELETE` a third of a test table's rows and watch `size` in `svv_table_info` not move. Then `VACUUM DELETE ONLY` and watch it drop.
4. Check `svl_auto_worker_action`. Decide whether your cluster is actually getting maintenance.
5. Deep-copy a table with a different `SORTKEY` and compare a filtered query before and after.

Exercise 2 is the one that teaches why stats matter.

Checklist

- ☐ `ANALYZE` at the end of every load procedure, on the predicate columns
- ☐ `ANALYZE` after every temp-table creation
- ☐ `svv_table_info` reviewed weekly for `unsorted` and `stats_off`
- ☐ `VACUUM` scheduled outside the load window, `TO 95 PERCENT`
- ☐ `VACUUM DELETE ONLY` scheduled if the load pattern deletes slices
- ☐ `TRUNCATE` used instead of `DELETE` for whole-table clears
- ☐ `svl_auto_worker_action` checked — I know whether auto maintenance runs here
- ☐ Deep copy used for big resorts and sort-key changes
- ☐ I check `vacuum_sort_benefit` before vacuuming anything large

You've got it when you can...

...be told a report got slower over three months, check two columns of `svv_table_info`, and fix it with an `ANALYZE` before the meeting ends.

MODULE 01 · LESSON 43

The System Tables You Actually Use

There are hundreds. You need about twelve. Learn these and the warehouse stops being opaque.

1 · WHAT IT IS

SYS_ views first, STL and SVV when you need detail

The SYS_ family is the modern monitoring interface and works on both provisioned and serverless. The older STL, STV, SVL and SVV views still hold detail SYS_ does not.

2 · HOW IT WORKS

FOUR YOU WILL OPEN EVERY WEEK

SYS_QUERY_HISTORY — `what ran, how long, who`
Queue time, execution time, cache hits, status. The first place you look for anything at all.

SVV_TABLE_INFO — `size, skew, unsorted, stats`
One row per table with every design number that matters. If you memorise one view, this is it.

STL_LOAD_ERRORS — `why COPY failed, per row`
The offending line, the column, the raw value and the reason. Never guess at a load failure again.

SVL_QUERY_SUMMARY — `is_diskbased, rows per step`
What actually happened, step by step, versus what EXPLAIN predicted. This is where spilling is confirmed.

```
save these as views in an ops schema. type them once.
```

3 · THE SQL

```
-- the daily driver
SELECT query_id, user_id,
queue_time/1e6 AS q,
execution_time/1e6 AS e,
LEFT(query_text, 60)
FROM sys_query_history
ORDER BY e DESC LIMIT 20;
```

4 · GOTCHAS

Times are microseconds. Divide by 1e6.
History is retained for days, not months — persist anything you want to trend.

STL views are leader-node only

IN PLAIN ENGLISH

The warehouse keeps a diary of everything it did and how long it took. Most people never open it, then wonder why the building is mysterious.

L43 · The System Tables You Actually Use

Slide: `_render/L43-system-tables.html`

What it is

There are hundreds of system views. **You need about twelve.** Learn these and the warehouse stops being opaque.

The families

PREFIX	WHAT IT IS	NOTE
`SYS_`	The modern monitoring interface	Works on **provisioned and serverless**. Start here.
`SVV_`	Views over system tables, current state	`svv_table_info` is the most valuable view in Redshift
`STL_`	Logged history, persisted to disk	Detailed, older, leader-node only
`STV_`	Live snapshots of transient state	Locks, in-flight queries
`SVL_`	Views joining logs for convenience	`svl_query_summary` is the one you need

Prefer `SYS_` when it has what you need, then drop to `SVV_` / `STL_` / `SVL_` for detail it does not expose. On Serverless, several `STL_` / `STV_` views are unavailable — another reason to reach for `SYS_` first.

The four you will open every week

1 · `sys_query_history` — what ran

```
SELECT query_id,
       user_id,
       queue_time     / 1e6 AS queue_secs,
       execution_time / 1e6 AS exec_secs,
       elapsed_time   / 1e6 AS total_secs,
       result_cache_hit,
       status,
       LEFT(query_text, 80) AS sql
FROM   sys_query_history
WHERE  start_time >= DATEADD('hour', -4, GETDATE())
ORDER  BY execution_time DESC
LIMIT  20;
```

The first place you look for anything at all. Queue time, execution time, cache hits, status, the SQL itself.

2 · `svv_table_info` — one row per table, every number that matters

```
SELECT "schema", "table",
       size                AS mb,
       tbl_rows,
       diststyle,
       sortkey1,
       skew_rows,          -- max/min rows per slice. >4 is bad
       unsorted,           -- % outside sort order
       stats_off,          -- % staleness
       vacuum_sort_benefit,
       max_varchar
FROM   svv_table_info
ORDER  BY size DESC;
```

If you memorise one view, this is it. It answers "is this table designed correctly", "is it maintained", and "is it skewed" in a single row.

3 · `stl_load_errors` — why `COPY` failed, per row

```
SELECT starttime, filename, line_number, colname, type, raw_field_value, err_reason
FROM   stl_load_errors
ORDER  BY starttime DESC
LIMIT  20;
```

The offending line, the column, the raw value, and the reason. **Never guess at a load failure again** (L22).

L43 · The System Tables You Actually Use

4 · `svl_query_summary` — what actually happened, step by step

```
SELECT stm, seg, step, label, rows, bytes, is_diskbased, workmem/1024/1024 AS workmem_mb
FROM   svl_query_summary
WHERE  query = 123456
ORDER  BY stm, seg, step;
```

`EXPLAIN` predicts; this reports. **`is_diskbased = 't'` on any step is your answer** (L28, L44).

The rest of the twelve

```
-- 5. how much am I actually paying for (serverless)
SELECT * FROM sys_serverless_usage ORDER BY end_time DESC LIMIT 24;

-- 6. did a load succeed, and how many files
SELECT query, filename, curtime, status FROM stl_load_commits
WHERE query = 123456 ORDER BY curtime;

-- 7. how selective was the scan (step 5 of L44)
SELECT slice, tbl, rows, rows_pre_filter, is_rrscan
FROM   stl_scan WHERE query = 123456 AND userid > 1;

-- 8. what is blocking my session, right now
SELECT * FROM stv_locks;
SELECT * FROM svv_transactions;

-- 9. column definitions, encodings, keys
SELECT * FROM pg_table_def WHERE schemaname = 'gold' AND tablename = 'fct_sales_line';
-- ⚠ needs the schema on the search_path to return anything. SVV_COLUMNS does not.

-- 10. materialized view freshness
SELECT * FROM svv_mv_info;

-- 11. is Redshift itself recommending a change
SELECT * FROM svv_alter_table_recommendations;

-- 12. is automatic maintenance actually running
SELECT * FROM svl_auto_worker_action ORDER BY eventtime DESC LIMIT 50;

-- and one more worth knowing: every schema, including external and shared
SELECT * FROM svv_all_schemas;
```

Number 11 is underused. Redshift's advisor writes its own suggestions there — sort key changes, distribution changes — based on your real workload.

Save them as views

Type these once, not every time:

```
CREATE SCHEMA IF NOT EXISTS ops;

CREATE OR REPLACE VIEW ops.v_slow_queries AS
SELECT query_id, user_id,
       queue_time/1e6 AS queue_secs, execution_time/1e6 AS exec_secs,
       result_cache_hit, status, LEFT(query_text, 100) AS sql
FROM   sys_query_history
WHERE  start_time >= DATEADD('day', -1, GETDATE())
  AND  execution_time > 10e6;

CREATE OR REPLACE VIEW ops.v_table_health AS
SELECT "schema", "table", size AS mb, tbl_rows, diststyle, sortkey1,
       skew_rows, unsorted, stats_off, vacuum_sort_benefit, max_varchar
FROM   svv_table_info;

CREATE OR REPLACE VIEW ops.v_needs_maintenance AS
SELECT * FROM ops.v_table_health
WHERE  stats_off > 10 OR (unsorted > 20 AND vacuum_sort_benefit > 20)
ORDER  BY mb DESC;

GRANT USAGE ON SCHEMA ops TO ROLE analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA ops TO ROLE analyst;
```

Build the `ops` schema on day one of a new cluster. It pays for itself the first time something is slow.

L43 · The System Tables You Actually Use

Gotchas

- **Times are microseconds.** Divide by `1e6`. Reporting a query as "40,000,000 seconds slow" is a rite of passage.
- **History is retained for days, not months.** If you want to trend performance, `UNLOAD` a daily snapshot to S3.
- **`STL_` and `STV_` are leader-node only** and some are unavailable on Serverless. `SYS_` is the portable choice.
- **`userid > 1` filters out Redshift's own internal queries** in the older views — without it, results are full of system noise.
- **A non-superuser sees only their own rows** in most views. Grant `SYSLOG ACCESS UNRESTRICTED` deliberately, to the people who need it.
- **`pg_table_def` needs the schema on the `search_path`** or it silently returns nothing. `svv_columns` does not have this problem and is the better choice in a script.
- **`query` vs `query_id`** — older views use `query`, `SYS_` views use `query_id`. Joining across families needs care.
- **`skew_rows` above 4** means your `DISTKEY` is on a low-cardinality or badly skewed column (L15).

Try it

1. Create the `ops` schema and all three views on a real cluster.
2. Run `ops.v_needs_maintenance`. Fix the worst row.
3. Find your slowest query in the last day and pull its `svl_query_summary`. Identify the slowest step.
4. Break a `COPY` deliberately and read `stl_load_errors` until the error makes sense without help.
5. Run `svv_alter_table_recommendations` and evaluate each suggestion — do not apply blindly, but understand why each was made.
6. Set up an `UNLOAD` of yesterday's `sys_query_history` to S3, so in three months you can answer "was this always slow?".

Exercise 6 is the one nobody does and everybody wishes they had.

Checklist

- ☐ `SYS_` first, older families for detail
- ☐ Times divided by `1e6`
- ☐ `ops` schema with saved views exists
- ☐ I can find: what ran, why it was slow, why a load failed, what is locked
- ☐ `svv_table_info` reviewed on a schedule
- ☐ Query history unloaded to S3 for long-term trending
- ☐ `svl_auto_worker_action` checked
- ☐ `userid > 1` in older-view queries

You've got it when you can…

…answer "why was the 6 a.m. report slow yesterday?" from the system views alone, three days later, without having been watching at the time.

MODULE 01 · LESSON 44 · ★ THE PLAYBOOK

Diagnosing A Slow Query

Six questions, in order. Answer them in sequence and you will find the cause almost every time without guessing once.

1 · WHAT IT IS

A fixed order, so you stop guessing

Almost everyone starts by rewriting the SQL. That is question six. The first five are cheaper, faster to check, and usually where the answer is.

2 · THE SIX QUESTIONS

ASK THEM IN THIS ORDER. STOP WHEN ONE ANSWERS YES.

1. **DID IT QUEUE, OR RUN SLOWLY?**
   queue_time vs execution_time in sys_query_history. A queuing problem is a WLM problem, not a SQL problem.
2. **ARE THE STATISTICS STALE?**
   stats_off in svv_table_info. ANALYZE is the cheapest fix there is, and it is the answer more often than anything else.
3. **IS ANYTHING BEING BROADCAST?**
   Search the EXPLAIN plan for DS_BCAST_INNER. Then make the small side DISTSTYLE ALL.
4. **IS IT SPILLING TO DISK?**
   is_diskbased in svl_query_summary. Caused by broadcast, oversized VARCHARs, or too many WLM slots.
5. **IS IT SCANNING MORE THAN IT NEEDS?**
   rows vs rows_pre_filter in stl_scan. A bad ratio means the zone maps are not pruning.

```
6 · only now rewrite the SQL. it is rarely the SQL.
```

3 · THE SQL

```
-- step 4: did any step spill?
SELECT step, label, rows,
is_diskbased, workmem
FROM svl_query_summary
WHERE query = 123456
ORDER BY stm, seg, step;

-- any 't' here is your answer
```

4 · GOTCHAS

The first run includes compilation. Always time the second, with the cache off.
Change one thing at a time, and re-measure.

confirm with EXPLAIN, not a stopwatch

IN PLAIN ENGLISH

A mechanic checks the fuel before rebuilding the engine. Five cheap checks come before the expensive one, and four times in five you never reach it.

L44 · Diagnosing A Slow Query ★

Slide: `_render/L44-diagnosing-slow-query.html`

What it is

Six questions, in order. Answer them in sequence and you will find the cause almost every time without guessing once.

Almost everyone starts by rewriting the SQL. **That is question six.** The first five are cheaper, faster to check, and usually where the answer is.

Print this page. Put it on the wall.

Step 0 · Get the query_id and set up an honest measurement

```
-- find it
SELECT query_id, user_id, start_time,
       queue_time/1e6 AS q, execution_time/1e6 AS e, status,
       LEFT(query_text, 120) AS sql
FROM   sys_query_history
WHERE  start_time >= DATEADD('hour', -6, GETDATE())
ORDER  BY execution_time DESC
LIMIT  20;

-- then, before you time anything (L34)
SET enable_result_cache_for_session TO off;
```

Discard the first run — that is compilation. Compare the median of runs 2 and 3.

Question 1 · Did it queue, or did it run slowly?

```
SELECT query_id, queue_time/1e6 AS queue_secs, execution_time/1e6 AS exec_secs, service_class
FROM   sys_query_history WHERE query_id = 123456;
```

RESULT	MEANING	GO TO
High queue, low exec	**A concurrency problem**	L41 — WLM, priorities, concurrency scaling. Stop here.
Low queue, high exec	A query problem	Question 2
Both high	Two problems	Fix the query first; it shortens the queue

A queuing problem is not a SQL problem. Rewriting the query will not help, and this is the most commonly misdiagnosed case in Redshift.

Question 2 · Are the statistics stale?

```
SELECT "schema", "table", size AS mb, tbl_rows, stats_off
FROM   svv_table_info
WHERE  "table" IN ('fct_sales_line', 'dim_store', 'dim_product')
ORDER  BY stats_off DESC;
```

`stats_off > 10` → `ANALYZE` **and re-measure.** This is the cheapest fix that exists and it is the answer more often than anything else — especially for "it was fine last week".

```
ANALYZE gold.fct_sales_line (sale_date, store_sk);
```

The evidence that this was the problem: run `EXPLAIN` before and after and compare the **estimated row counts**. If the estimate was out by orders of magnitude, the planner was choosing badly on bad information.

L44 · Diagnosing A Slow Query ★

Question 3 · Is anything being broadcast?

```
EXPLAIN <the query>;
```

Search the plan for `DS_BCAST_INNER`. That is the whole table being copied to every node (L28).

```
XN Hash Join DS_BCAST_INNER  (cost=...)     ← found it
```

The fix, in order (L29):

1. `ANALYZE` — already done in question 2
2. `ALTER TABLE gold.dim_store ALTER DISTSTYLE ALL;` — if it is a small dimension
3. Align the `DISTKEY` s of two large tables on the join column
4. Check the join key types match exactly — a cast silently defeats co-location

Step 2 solves it most of the time and takes one statement.

Question 4 · Is it spilling to disk?

```
SELECT stm, seg, step, label, rows, bytes,
       is_diskbased, workmem/1024/1024 AS workmem_mb
FROM   svl_query_summary
WHERE  query = 123456
ORDER  BY stm, seg, step;
```

Any `is_diskbased = 't'` is your answer. Memory ran out and the step went to disk, which is orders of magnitude slower.

Three causes, in order of likelihood:

1. **A broadcast** blew up the row count — go back to question 3
2. **Oversized `VARCHAR` s.** Redshift allocates by declared width, not actual content. A `VARCHAR(65535)` holding 20 characters still reserves 64 KB per row in memory (L09):

   ```
   SELECT "table", max_varchar FROM svv_table_info WHERE max_varchar > 1000 ORDER BY 2 DESC;
   ```

3. **Too many WLM slots** — each query got a smaller memory share (L41)

Also visible in query history:

```
SELECT query_id, LEFT(query_text,60) FROM sys_query_history
WHERE  start_time >= DATEADD('day',-1,GETDATE())
ORDER  BY execution_time DESC LIMIT 10;
-- then check svl_query_summary for each
```

Question 5 · Is it scanning more than it needs?

```
SELECT slice, tbl, rows, rows_pre_filter,
       ROUND(100.0 * rows / NULLIF(rows_pre_filter,0), 1) AS pct_kept,
       is_rrscan
FROM   stl_scan
WHERE  query = 123456 AND userid > 1
ORDER  BY rows_pre_filter DESC;
```

`rows_pre_filter` is how many rows were read; `rows` is how many survived. A ratio of 1% means you read 100× more than you used — the zone maps are not pruning (L16).

Causes:

L44 · Diagnosing A Slow Query ★

- **A function wraps the sort column** in the `WHERE` — `DATE_TRUNC('month', sale_date) = ...` (L32). Rewrite as a bare-column range.
- **The `SORTKEY` is on the wrong column** for how the table is actually queried.
- **The table is badly unsorted** — check `unsorted` and `vacuum_sort_benefit` (L42).

Also check **skew** while you are here:

```
SELECT "table", skew_rows FROM svv_table_info WHERE skew_rows > 4 ORDER BY 2 DESC;
```

`skew_rows > 4` means one slice holds four times the rows of another, so one node does most of the work while the rest wait. A bad `DISTKEY` (L15).

Question 6 · Only now, rewrite the SQL

If questions 1–5 all came back clean, the query itself is the problem. In rough order of payoff:

- **Aggregate before joining.** Join two million-row summaries, not two billion-row facts.
- **Filter earlier**, inside the CTE that scans the table (L31).
- **Remove correlated subqueries** — they become per-row loops (L31).
- **Replace self-joins with window functions** (L30).
- **`NOT EXISTS` instead of `NOT IN`** (L31).
- **Select only the columns you need.** `SELECT *` on a columnar store reads every column (L01).
- **Consider a materialized view** if this is a repeated dashboard query (L11).

Confirm with `EXPLAIN`, not a stopwatch. A faster time with an identical plan is noise or cache.

The whole playbook as one script

```
-- ============ REDSHIFT SLOW QUERY PLAYBOOK · substitute your query_id ============
\set qid 123456

-- 1. queue or execution?
SELECT query_id, queue_time/1e6 q_secs, execution_time/1e6 e_secs, service_class, status
FROM   sys_query_history WHERE query_id = :qid;

-- 2. stale stats?
SELECT "schema","table",size mb,tbl_rows,stats_off,unsorted,skew_rows,max_varchar,
       vacuum_sort_benefit
FROM   svv_table_info
WHERE  stats_off > 5 OR unsorted > 10 OR skew_rows > 4
ORDER  BY size DESC;

-- 3. broadcast?  -> run EXPLAIN and grep for DS_BCAST_INNER

-- 4. spill?
SELECT stm,seg,step,label,rows,is_diskbased,workmem/1024/1024 wm_mb
FROM   svl_query_summary WHERE query = :qid ORDER BY stm,seg,step;

-- 5. scan efficiency?
SELECT tbl,SUM(rows) kept,SUM(rows_pre_filter) read,
       ROUND(100.0*SUM(rows)/NULLIF(SUM(rows_pre_filter),0),1) pct
FROM   stl_scan WHERE query = :qid AND userid > 1 GROUP BY tbl;

-- 6. only now, the SQL.
```

Save it as `ops/slow-query.sql` in the repo.

The four causes, ranked by how often they are the answer

From experience across real clusters, roughly:

L44 · Diagnosing A Slow Query ★

1. **Stale statistics** — `ANALYZE`
2. **A broadcast join** — `DISTSTYLE ALL` on the small side
3. **Disk spill from oversized `VARCHAR`s** — fix the DDL
4. **A non-sargable date predicate** — rewrite as a bare-column range

All four are fixed without touching the query's logic. That is why question 6 is last.

Gotchas

- **The first run includes compilation.** Time the second, with the result cache off.
- **Change one thing at a time and re-measure.** Two changes at once tells you nothing.
- **Confirm with `EXPLAIN`.** A better time with the same plan is not a fix.
- `userid > 1` in `stl_scan` or you will drown in system rows.
- **Times are microseconds.**
- **The plan can change under you** — an `ANALYZE` between two `EXPLAIN`s is a variable, so note when you ran it.
- **A query that is slow only in production** is usually a data-volume or concurrency difference, not a SQL difference.
- **Do not tune around a broken design.** If a fact table is `DISTSTYLE EVEN` and joined constantly on `store_sk`, no amount of query tuning fixes it — rebuild it (L42).

Try it

Do this as a group exercise, on a real slow query:

1. One person drives, everyone else watches the six questions in order.
2. Write down the answer to each question before moving on.
3. Whoever suggests rewriting the SQL before question 6 buys the coffee.
4. When you find the cause, fix it, re-measure with the cache off, and confirm with `EXPLAIN`.
5. Write the diagnosis in a one-paragraph note in the repo — what was slow, what the cause was, what fixed it.

That last step builds the institutional memory that stops the same problem recurring.

Checklist

- ☐ I have the six questions memorised, in order
- ☐ Result cache off before any measurement
- ☐ First run discarded
- ☐ Question 1 answered before touching the SQL
- ☐ `ops/slow-query.sql` saved in the repo
- ☐ One change at a time, re-measured each time
- ☐ Every fix confirmed with `EXPLAIN`
- ☐ Diagnoses written down where the team can find them

You've got it when you can...

...be handed a query that got slow, work the six questions out loud in front of the team, and reach the cause without once saying "let me try changing this and see".

MODULE 01 · LESSON 45 · THE MASTERY MAP

Cost, Sizing and What To Remember

Forty-four lessons compress to about ten decisions. If you only carry ten things out of this module, carry these.

1 · WHAT DRIVES THE BILL

Serverless bills RPU-seconds. Provisioned bills nodes by the hour.

A spiky or intermittent workload is cheaper on serverless. A cluster running hard sixteen hours a day is cheaper provisioned, and cheaper still reserved.

2 · THE TEN THINGS

1. It is columnar and MPP. Never SELECT *, never row-at-a-time.
2. There are no indexes. DISTKEY and SORTKEY are the design.
3. Size every VARCHAR honestly. Oversizing causes disk spill.
4. Constraints are hints. Uniqueness is a test you write and run.
5. Load with COPY from S3. Never a loop of INSERTs.
6. Every load is idempotent for its batch date, or it is not finished.
7. Hunt DS_BCAST_INNER. Small dimensions go DISTSTYLE ALL.
8. ANALYZE after every load. It is the cheapest fix that exists.
9. Store UTC. Convert once, at the reporting boundary.
10. Diagnose in order. Rewriting the SQL is the last step, not the first.

3 · WHERE THE MONEY GOES

```
1 compute — RPUs or node hours
2 managed storage per TB-month
3 concurrency scaling beyond credit
4 Spectrum per TB scanned
5 cross-region snapshot transfer

-- 1 and 2 are almost the whole bill
```

4 · THE FIVE HABITS THAT SAVE MONEY

Set a serverless max-RPU ceiling and an alarm.
Give the workgroup a query timeout.
Materialize what every dashboard recomputes.
Reserve capacity once the pattern is known.
Delete the tables nobody has queried in a year.

WHERE TO GO NEXT

Module 00 for the lakehouse picture · Module 02 for the Tamimi build · Module 03 for the architectures. Then build something and break it in dev.

L45 · Cost, Sizing and The Mastery Map

Slide: `_render/L45-cost-sizing-mastery.html`

Part 1 · What drives the bill

Serverless vs provisioned

Serverless bills **RPU-seconds** — you set a base and a max RPU, and pay for what runs. There is a per-query minimum charge, and the workgroup auto-pauses when idle.

Provisioned bills **nodes by the hour**, whether or not anything is running. Reserved instances cut that substantially for a one- or three-year commitment.

WORKLOAD	CHEAPER ON
Spiky, intermittent, a few hours a day	**Serverless**
Dev and test environments	**Serverless** — they auto-pause overnight
Running hard 16+ hours a day	**Provisioned**, and cheaper still reserved
Unpredictable, new project	**Serverless** — until you know the pattern

Start serverless. Move to provisioned reserved once you have three months of usage data showing a steady baseline. Doing it the other way round means committing to a shape you have not measured.

Where the money actually goes

1. **Compute** — RPU-seconds or node hours
2. **Managed storage** — per TB-month, billed separately from compute (L04)
3. **Concurrency scaling** beyond the accrued free credit (L41)
4. **Spectrum** — per TB scanned in S3 (L12)
5. **Cross-region snapshot transfer**

Items 1 and 2 are almost the whole bill. Do not spend time optimising 3–5 until you have looked at 1 and 2.

Watch it

```
-- serverless: RPU-hours by period
SELECT DATE_TRUNC('day', end_time) AS d,
       SUM(charged_seconds) / 3600.0 AS rpu_hours
FROM   sys_serverless_usage
GROUP  BY 1 ORDER BY 1 DESC LIMIT 30;

-- which queries consumed the most compute yesterday
SELECT user_id, COUNT(*) AS queries,
       SUM(execution_time)/1e6/3600 AS exec_hours
FROM   sys_query_history
WHERE  start_time >= DATEADD('day', -1, GETDATE())
GROUP  BY 1 ORDER BY 3 DESC;

-- storage by table — the second half of the bill
SELECT "schema", "table", size AS mb, tbl_rows
FROM   svv_table_info ORDER BY size DESC LIMIT 30;
```

That second query is the useful one politically as well as technically: it tells you *whose* workload is driving the compute bill.

Part 2 · Sizing

Serverless: start at **8 RPUs base**, set a **max** (this is your cost ceiling), and watch queue time. Raise the base if queries queue; raise the max if peaks are throttled.

Provisioned: `ra3.xlplus` for small, `ra3.4xlarge` mid, `ra3.16xlarge` large. Because RA3 separates compute from storage (L04), **size for compute and concurrency, not for data volume** — a common and expensive mistake carried over from DC2-era thinking.

Two nodes minimum for anything production — a single-node cluster has no redundancy.

L45 · Cost, Sizing and The Mastery Map

Five habits that save real money

1. **Set a serverless max-RPU ceiling and a CloudWatch billing alarm.** Without a ceiling, one runaway query can scale up hard.
2. **Give the workgroup a query timeout** (`max_query_execution_time`). An unbounded query is an unbounded bill.
3. **Materialize what every dashboard recomputes** (L11). Computing the same aggregate 400 times a day is 400× the cost of computing it once.
4. **Reserve capacity once the pattern is known** — not before.
5. **Delete the tables nobody has queried in a year.** Storage is billed per TB-month forever, and every warehouse accumulates abandoned tables.

For habit 5:

```
-- tables nobody has scanned recently (widen the window as your history allows)
SELECT t."schema", t."table", t.size AS mb
FROM   svv_table_info t
WHERE  t.size > 1000
  AND  NOT EXISTS (
         SELECT 1 FROM sys_query_history h
         WHERE  h.query_text ILIKE '%' || t."table" || '%'
           AND  h.start_time >= DATEADD('day', -30, GETDATE()))
ORDER  BY t.size DESC;
```

⚠️ That is a heuristic, not proof — a table referenced only inside a view or procedure will not appear in query text. Confirm before dropping anything, and snapshot first.

Part 3 · The Mastery Map — the ten things

Forty-four lessons compress to about ten decisions. **If you only carry ten things out of this module, carry these.**

1 · It is columnar and MPP. Never `SELECT *` , never row-at-a-time. Reading one column of a billion rows is cheap; reading every column is not. → L01, L02

2 · There are no indexes. `DISTKEY` and `SORTKEY` are the entire physical design, and you choose them from how the table is queried. → L14, L15, L16, L20

3 · Size every `VARCHAR` honestly. Redshift allocates memory by declared width. Oversizing causes disk spill, which is the most common silent performance bug in the product. → L09

4 · Constraints are hints. Redshift does not enforce `PRIMARY KEY` or `UNIQUE` . Uniqueness is a test you write and run on every load. → L18

5 · Load with `COPY` from S3. Never a loop of `INSERT` s — that damages the cluster, not just the job. → L21, L39

6 · Every load is idempotent for its batch date, or it is not finished. Delete the slice and reload it, or `MERGE` . → L24, L26, L35

7 · Hunt `DS_BCAST_INNER` . Small dimensions go `DISTSTYLE ALL` . This one fix accounts for a large share of all Redshift speedups. → L28, L29

8 · `ANALYZE` after every load. It is the cheapest fix that exists and the answer to most "it was fine last week" reports. → L42

9 · Store UTC. Convert once, at the reporting boundary. Precompute the local business day at load time and make it the `SORTKEY` . → L32

10 · Diagnose in order. Queue → stats → broadcast → spill → scan → *then* the SQL. Rewriting the SQL is the last step, not the first. → L44

Part 4 · The 90-day plan

Days 1–14 — read and query. Build the `ops` schema (L43). Run every diagnostic query in this module against a real cluster. Get comfortable reading `svv_table_info` .

L45 · Cost, Sizing and The Mastery Map

Days 15–30 — build one fact and two dimensions. Choose the keys deliberately and write down *why* for each. Load them with `COPY` . Write the uniqueness tests. Make the load idempotent and prove it by running it three times.

Days 31–60 — make it a pipeline. Move the logic into stored procedures. Call them from Node via the Data API. Orchestrate with Step Functions. Add the run log and all three alerts. Break it deliberately and fix it.

Days 61–90 — operate it. Run the six-question playbook on real slow queries. Set up the maintenance schedule. Watch the cost. Then go back to your day-15 key choices and see which ones you would now make differently.

That last exercise is the one that turns knowledge into judgement.

Where to go next

MODULE	WHAT IT GIVES YOU
Module 00 · Basics	Warehouse vs lake vs lakehouse, the whole AWS data landscape, who can read and write what
Module 02 · Foundation	The medallion architecture and the Tamimi build in detail
Module 03 · Foundation Architectures	35 architecture diagrams — the as-built and the target state

Then build something and break it in dev. Nothing in this module substitutes for having watched `DS_BCAST_INNER` disappear from your own plan.

Checklist

- ☐ I know whether we are serverless or provisioned, and why
- ☐ A max-RPU ceiling and a billing alarm exist
- ☐ The workgroup has a query timeout
- ☐ I can name the two line items that are most of the bill
- ☐ I have run the compute-by-user query and know who drives cost
- ☐ Repeated dashboard aggregates are materialized
- ☐ I can recite the ten things without looking
- ☐ I have a 90-day plan and a date in the calendar to revisit my key choices

You've got it when you can…

…sit in a design review, be shown a proposed fact table, and ask the right four questions about it — distribution, sort, `VARCHAR` widths, and how the load reruns — before anyone has written a line of SQL.

End of Module 01. 45 lessons across seven parts: the mental model, the objects, physical design, loading, querying, code, and operations.

Author: Surender Sara · Northbay Solutions